

Khóa học C# Advance: Mastery of C#

1. Advanced Collections and LINQ Extensions	12
1.1. Working with advanced LINQ queries.	12
Mục tiêu	12
1. Giới thiệu LINQ nâng cao	12
2. Phép nối (Joins)	12
2.1. Nội dung	12
2.2. Ví dụ minh họa	12
2.3. Left Join	13
3. Phân vùng (Partitioning)	13
3.1. Toán tử quan trọng	13
3.2. Ví dụ minh họa	13
4. Phân nhóm (Grouping)	14
4.1. Toán tử GroupBy	14
4.2. Ví dụ minh họa GroupBy với 1 key	15
4.3. Ví dụ GroupBy với nhiều Key	15
4.4. Group By với điều kiện Having	16
4.5. Kết hợp Join, GroupBy, và Having	18
5. Lazy Evaluation	22
5.1. Khái niệm	22
5.2. Ví dụ minh họa	22
6. Thực hành	22
7. Tổng kết	22
1.2. Understanding and implementing custom collections.	23
Mục tiêu bài học:	23
1. Tại sao cần Custom Collections?	23
2. Các bước cơ bản để tạo Custom Collection	23
2.1. Sử dụng giao diện IEnumerable<T>	23
2.2. Sử dụng giao diện ICollection<T>	24

2.3. Sử dụng giao diện <code>ICollection<T></code>	24
3. Các lưu ý khi triển khai Custom Collection	25
4. Bài tập thực hành	25
5. Kết luận	26
1.3. Practical use cases and performance considerations.	26
I. Mục tiêu bài học	26
II. Nội dung chi tiết	26
1. Practical Use Cases của C#	26
2. Performance Considerations (Hiệu suất)	27
III. Bài tập thực hành	28
IV. Tóm tắt	28
2. Reflection and Dynamic Types	28
2.1. Using reflection to inspect and modify objects at runtime.	28
1. Khái niệm cơ bản về Reflection	28
2. Ứng dụng Reflection	29
2.1 Kiểm tra thông tin kiểu	29
2.2 Tạo đối tượng và gọi phương thức tại runtime	30
2.3 Thay đổi trường hoặc thuộc tính không công khai (private)	30
3. Lưu ý khi sử dụng Reflection	31
4. Bài tập thực hành	31
2.2. Introduction to dynamic types and the dynamic keyword.	32
1. dynamic là gì?	32
2. Đặc điểm của dynamic	32
3. Sử dụng dynamic trong thực tế	32
3.1 Khai báo biến dynamic	32
3.2 Gọi phương thức với dynamic	32
3.3 Truy cập đối tượng COM hoặc dữ liệu JSON	33
4. Lợi ích của dynamic	33
5. Rủi ro khi sử dụng dynamic	33
6. So sánh dynamic và object	33
7. Khi nào nên sử dụng dynamic?	34
8. Ví dụ thực tế	34

Tóm lại	34
2.3. Practical applications and use cases for reflection.	34
1. Tìm hiểu Reflection trong thực tế	35
2. Các trường hợp sử dụng Reflection	35
2.1. Dependency Injection (DI)	35
2.2. Thực hiện Mapping từ dữ liệu	35
2.3. Kiểm tra và xác thực (Validation)	36
2.4. Tạo Unit Test tự động	37
2.5. Phân tích và xử lý dữ liệu động (Dynamic Data Handling)	37
3. Lưu ý khi sử dụng Reflection	37
4. Thực hành	38
3. Memory Management and Garbage Collection	38
3.1. How memory management works in .NET.	38
1. Giới thiệu về Bộ nhớ trong .NET	38
2. Quản lý bộ nhớ trong .NET	38
3. Cách thức hoạt động của Garbage Collector	38
4. Các kỹ thuật tối ưu hóa quản lý bộ nhớ	39
5. Quản lý bộ nhớ trong các trường hợp đặc biệt	39
6. Best Practices	39
7. Công cụ hỗ trợ phân tích bộ nhớ	39
8. Tóm tắt	40
9. Ví dụ về Garbage Collection (GC)	40
10. Sử dụng Dispose và using để quản lý tài nguyên	41
11. Sử dụng WeakReference để tránh giữ đối tượng trong bộ nhớ	41
12. Tối ưu hóa với Object Pooling	42
13. Memory Profiling và Debugging	44
13. Tóm tắt	44
3.2. Garbage collection and optimization tips.	44
1. Giới thiệu về Garbage Collection (GC)	44
2. Các thể hệ trong Garbage Collection	44
3. Các loại thu gom garbage collection	44
4. Làm thế nào để tối ưu hóa Garbage Collection	45
5. Tips tối ưu hóa khác	45

6. Lập trình dựa trên cơ chế GC	45
7. Công cụ hỗ trợ GC trong C#	45
8. Thực hành và ví dụ	46
9. Kết luận	46
10. Giảm thiểu số lượng đối tượng ngắn hạn	46
11. Sử dụng cấu trúc dữ liệu hiệu quả (Memory Pooling)	46
12. Giảm thiểu việc sử dụng các đối tượng có kích thước lớn	47
13. Giảm độ phân mảnh của heap	47
14. Sử dụng Dispose đúng cách	47
15. Sử dụng WeakReference	48
16. Tránh việc cấp phát bộ nhớ quá thường xuyên	48
17. Tối ưu hóa với MemoryManagement API (GC.Collect)	48
18. Công cụ hỗ trợ GC	49
19. Lập trình dựa trên cơ chế GC	49
20. Kết luận	49
3.3. Using IDisposable and the using statement.	49
1. IDisposable Interface	49
2. Sử Dụng Dispose()	50
3. Câu Lệnh using	50
4. Ví Dụ Cụ Thể	50
5. Kết Hợp với try-catch-finally	51
6. Ưu Điểm của IDisposable và using	51
7. Kết Luận	52
4. Multithreading and Parallel Programming	52
4.1. Basics of multithreading in C#.	52
1. Khái niệm cơ bản về Multithreading	52
2. Thread trong C#	52
3. Tạo và chạy một Thread	52
4. Quản lý luồng bằng Task và async/await	53
5. Chạy nhiều tác vụ đồng thời	54
6. Điều phối luồng với lock	54
7. Tóm tắt	55

4.2. Parallel programming and the Task Parallel Library (TPL).	55
1. Giới thiệu về Lập trình Song Song	55
2. Giới thiệu về Task Parallel Library (TPL)	56
3. Các lớp chính trong TPL	56
4. Quản lý Song song với Task .WhenAll và Task .WhenAny	56
5. Quản lý Ngoại lệ trong TPL	57
6. Lợi ích và Thách thức khi sử dụng TPL	57
7. Kết luận	58
4.3. Thread safety and synchronization.	58
1. Thread Safety (Thread-Safety) là gì?	58
2. Synchronization (Đồng Bộ Hóa)	58
3. Các Cơ Chế Synchronization trong C#	58
4. Race Condition và Deadlock	59
5. Ví Dụ Thực Tế	60
6. Lời khuyên khi làm việc với Thread Safety và Synchronization	60
7. Tóm tắt	60
5. Design Patterns in C#	60
5.1. Common design patterns (Singleton, Factory, Observer, etc.).	60
Mục tiêu bài học	60
Nội dung bài học	61
I. Giới thiệu về Design Patterns	61
II. Các Design Patterns phổ biến	61
1. Singleton Pattern	61
2. Factory Pattern	62
3. Observer Pattern	62
III. So sánh và áp dụng	63
IV. Bài tập thực hành	64
V. Kết luận	64
5.2. Practical applications and benefits.	64
Mục tiêu bài học	64
Nội dung chính	64
Tổng kết	66
Bài tập	66

5.3. Choosing the right pattern for different scenarios.	67
Mục tiêu bài học:	67
Nội dung chi tiết	67
1. Tổng quan về Design Patterns	67
2. Phân tích yêu cầu và xác định mẫu phù hợp	67
3. Ưu điểm và nhược điểm của các mẫu thiết kế	68
4. Thực hành áp dụng Design Pattern	68
5. Kết luận	68
5.4. Nguyên lý SOLID	68
1. Single Responsibility Principle (SRP) - Nguyên lý đơn trách nhiệm	69
2. Open/Closed Principle (OCP) - Nguyên lý mở/đóng	69
3. Liskov Substitution Principle (LSP) - Nguyên lý thay thế Liskov	70
4. Interface Segregation Principle (ISP) - Nguyên lý phân tách Interface	71
5. Dependency Inversion Principle (DIP) - Nguyên lý đảo ngược phụ thuộc	72
Kết luận:	73
6. Unit Testing and Test-Driven Development (TDD)	73
6.1. Introduction to unit testing frameworks (e.g., MSTest, NUnit, xUnit).	73
1. Kiểm thử đơn vị là gì?	73
2. Tổng quan về các framework kiểm thử đơn vị	74
3. So sánh MSTest, NUnit và xUnit	74
4. Kiến trúc cơ bản của một bài kiểm thử đơn vị	74
5. Cài đặt và cấu hình	75
6. Demo cơ bản với xUnit	75
7. Kết luận	76
Bài tập	76
6.2. Writing and running tests.	76
Mục tiêu	76
1. Giới thiệu về Unit Test	77
2. Cài đặt xUnit	77
2.1. Thêm xUnit vào dự án	77
2.2. Cấu trúc thư mục	77
3. Viết Unit Test	77
3.1. Một ví dụ đơn giản	77

3.2. Tạo nhiều trường hợp kiểm thử	78
4. Chạy Unit Test	79
4.1. Dùng Test Explorer	79
4.2. Dùng lệnh dotnet test	79
5. Phương pháp hay khi viết Unit Test	79
6. Nâng cao	79
6.1. Mock và Stub	79
6.2. Test Asynchronous Methods	79
Bài tập thực hành	80
6.3. Fundamentals of TDD in C# projects.	80
1. Giới thiệu về Test-Driven Development (TDD)	80
2. Chu trình TDD: Red -> Green -> Refactor	80
3. Cấu trúc một bài test trong C#: AAA	81
4. Các công cụ TDD phổ biến trong C#	81
5. Áp dụng TDD trong dự án C#	81
6. Lưu ý quan trọng khi áp dụng TDD	82
7. Bài tập thực hành	82
8. Kết luận	82
7. Advanced .NET Features	82
7.1. Dependency Injection (DI) and Inversion of Control (IoC).	82
1. Mục tiêu bài học	82
2. Nội dung bài học	83
2.1. Dependency Injection là gì?	83
2.2. Inversion of Control (IoC)	83
2.3. DI Containers	83
3. Các loại Dependency Injection	84
4. DI trong ASP.NET Core	85
5. Bài tập thực hành	85
6. Kết luận	86
7.2. Using attributes and annotations.	86
1. Tổng quan về Attributes trong C#	86
2. Các Attribute sẵn có trong C#	86
3. Annotations trong Entity Framework	87

4. Tạo Custom Attribute	87
5. Sử dụng Reflection để đọc Attributes	88
6. Thực hành	88
7. Kết luận	88
7.3. Working with Expression Trees.	88
1. Giới thiệu Expression Trees	88
2. Lợi ích của Expression Trees	89
3. Cấu trúc Expression Trees	89
4. Tạo một Expression Tree	89
5. Phân tích Expression Tree	90
6. Ứng dụng thực tế	90
7. Bài tập thực hành	91
8. Tổng kết	91
8. Performance Optimization and Best Practices	91
8.1. Profiling and optimizing C# code.	91
Mục tiêu	91
Nội dung chi tiết	91
1. Giới thiệu về Profiling	91
2. Các công cụ Profiling phổ biến	92
3. Các kỹ thuật tối ưu hóa mã C#	92
4. Thực hành: Profiling và tối ưu hóa mã	92
5. Tổng kết	93
8.2. Tips for memory and performance optimization.	93
1. Hiểu về bộ nhớ trong C#	93
2. Quản lý bộ nhớ và tài nguyên	93
3. Tối ưu hóa bộ nhớ	93
4. Tối ưu hóa hiệu suất	94
5. Sử dụng các công cụ phân tích hiệu suất	94
6. Thực hành và ví dụ	94
7. Kết luận	94
8.3. 10 thủ thuật tối ưu Code trong C#	94
1. Sử dụng StringBuilder thay cho việc nối chuỗi (+)	94
2. Sử dụng readonly khi có thể	95

3. Tránh sử dụng LINQ khi hiệu năng là ưu tiên	96
4. Sử dụng ArrayPool<T> thay cho việc cấp phát mảng liên tục	96
5. Sử dụng Span<T> để xử lý dữ liệu không cần cấp phát mới	98
6. Sử dụng ValueTask thay cho Task trong trường hợp đơn giản	99
7. Sử dụng Dictionary thay vì List khi tìm kiếm	99
8. Sử dụng ConfigureAwait(false) trong các ứng dụng không phải giao diện	99
9. Giảm kích thước struct	100
10. Sử dụng Parallel.For để xử lý song song	100
8.4. Best practices for clean, maintainable code.	101
1. Tuân thủ nguyên tắc SOLID	101
2. Tên biến và phương thức rõ ràng, dễ hiểu	101
3. Tách biệt logic và UI	101
4. Không viết mã lặp lại (DRY - Don't Repeat Yourself)	101
5. Sử dụng Dependency Injection (DI)	102
6. Viết mã dễ kiểm thử (Testable Code)	102
7. Thực hiện xử lý lỗi hợp lý	102
8. Giữ mã nguồn ngắn gọn và rõ ràng	102
9. Sử dụng các công cụ phân tích mã	102
10. Tuân thủ các chuẩn và hướng dẫn lập trình	102
11. Refactor định kỳ	102
12. Sử dụng các mô hình thiết kế phần mềm	103
8.5. Clean Code trong C# (có so sánh với mã bản)	103
1. Đặt tên biến, phương thức rõ ràng	103
2. Xử lý lỗi rõ ràng	103
3. Tránh lặp lại mã (DRY - Don't Repeat Yourself)	104
4. Sử dụng cấu trúc điều kiện gọn gàng	105
5. Sử dụng Dependency Injection (DI)	105
6. Sử dụng LINQ thay cho vòng lặp phức tạp	106
7. Viết hàm đơn nhiệm	107
Kết luận	108
9. Security in C# Development	108
9.1. Best Practices for Secure Coding in C#.	108

1. Giới thiệu về bảo mật trong lập trình	108
2. Input Validation and Sanitization	108
3. Sử dụng Prepared Statements và Parameterized Queries	109
4. Sử dụng Hashing và Salt cho Mật khẩu	109
5. Sử dụng HTTPS thay vì HTTP	109
6. Tránh việc lưu trữ thông tin nhạy cảm một cách không an toàn	110
7. Xử lý lỗi một cách an toàn	110
8. Quản lý quyền truy cập (Authorization & Authentication)	110
9. Code Review và Static Code Analysis	111
10. Security Testing và Penetration Testing	111
11. Tổng kết	111
9.2. Implementing Cryptography in C#.	111
Mục tiêu bài học:	111
Nội dung bài giảng	111
1. Giới thiệu về Cryptography	111
2. Các lớp Cryptography trong .NET	112
3. Mã hóa đối xứng (Symmetric Encryption) với AES	112
4. Mã hóa bất đối xứng (Asymmetric Encryption) với RSA	113
5. Hashing và HMAC	114
6. Quản lý khóa và bảo mật trong Cryptography	114
7. Tổng kết và thực hành	114
Kết luận	115
9.3. Secure Authentication and Authorization.	115
1. Giới thiệu về Authentication và Authorization	115
2. Các phương pháp Authentication phổ biến trong C#	115
3. Xây dựng Secure Authentication trong C#	115
4. JWT Authentication	115
5. Sử dụng OAuth 2.0 trong C#	116
6. Authorization trong C#	116
7. Best Practices để Bảo vệ Authentication và Authorization	116
8. Thực hành	116
9. Kết luận	116
10. Interoperability and Integration	117

10.1. Consuming and Building RESTful APIs.	117
Mục tiêu bài giảng	117
1. Giới thiệu về RESTful API	117
2. Các phương thức HTTP và ứng dụng trong RESTful API	117
3. Tiêu thụ RESTful API trong C#	117
4. Xử lý dữ liệu JSON trong C#	118
5. Xây dựng RESTful API với ASP.NET Core	119
6. Xử lý lỗi và trả về mã trạng thái HTTP	120
7. Kết luận	120
10.2. Working with gRPC and Message Queues.	121
Mục tiêu học:	121
I. Giới thiệu về gRPC	121
II. Giới thiệu về Message Queues	122
III. Kết hợp gRPC và Message Queues	123
IV. Bài tập thực hành	123
V. Kết luận	124

1. Advanced Collections and LINQ Extensions

1.1. Working with advanced LINQ queries.

Mục tiêu

- Hiểu rõ cách sử dụng các phép nối (joins) trong LINQ.
 - Biết cách phân vùng dữ liệu với **Skip**, **Take**, và các toán tử liên quan.
 - Nắm vững kỹ thuật phân nhóm (grouping) dữ liệu.
 - Hiểu về **lazy evaluation** trong LINQ và tầm quan trọng của nó.
-

1. Giới thiệu LINQ nâng cao

LINQ (Language Integrated Query) là công cụ mạnh mẽ để làm việc với dữ liệu trong C#. Trong bài này, chúng ta tập trung vào:

- **Phép nối (Join):** Để liên kết dữ liệu từ nhiều nguồn.
 - **Phân vùng (Partitioning):** Giới hạn hoặc phân chia dữ liệu.
 - **Phân nhóm (Grouping):** Gom nhóm dữ liệu dựa trên điều kiện cụ thể.
 - **Lazy Evaluation:** Điều gì xảy ra khi truy vấn được thực thi.
-

2. Phép nối (Joins)

2.1. Nội dung

LINQ hỗ trợ các kiểu nối dữ liệu như:

- **Inner Join:** Kết hợp hai tập dữ liệu dựa trên điều kiện phù hợp.
- **Group Join:** Kết hợp và nhóm dữ liệu cùng lúc.
- **Left Join** (bằng cách giả lập).

2.2. Ví dụ minh họa

```
var students = new[]
{
    new { Id = 1, Name = "Alice" },
    new { Id = 2, Name = "Bob" },
    new { Id = 3, Name = "Charlie" }
};

var scores = new[]
{
```

```
        new { StudentId = 1, Score = 90 },
        new { StudentId = 2, Score = 80 }
    };

// Inner Join
var query = from student in students
            join score in scores on student.Id equals score.StudentId
            select new { student.Name, score.Score };

foreach (var item in query)
{
    Console.WriteLine($"{item.Name} - {item.Score}");
}
```

2.3. Left Join

```
var leftJoin = from student in students
               join score in scores on student.Id equals score.StudentId into
studentScores
               from score in studentScores.DefaultIfEmpty()
               select new { student.Name, Score = score?.Score ?? 0 };

foreach (var item in leftJoin)
{
    Console.WriteLine($"{item.Name} - {item.Score}");
}
```

3. Phân vùng (Partitioning)

3.1. Toán tử quan trọng

- **Take**: Lấy số lượng phần tử đầu tiên.
- **Skip**: Bỏ qua số lượng phần tử đầu tiên.
- **TakeWhile** & **SkipWhile**: Hoạt động dựa trên điều kiện.

3.2. Ví dụ minh họa

```
var numbers = Enumerable.Range(1, 10);

// Lấy 3 phần tử đầu tiên
var firstThree = numbers.Take(3);
Console.WriteLine("First 3 numbers: " + string.Join(", ", firstThree));

// Bỏ qua 5 phần tử đầu tiên
var skipFive = numbers.Skip(5);
Console.WriteLine("After skipping 5: " + string.Join(", ", skipFive));
```

4. Phân nhóm (Grouping)

4.1. Toán tử `GroupBy`

Phân nhóm dữ liệu dựa trên khóa cụ thể.

4.2. Ví dụ minh họa GroupBy với 1 key

```
var people = new[]
{
    new { Name = "Alice", Age = 25 },
    new { Name = "Bob", Age = 30 },
    new { Name = "Charlie", Age = 25 },
    new { Name = "David", Age = 30 }
};

var grouped = people.GroupBy(p => p.Age);

foreach (var group in grouped)
{
    Console.WriteLine($"Age Group: {group.Key}");
    foreach (var person in group)
    {
        Console.WriteLine($" - {person.Name}");
    }
}
```

4.3. Ví dụ GroupBy với nhiều Key

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        // Dữ liệu mẫu
        var employees = new List<Employee>
        {
            new Employee { Name = "Alice", Department = "HR", Location = "New York",
Salary = 60000 },
            new Employee { Name = "Bob", Department = "IT", Location = "New York",
Salary = 80000 },
            new Employee { Name = "Charlie", Department = "IT", Location =
"Chicago", Salary = 70000 },
            new Employee { Name = "David", Department = "HR", Location = "Chicago",
Salary = 55000 },
            new Employee { Name = "Eve", Department = "IT", Location = "New York",
Salary = 85000 },
        };

        // Group by Department và Location
        var groupedEmployees = employees
            .GroupBy(e => new { e.Department, e.Location })
            .Select(g => new
            {
                Department = g.Key.Department,
                Location = g.Key.Location,
                TotalSalary = g.Sum(e => e.Salary),
                Employees = g.ToList()
            });
    }
}
```

```
// In kết quả
foreach (var group in groupedEmployees)
{
    Console.WriteLine($"Department: {group.Department}, Location:
{group.Location}");
    Console.WriteLine($" Total Salary: {group.TotalSalary}");
    foreach (var employee in group.Employees)
    {
        Console.WriteLine($"    - {employee.Name}, Salary:
{employee.Salary}");
    }
}

}

class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public string Location { get; set; }
    public int Salary { get; set; }
}
```

Kết quả:

Department: HR, Location: New York
Total Salary: 60000
- Alice, Salary: 60000
Department: IT, Location: New York
Total Salary: 165000
- Bob, Salary: 80000
- Eve, Salary: 85000
Department: IT, Location: Chicago
Total Salary: 70000
- Charlie, Salary: 70000
Department: HR, Location: Chicago
Total Salary: 55000
- David, Salary: 55000

Giải thích:

- GroupBy(e => new { e.Department, e.Location }): Nhóm các phần tử dựa trên hai thuộc tính Department và Location.
- Sử dụng Select để tính tổng lương (TotalSalary) cho mỗi nhóm và thu thập danh sách các nhân viên trong nhóm đó.

4.4. Group By với điều kiện Having

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        // Dữ liệu mẫu
        var employees = new List<Employee>
        {
```



```
        new Employee { Name = "Alice", Department = "HR", Location = "New York",
Salary = 60000 },
        new Employee { Name = "Bob", Department = "IT", Location = "New York",
Salary = 80000 },
        new Employee { Name = "Charlie", Department = "IT", Location =
"Chicago", Salary = 70000 },
        new Employee { Name = "David", Department = "HR", Location = "Chicago",
Salary = 55000 },
        new Employee { Name = "Eve", Department = "IT", Location = "New York",
Salary = 85000 },
    };

    // Group by Department và Location, sau đó lọc các nhóm có tổng lương lớn
    hơn 100000
    var groupedEmployees = employees
        .GroupBy(e => new { e.Department, e.Location })
        .Where(g => g.Sum(e => e.Salary) > 100000) // Điều kiện HAVING
        .Select(g => new
        {
            Department = g.Key.Department,
            Location = g.Key.Location,
            TotalSalary = g.Sum(e => e.Salary),
            Employees = g.ToList()
        });

    // In kết quả
    foreach (var group in groupedEmployees)
    {
        Console.WriteLine($"Department: {group.Department}, Location:
{group.Location}");
        Console.WriteLine($"  Total Salary: {group.TotalSalary}");
        foreach (var employee in group.Employees)
        {
            Console.WriteLine($"    - {employee.Name}, Salary:
{employee.Salary}");
        }
    }
}

class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public string Location { get; set; }
    public int Salary { get; set; }
}
```

Kết quả:

Department: IT, Location: New York
Total Salary: 165000
- Bob, Salary: 80000
- Eve, Salary: 85000

Giải thích:

1. **GroupBy(e => new { e.Department, e.Location })**: Nhóm các nhân viên theo Department và Location.
2. **Where(g => g.Sum(e => e.Salary) > 100000)**: Áp dụng điều kiện HAVING chỉ giữ lại các nhóm có tổng lương lớn hơn 100000.
3. **Select(...)**: Lựa chọn các thông tin cần thiết, bao gồm tổng lương và danh sách nhân viên trong nhóm.

Điều kiện HAVING trong LINQ được thực hiện sau GroupBy bằng cách sử dụng Where để lọc các nhóm dựa trên kết quả tổng hợp.

4.5. Kết hợp Join, GroupBy, và Having

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        // Bảng nhân viên (Employees)
        var employees = new List<Employee>
        {
            new Employee { Id = 1, Name = "Alice", DepartmentId = 1, Salary = 60000 },
            new Employee { Id = 2, Name = "Bob", DepartmentId = 2, Salary = 80000 },
            new Employee { Id = 3, Name = "Charlie", DepartmentId = 2, Salary = 70000 },
            new Employee { Id = 4, Name = "David", DepartmentId = 1, Salary = 55000 },
            new Employee { Id = 5, Name = "Eve", DepartmentId = 2, Salary = 85000 },
        };

        // Bảng phòng ban (Departments)
        var departments = new List<Department>
        {
            new Department { Id = 1, Name = "HR", Location = "New York" },
            new Department { Id = 2, Name = "IT", Location = "Chicago" },
        };

        // Join giữa Employees và Departments, sau đó GroupBy và lọc bằng điều kiện
        // HAVING
        var groupedData = employees
            .Join(departments,
                e => e.DepartmentId,
                d => d.Id,
                (e, d) => new { EmployeeName = e.Name, e.Salary, DepartmentName = d.Name, d.Location }) // Join dữ liệu
            .GroupBy(ed => new { ed.DepartmentName, ed.Location }) // Nhóm theo
            // Name và Location
            .Where(g => g.Sum(ed => ed.Salary) > 120000) // Điều kiện HAVING: Tổng
            // lương > 120,000
            .Select(g => new
            {
                DepartmentName = g.Key.DepartmentName,
                Location = g.Key.Location,
                TotalSalary = g.Sum(ed => ed.Salary),
                Employees = g.Select(ed => new { ed.EmployeeName, ed.Salary })
            })
            .ToList();

        Console.WriteLine("Kết quả sau Join, GroupBy và HAVING:");
        foreach (var group in groupedData)
        {
            Console.WriteLine($"Phòng ban: {group.DepartmentName}, Vị trí: {group.Location}, Tổng lương: {group.TotalSalary}");
            foreach (var employee in group.Employees)
            {
                Console.WriteLine($"  Nhân viên: {employee.EmployeeName}, Lương: {employee.Salary}");
            }
        }
    }
}
```

```
        Department = g.Key.DepartmentName,
        Location = g.Key.Location,
        TotalSalary = g.Sum(ed => ed.Salary),
        Employees = g.ToList()
    });

    // In kết quả
    foreach (var group in groupedData)
    {
        Console.WriteLine($"Department: {group.Department}, Location:
{group.Location}");
        Console.WriteLine($"  Total Salary: {group.TotalSalary}");
        foreach (var employee in group.Employees)
        {
            Console.WriteLine($"    - Employee: {employee.EmployeeName}, Salary:
{employee.Salary}");
        }
    }
}

class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int DepartmentId { get; set; }
    public int Salary { get; set; }
}

class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }
}
```

Kết quả:

Department: IT, Location: Chicago

Total Salary: 235000

- Employee: Bob, Salary: 80000
- Employee: Charlie, Salary: 70000
- Employee: Eve, Salary: 85000

Để kết hợp GroupBy với Join trong LINQ, bạn có thể sử dụng cú pháp join trước, sau đó thực hiện GroupBy và thêm điều kiện lọc (having) như trong ví dụ dưới đây.

Ví dụ: Kết hợp Join, GroupBy, và Having

```
// Bảng nhân viên (Employees)
var employees = new List<Employee>
{
    new Employee { Id = 1, Name = "Alice", DepartmentId = 1, Salary = 60000
},
    new Employee { Id = 2, Name = "Bob", DepartmentId = 2, Salary = 80000 },
    new Employee { Id = 3, Name = "Charlie", DepartmentId = 2, Salary =
70000 },
}
```

```
        new Employee { Id = 4, Name = "David", DepartmentId = 1, Salary = 55000
    },
        new Employee { Id = 5, Name = "Eve", DepartmentId = 2, Salary = 85000 }
    };

// Bảng phòng ban (Departments)
var departments = new List<Department>
{
    new Department { Id = 1, Name = "HR", Location = "New York" },
    new Department { Id = 2, Name = "IT", Location = "Chicago" },
};

// Join giữa Employees và Departments, sau đó GroupBy và lọc bằng điều kiện HAVING
var groupedData = employees
    .Join(departments,
        e => e.DepartmentId,
        d => d.Id,
        (e, d) => new { e.Name, e.Salary, d.Name, d.Location }) // Join dữ liệu
    .GroupBy(ed => new { ed.Name, ed.Location }) // Nhóm theo Name và Location
    .Where(g => g.Sum(ed => ed.Salary) > 1200)
    .GroupBy(ed => new { ed.Name, ed.Location }) // Nhóm theo Name và Location
    .Where(g => g.Sum(ed => ed.Salary) > 120000) // Điều kiện HAVING: Tổng lương >
120,000
    .Select(g => new
    {
        Department = g.Key.Name,
        Location = g.Key.Location,
        TotalSalary = g.Sum(ed => ed.Salary),
        Employees = g.ToList()
    });

// In kết quả
foreach (var group in groupedData)
{
    Console.WriteLine($"Department: {group.Department}, Location:
{group.Location}");
    Console.WriteLine($" Total Salary: {group.TotalSalary}");
    foreach (var employee in group.Employees)
    {
        Console.WriteLine($" - Employee: {employee.Name}, Salary:
{employee.Salary}");
    }
}

class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int DepartmentId { get; set; }
    public int Salary { get; set; }
}

class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }
}
```

Kết quả:

Department: IT, Location: Chicago

Total Salary: 235000

- Employee: Bob, Salary: 80000
- Employee: Charlie, Salary: 70000
- Employee: Eve, Salary: 85000

Giải thích:

1. **Join:**
 - o Liên kết hai tập dữ liệu employees và departments qua DepartmentId (e.DepartmentId và d.Id).
 - o Kết quả là một đối tượng ẩn danh chứa dữ liệu từ cả hai bảng.
2. **GroupBy:**
 - o Nhóm dữ liệu theo Name của phòng ban (ed.Name) và Location.
3. **Where:**
 - o Lọc các nhóm có tổng lương (Sum(ed => ed.Salary)) lớn hơn 120,000. Đây tương tự như HAVING trong SQL.
4. **Select:**
 - o Chọn thông tin cần thiết để hiển thị, bao gồm Department, Location, TotalSalary, và danh sách nhân viên (Employees).

5. Lazy Evaluation

5.1. Khái niệm

- LINQ sử dụng **lazy evaluation**, nghĩa là truy vấn chỉ thực thi khi thực sự cần dữ liệu.
- Điều này giúp tối ưu hiệu suất nhưng cần cẩn trọng để tránh vấn đề khi dữ liệu thay đổi trong lúc thực thi.

5.2. Ví dụ minh họa

```
var numbers = Enumerable.Range(1, 10);  
var query = numbers.Where(n => n > 5);  
  
numbers = Enumerable.Range(1, 20); // Thay đổi nguồn dữ liệu  
  
Console.WriteLine("Filtered numbers: " + string.Join(", ", query));
```

6. Thực hành

1. Tạo một danh sách sinh viên và điểm số, thực hiện **Inner Join** và **Left Join**.

2. Sử dụng dữ liệu chuỗi, áp dụng **Skip**, **Take**, và **GroupBy** để phân tích.
 3. Minh họa **lazy evaluation** với danh sách thay đổi.
-

7. Tổng kết

- LINQ nâng cao giúp xử lý dữ liệu phức tạp hiệu quả.
- Hiểu rõ và áp dụng tốt các kỹ thuật như join, phân vùng, grouping sẽ làm tăng chất lượng mã.
- Lazy evaluation là một khái niệm quan trọng cần lưu ý khi tối ưu hóa hiệu năng.

1.2. Understanding and implementing custom collections.

Mục tiêu bài học:

- Hiểu khái niệm Custom Collections và tại sao cần sử dụng.
 - Tìm hiểu cách xây dựng một Custom Collection từ đầu.
 - Áp dụng các giao diện quan trọng trong .NET như `IEnumerable<T>`, `ICollection<T>`, và `IList<T>`.
-

1. Tại sao cần Custom Collections?

- **Custom Collections** cho phép bạn tùy chỉnh logic lưu trữ và truy xuất dữ liệu khi các lớp như `List<T>` hoặc `Dictionary<K, V>` không đáp ứng đủ yêu cầu.
 - Một số trường hợp sử dụng:
 - Kiểm soát cách dữ liệu được thêm, xóa, hoặc sửa đổi.
 - Hỗ trợ các hành vi tùy chỉnh như tính toán khi truy xuất dữ liệu.
 - Đảm bảo các ràng buộc (constraints) đặc biệt trên dữ liệu.
-

2. Các bước cơ bản để tạo Custom Collection

2.1. Sử dụng giao diện `IEnumerable<T>`

Giao diện này cho phép duyệt qua các phần tử trong collection bằng vòng lặp `foreach`.

```
using System;
using System.Collections;
using System.Collections.Generic;
public class CustomCollection<T> : IEnumerable<T>
{
    private List<T> _items = new List<T>();

    public void Add(T item) => _items.Add(item);

    public IEnumerator<T> GetEnumerator() => _items.GetEnumerator();

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

// Sử dụng:
var collection = new CustomCollection<int>();
collection.Add(1);
collection.Add(2);
collection.Add(3);

foreach (var item in collection)
{
    Console.WriteLine(item);
}
```

2.2. Sử dụng giao diện ICollection<T>

ICollection<T> bổ sung các phương thức như Add, Remove, Count, và IsReadOnly.

```
using System;
using System.Collections.Generic;

public class CustomCollection<T> : ICollection<T>
{
    private List<T> _items = new List<T>();

    public int Count => _items.Count;
    public bool IsReadOnly => false;

    public void Add(T item) => _items.Add(item);
    public void Clear() => _items.Clear();
    public bool Contains(T item) => _items.Contains(item);
    public void CopyTo(T[] array, int arrayIndex) => _items.CopyTo(array,
arrayIndex);
    public bool Remove(T item) => _items.Remove(item);

    public IEnumerator<T> GetEnumerator() => _items.GetEnumerator();
    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator() =>
GetEnumerator();
}

// Sử dụng:
var collection = new CustomCollection<string>();
collection.Add("C#");
```

```
collection.Add("Advanced");  
Console.WriteLine($"Count: {collection.Count}");
```

2.3. Sử dụng giao diện `ICollection<T>`

`ICollection<T>` mở rộng `ICollection<T>` để hỗ trợ truy cập phần tử qua chỉ số.

```
using System;  
using System.Collections.Generic;  
  
public class CustomCollection<T> : ICollection<T>  
{  
    private List<T> _items = new List<T>();  
  
    public T this[int index]  
    {  
        get => _items[index];  
        set => _items[index] = value;  
    }  
  
    public int Count => _items.Count;  
    public bool IsReadOnly => false;  
  
    public void Add(T item) => _items.Add(item);  
    public void Clear() => _items.Clear();  
    public bool Contains(T item) => _items.Contains(item);  
    public void CopyTo(T[] array, int arrayIndex) => _items.CopyTo(array,  
arrayIndex);  
    public IEnumerator<T> GetEnumerator() => _items.GetEnumerator();  
    public int IndexOf(T item) => _items.IndexOf(item);  
    public void Insert(int index, T item) => _items.Insert(index, item);  
    public bool Remove(T item) => _items.Remove(item);  
    public void RemoveAt(int index) => _items.RemoveAt(index);  
  
    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator() =>  
GetEnumerator();  
}  
  
// Sử dụng:  
var collection = new CustomCollection<int>();  
collection.Add(10);  
collection.Add(20);  
collection[1] = 30; // Sửa giá trị tại index 1  
Console.WriteLine($"Element at index 1: {collection[1]}");
```

3. Các lưu ý khi triển khai Custom Collection

1. Hiệu suất:

- Tối ưu hóa các phương thức như `Contains` hoặc `IndexOf` nếu dữ liệu phức tạp.
 - 2. **Xử lý ngoại lệ:**
 - Kiểm tra chỉ số hợp lệ trong các phương thức như `this[int index]` hoặc `Insert`.
 - 3. **Tương thích với LINQ:**
 - Tích hợp các phương thức `IEnumerable<T>` để hỗ trợ các truy vấn LINQ.
-

4. Bài tập thực hành

1. Tạo một **Custom Collection** lưu trữ số nguyên, không cho phép thêm số trùng lặp.
 2. Thêm tính năng ghi log khi có phần tử được thêm hoặc xóa khỏi collection.
 3. Viết unit test để kiểm tra các chức năng của Custom Collection.
-

5. Kết luận

- Việc hiểu và triển khai Custom Collections giúp bạn tùy chỉnh hành vi lưu trữ và truy xuất dữ liệu theo yêu cầu cụ thể.
- Bài học này cung cấp nền tảng cho các ứng dụng phức tạp hơn trong thực tế như caching, quản lý bộ nhớ, hoặc xử lý dữ liệu đặc thù.

1.3. Practical use cases and performance considerations.

I. Mục tiêu bài học

- Hiểu rõ các trường hợp ứng dụng thực tế của C# trong các dự án phần mềm.
 - Nắm vững các cân nhắc hiệu suất khi sử dụng C# trong các bài toán lớn.
 - Cách tối ưu mã nguồn và sử dụng các công cụ hỗ trợ.
-

II. Nội dung chi tiết

1. Practical Use Cases của C#

C# là một ngôn ngữ mạnh mẽ và linh hoạt, phù hợp với nhiều loại dự án phần mềm:

1. **Phát triển ứng dụng web (Web Development):**
 - **Công nghệ:** ASP.NET Core.
 - **Ứng dụng:** Xây dựng hệ thống thương mại điện tử, trang web quản lý doanh nghiệp (CMS), hoặc APIs.
 - **Ví dụ:** Sử dụng ASP.NET Core Web API để xây dựng backend cho ứng dụng e-commerce.
 2. **Ứng dụng máy tính để bàn (Desktop Applications):**
 - **Công nghệ:** Windows Forms, WPF, hoặc MAUI.
 - **Ứng dụng:** Các công cụ nội bộ hoặc phần mềm phục vụ khách hàng.
 - **Ví dụ:** Ứng dụng quản lý nhân sự hoặc tài chính.
 3. **Ứng dụng di động (Mobile Applications):**
 - **Công nghệ:** Xamarin hoặc MAUI.
 - **Ứng dụng:** Phát triển ứng dụng đa nền tảng (Android/iOS).
 - **Ví dụ:** Ứng dụng quản lý công việc cá nhân hoặc nhóm.
 4. **Lập trình game:**
 - **Công nghệ:** Unity (sử dụng C# cho scripting).
 - **Ứng dụng:** Game 3D, game VR/AR.
 - **Ví dụ:** Xây dựng game học tập thực tế ảo.
 5. **Ứng dụng xử lý dữ liệu (Data Processing):**
 - **Công nghệ:** LINQ, Entity Framework Core, hoặc tích hợp với các thư viện .NET.
 - **Ứng dụng:** Xử lý dữ liệu lớn, phân tích dữ liệu, hoặc đồng bộ hóa cơ sở dữ liệu.
-

2. Performance Considerations (Hiệu suất)

Khi làm việc với các dự án thực tế, hiệu suất đóng vai trò quan trọng. Sau đây là các cân nhắc:

1. **Quản lý bộ nhớ:**
 - **Công cụ:**
 - **Garbage Collector (GC):** Cần hiểu cách hoạt động của GC để giảm thiểu việc sử dụng bộ nhớ không cần thiết.
 - **Dispose và IDisposable:** Giải phóng tài nguyên đúng cách.
 - **Ví dụ:** Đảm bảo giải phóng các đối tượng kết nối cơ sở dữ liệu hoặc tệp tin.
2. **Xử lý song song (Parallelism):**
 - **Công cụ:** **Task Parallel Library (TPL)**, **async/await**, hoặc **Parallel.ForEach**.
 - **Ví dụ:** Tăng tốc quá trình xử lý dữ liệu lớn bằng cách chia nhỏ công việc và chạy song song.
3. **Lập trình không đồng bộ (Asynchronous Programming):**
 - **Tình huống sử dụng:**
 - Tăng hiệu suất khi thực hiện các thao tác I/O (API call, truy vấn database).
 - **Ví dụ:** Sử dụng **HttpClient** để gọi API mà không chặn luồng chính.

4. **Tối ưu hóa truy vấn dữ liệu:**
 - **Công cụ:** Sử dụng LINQ hiệu quả, tránh các truy vấn lồng nhau phức tạp.
 - **Ví dụ:** Tránh sử dụng `ToList()` quá sớm khi làm việc với `IQueryable`.
 5. **Hiệu suất thuật toán:**
 - **Nguyên tắc:** Sử dụng các thuật toán tối ưu về độ phức tạp thời gian (Big-O).
 - **Ví dụ:** Chọn `Dictionary` thay vì `List` khi cần tra cứu nhanh.
 6. **Công cụ đo lường và phân tích:**
 - **Profiler:** Visual Studio Profiler, JetBrains DotTrace.
 - **Benchmark:** Sử dụng thư viện `BenchmarkDotNet` để so sánh hiệu suất các đoạn mã.
-

III. Bài tập thực hành

1. **Case Study 1:** Tối ưu hóa hiệu suất của ứng dụng ASP.NET Core API với 1000 requests/giây.
 - Mô phỏng bằng JMeter hoặc Postman.
 - Áp dụng `async/await` và kiểm tra sự khác biệt.
 2. **Case Study 2:** Sử dụng LINQ để xử lý danh sách lớn và so sánh thời gian thực thi với vòng lặp thông thường.
 3. **Case Study 3:** Tích hợp `Parallel.ForEach` để xử lý song song tập dữ liệu lớn (ví dụ: 1 triệu bản ghi).
-

IV. Tóm tắt

- C# là lựa chọn tốt cho các dự án từ ứng dụng web, desktop, đến di động và game.
- Tối ưu hóa hiệu suất không chỉ giúp ứng dụng chạy nhanh hơn mà còn tiết kiệm tài nguyên hệ thống.
- Sử dụng đúng công cụ và kỹ thuật (GC, `async/await`, LINQ) sẽ mang lại hiệu quả vượt trội.

2. Reflection and Dynamic Types

2.1. Using reflection to inspect and modify objects at runtime.

Reflection là một tính năng mạnh mẽ trong C#, cho phép chúng ta kiểm tra các thông tin của kiểu (type), đối tượng (object), và thậm chí thay đổi trạng thái hoặc gọi các phương thức của

đối tượng đó tại thời gian chạy. Điều này rất hữu ích trong các tình huống như xây dựng framework, thực hiện kiểm thử, hoặc xử lý những kịch bản mà mã nguồn không cố định trước.

1. Khái niệm cơ bản về Reflection

Reflection được hỗ trợ bởi namespace **System.Reflection**. Các thành phần quan trọng của Reflection bao gồm:

- **Type**: Đại diện cho thông tin của kiểu (class, interface, struct, enum, v.v.).
 - **PropertyInfo**: Đại diện cho thông tin của các thuộc tính trong lớp.
 - **MethodInfo**: Đại diện cho thông tin của các phương thức.
 - **FieldInfo**: Đại diện cho thông tin của các trường (field).
-

2. Ứng dụng Reflection

2.1 Kiểm tra thông tin kiểu

```
using System;
using System.Reflection;

class Program
{
    static void Main()
    {
        Type type = typeof(SampleClass);

        Console.WriteLine("Tên lớp: " + type.Name);
        Console.WriteLine("Namespace: " + type.Namespace);

        Console.WriteLine("Các thuộc tính:");
        foreach (PropertyInfo prop in type.GetProperties())
        {
            Console.WriteLine($"- {prop.Name} ({prop.PropertyType.Name})");
        }

        Console.WriteLine("Các phương thức:");
        foreach (MethodInfo method in type.GetMethods())
        {
            Console.WriteLine($"- {method.Name}");
        }
    }
}

class SampleClass
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

```
}    public void Display() => Console.WriteLine("Display method called.");
```

Kết quả:

Tên lớp: SampleClass

Namespace: YourNamespace

Các thuộc tính:

- Id (Int32)
- Name (String)

Các phương thức:

- Display
- ToString
- GetHashCode

...

2.2 Tạo đối tượng và gọi phương thức tại runtime

```
using System;
using System.Reflection;

class Program
{
    static void Main()
    {
        Type type = typeof(SampleClass);

        // Tạo đối tượng động
        object instance = Activator.CreateInstance(type);

        // Gán giá trị cho thuộc tính "Name"
        PropertyInfo nameProperty = type.GetProperty("Name");
        nameProperty?.SetValue(instance, "Reflection Example");

        // Gọi phương thức "Display"
        MethodInfo displayMethod = type.GetMethod("Display");
        displayMethod?.Invoke(instance, null);
    }
}

class SampleClass
{
    public string Name { get; set; }

    public void Display() => Console.WriteLine($"Hello, {Name}!");
}
```

```
}
```

Kết quả:

Hello, Reflection Example!

2.3 Thay đổi trường hoặc thuộc tính không công khai (private)

```
using System;
using System.Reflection;

class Program
{
    static void Main()
    {
        Type type = typeof(SampleClass);
        object instance = Activator.CreateInstance(type);

        // Lấy thông tin field private
        FieldInfo privateField = type.GetField("_secret", BindingFlags.NonPublic |
BindingFlags.Instance);

        // Thay đổi giá trị field private
        privateField?.SetValue(instance, "Updated Secret Value");

        // Gọi phương thức để kiểm tra
        MethodInfo revealMethod = type.GetMethod("RevealSecret");
        revealMethod?.Invoke(instance, null);
    }
}

class SampleClass
{
    private string _secret = "Original Secret Value";

    public void RevealSecret() => Console.WriteLine($"Secret: {_secret}");
}
```

Kết quả:

makefile

Secret: Updated Secret Value

3. Lưu ý khi sử dụng Reflection

1. **Hiệu suất:** Reflection chậm hơn so với truy cập trực tiếp, nên không nên lạm dụng.
 2. **Bảo mật:** Việc thao tác trên các thành phần private hoặc protected cần được cân nhắc kỹ lưỡng.
 3. **Khả năng bảo trì:** Mã sử dụng Reflection thường khó đọc và bảo trì hơn.
-

4. Bài tập thực hành

1. Viết chương trình sử dụng Reflection để kiểm tra tất cả các thuộc tính và phương thức trong một lớp, bao gồm cả các thành phần private.
2. Thực hiện thay đổi giá trị của một trường private và kiểm tra xem thay đổi đó có hiệu lực hay không.

2.2. Introduction to dynamic types and the dynamic keyword.

1. **dynamic** là gì?

- **dynamic** là một kiểu dữ liệu được giới thiệu trong C# 4.0.
 - Biến kiểu **dynamic** sẽ bỏ qua việc kiểm tra kiểu tại thời điểm biên dịch (**compile-time**), nhưng kiểm tra kiểu tại thời điểm chạy (**runtime**).
 - Điều này cho phép bạn làm việc với các đối tượng mà kiểu của chúng không thể biết trước trong quá trình biên dịch.
-

2. Đặc điểm của **dynamic**

- Tương tự **object**, nhưng tiện lợi hơn khi làm việc với các kiểu dữ liệu không xác định.
 - Các thành viên của đối tượng **dynamic** không được kiểm tra kiểu tại **compile-time**.
 - Nếu có lỗi (như gọi phương thức không tồn tại), lỗi sẽ chỉ xuất hiện tại **runtime**.
-

3. Sử dụng **dynamic** trong thực tế

3.1 Khai báo biến **dynamic**

```
dynamic variable = "Hello, world!";  
Console.WriteLine(variable); // In ra: Hello, world!
```

```
variable = 42;  
Console.WriteLine(variable); // In ra: 42  
  
variable = true;  
Console.WriteLine(variable); // In ra: True
```

3.2 Gọi phương thức với **dynamic**

```
using System.Dynamic;  
  
dynamic obj = new ExpandoObject();  
obj.Name = "John Doe"; // Gán động một thuộc tính  
obj.SayHello = new Action(() => Console.WriteLine("Hello!")); // Gán động một phương  
thức  
  
Console.WriteLine(obj.Name); // In ra: John Doe  
obj.SayHello(); // In ra: Hello!
```

3.3 Truy cập đối tượng COM hoặc dữ liệu JSON

- **dynamic** thường được sử dụng để làm việc với các thư viện COM như Excel hoặc các API trả về JSON.

Ví dụ làm việc với JSON:

```
using System.Dynamic;  
  
dynamic obj = new ExpandoObject();  
obj.Name = "John Doe"; // Gán động một thuộc tính  
obj.SayHello = new Action(() => Console.WriteLine("Hello!")); // Gán động một phương  
thức  
  
Console.WriteLine(obj.Name); // In ra: John Doe  
obj.SayHello(); // In ra: Hello!
```

4. Lợi ích của **dynamic**

1. Giảm sự phức tạp khi làm việc với dữ liệu không xác định kiểu (như JSON, XML).
 2. Làm việc hiệu quả với các API hoặc thư viện không sử dụng kiểu mạnh.
 3. Hữu ích khi làm việc với các hệ thống kiểu ngôn ngữ khác (COM Interop, Reflection).
-

5. Rủi ro khi sử dụng **dynamic**

- **Mất an toàn kiểu dữ liệu:** Vì không kiểm tra tại thời điểm biên dịch, dễ xảy ra lỗi runtime.
 - **Hiệu suất thấp hơn:** Phải dùng **Reflection** để kiểm tra và xử lý kiểu tại runtime.
 - **Khó bảo trì mã:** Nếu sử dụng không cẩn thận, mã nguồn dễ trở nên khó đọc và khó hiểu.
-

6. So sánh **dynamic** và **object**

Đặc điểm	dynamic	object
Kiểm tra kiểu	Tại runtime	Tại compile-time
Gọi thành viên	Trực tiếp, không cần ép kiểu	Cần ép kiểu trước khi sử dụng
Linh hoạt	Cao	Thấp

7. Khi nào nên sử dụng **dynamic**?

1. Làm việc với dữ liệu JSON/XML hoặc các đối tượng có kiểu không xác định trước.
 2. Tích hợp với các hệ thống hoặc API không dùng kiểu mạnh.
 3. Tạo các kịch bản lập trình linh hoạt hoặc nhanh chóng.
-

8. Ví dụ thực tế

Làm việc với Excel COM Interop:

```
dynamic excelApp =  
Activator.CreateInstance(Type.GetTypeFromProgID("Excel.Application"));  
excelApp.Visible = true;  
dynamic workbook = excelApp.Workbooks.Add();  
dynamic sheet = workbook.Sheets[1];  
sheet.Cells[1, 1].Value = "Hello from C#";
```

Tóm lại

dynamic là một công cụ mạnh mẽ khi cần linh hoạt trong xử lý dữ liệu hoặc tích hợp các thư viện không hỗ trợ kiểu mạnh. Tuy nhiên, bạn cần sử dụng cẩn thận để tránh lỗi runtime và giữ mã dễ bảo trì.

2.3. Practical applications and use cases for reflection.

Reflection là một công cụ mạnh mẽ trong .NET, cho phép bạn kiểm tra và thao tác với metadata của các đối tượng tại runtime. Trong bài giảng này, chúng ta sẽ tập trung vào các ứng dụng thực tiễn và các trường hợp sử dụng phổ biến của Reflection.

1. Tìm hiểu Reflection trong thực tế

Reflection có thể được áp dụng trong nhiều kịch bản thực tiễn, chẳng hạn như:

- **Tạo các công cụ tự động hóa:** Tạo trình phân tích mã, các framework, hoặc công cụ phát triển.
 - **Plugin và module:** Tải và thực thi mã hoặc module một cách động.
 - **Thực hiện Dependency Injection (DI):** Giúp giải quyết các dependencies tại runtime.
 - **Xử lý dữ liệu runtime:** Đọc hoặc ghi dữ liệu không xác định trước.
-

2. Các trường hợp sử dụng Reflection

2.1. Dependency Injection (DI)

Trong các hệ thống phức tạp, DI thường được sử dụng để khởi tạo các đối tượng. Reflection giúp tự động xác định constructor và các dependencies để inject.

Ví dụ:

```
public class ServiceA { }
public class ServiceB
{
    private readonly ServiceA _serviceA;
    public ServiceB(ServiceA serviceA)
    {
        _serviceA = serviceA;
    }
}

public static T CreateInstance<T>()
{

```

```
var constructor = typeof(T).GetConstructors().First();
var parameters = constructor.GetParameters()
    .Select(p => Activator.CreateInstance(p.ParameterType))
    .ToArray();
return (T)constructor.Invoke(parameters);
}

// Sử dụng:
var serviceB = CreateInstance<ServiceB>();
Console.WriteLine(serviceB);
```

2.2. Thực hiện Mapping từ dữ liệu

Reflection thường được sử dụng trong các thư viện ORM như Entity Framework để ánh xạ dữ liệu từ cơ sở dữ liệu vào các đối tượng.

Ví dụ:

```
public static T MapToObject<T>(Dictionary<string, object> data) where T : new()
{
    var obj = new T();
    foreach (var prop in typeof(T).GetProperties())
    {
        if (data.ContainsKey(prop.Name) && prop.CanWrite)
        {
            prop.SetValue(obj, Convert.ChangeType(data[prop.Name],
prop.PropertyType));
        }
    }
    return obj;
}

// Sử dụng:
var data = new Dictionary<string, object>
{
    { "Id", 1 },
    { "Name", "Reflection Example" }
};

var result = MapToObject<MyClass>(data);
Console.WriteLine(result.Name);
```

2.3. Kiểm tra và xác thực (Validation)

Reflection hữu ích khi bạn cần kiểm tra các thuộc tính của một lớp dựa trên các custom attribute.

Ví dụ:

```
[AttributeUsage(AttributeTargets.Property)]
public class RequiredAttribute : Attribute { }

public class User
```

```
{
    [Required]
    public string Name { get; set; }
    public int Age { get; set; }
}

public static bool Validate<T>(T obj)
{
    foreach (var prop in typeof(T).GetProperties())
    {
        var isRequired = Attribute.IsDefined(prop, typeof(RequiredAttribute));
        var value = prop.GetValue(obj);
        if (isRequired && (value == null ||
string.IsNullOrEmpty(value.ToString())))
        {
            return false;
        }
    }
    return true;
}

// Sử dụng:
var user = new User { Name = null };
Console.WriteLine(Validate(user)); // Kết quả: False
```

2.4. Tạo Unit Test tự động

Reflection giúp tạo các Unit Test tự động bằng cách kiểm tra các method public hoặc private.

Ví dụ:

```
using System.Reflection;

public static void TestMethods<T>()
{
    foreach (var method in typeof(T).GetMethods(BindingFlags.Public |
BindingFlags.Instance))
    {
        Console.WriteLine($"Testing method: {method.Name}");
        // Giả định rằng method không có tham số
        method.Invoke(Activator.CreateInstance(typeof(T)), null);
    }
}

// Sử dụng:
TestMethods<MyTestClass>();
```

2.5. Phân tích và xử lý dữ liệu động (Dynamic Data Handling)

Khi làm việc với các kiểu dữ liệu không biết trước tại runtime, Reflection rất hữu ích.

Ví dụ:

```
public static void DisplayObjectInfo(object obj)
```

```
{
    var type = obj.GetType();
    Console.WriteLine($"Type: {type.Name}");
    foreach (var prop in type.GetProperties())
    {
        Console.WriteLine($"{prop.Name}: {prop.GetValue(obj)}");
    }
}

// Sử dụng:
DisplayObjectInfo(new { Id = 1, Name = "Dynamic Object" });
```

3. Lưu ý khi sử dụng Reflection

- Reflection có thể làm giảm hiệu năng vì cần xử lý tại runtime.
 - Sử dụng nó đúng cách, tránh làm tăng độ phức tạp không cần thiết.
 - Kiểm tra quyền truy cập vì có thể vi phạm nguyên tắc encapsulation.
-

4. Thực hành

- Xây dựng một ứng dụng console sử dụng Reflection để tạo và quản lý các plugin động.
 - Áp dụng Reflection vào một dự án có sử dụng Attribute để kiểm tra tính hợp lệ của dữ liệu đầu vào.
-

Reflection mở ra khả năng tùy chỉnh cao trong các ứng dụng C# nhưng cần sử dụng cẩn thận để tránh ảnh hưởng đến hiệu suất và tính ổn định.

3. Memory Management and Garbage Collection

3.1. How memory management works in .NET.

1. Giới thiệu về Bộ nhớ trong .NET

- **Bộ nhớ heap và stack:**
 - **Stack:** Dùng để lưu trữ các biến giá trị và các tham số hàm. Bộ nhớ này được cấp phát và giải phóng tự động khi các hàm được gọi và trả về.
 - **Heap:** Dùng để lưu trữ các đối tượng (dạng tham chiếu), chẳng hạn như các lớp trong .NET. Bộ nhớ heap cần được quản lý bằng garbage collector (GC).

2. Quản lý bộ nhớ trong .NET

- **Garbage Collection (GC):**
 - GC trong .NET là một cơ chế tự động giúp giải phóng bộ nhớ không còn được sử dụng nữa, điều này giúp giảm thiểu các vấn đề liên quan đến rò rỉ bộ nhớ.
 - **Generations (Các thế hệ):** GC phân chia bộ nhớ heap thành các thế hệ (gen0, gen1, gen2). Các đối tượng mới sẽ được cấp phát ở **gen0** và sẽ di chuyển qua các thế hệ cao hơn khi tồn tại lâu hơn.
 - **Young Generation và Old Generation:** Các đối tượng mới thường được giữ trong young generation. Khi các đối tượng tồn tại lâu hơn (ví dụ, một đối tượng đã sống qua nhiều lần thu gom rác), chúng sẽ được chuyển lên old generation.

3. Cách thức hoạt động của Garbage Collector

- **Các giai đoạn của GC:**
 - **Mark Phase:** Xác định các đối tượng nào còn được tham chiếu (reachable).
 - **Sweep Phase:** Giải phóng bộ nhớ của các đối tượng không còn tham chiếu.
 - **Compact Phase:** Di chuyển các đối tượng còn lại trong heap để giảm thiểu fragmentation (phân mảnh bộ nhớ).
- **GC Triggers:**
 - GC có thể được kích hoạt khi bộ nhớ gần đầy hoặc khi hệ thống yêu cầu thu gom rác để giải phóng bộ nhớ.

4. Các kỹ thuật tối ưu hóa quản lý bộ nhớ

- **Sử dụng `Dispose()` và `using` để giải phóng tài nguyên:**
 - Các đối tượng không phải là bộ nhớ heap (như các tài nguyên hệ thống, kết nối cơ sở dữ liệu) có thể cần được giải phóng thủ công thông qua phương thức `Dispose()` hoặc từ khối `using`.
- **Sử dụng Weak References:**
 - Weak references cho phép bạn tham chiếu đến một đối tượng mà không giữ nó trong bộ nhớ (dễ dàng cho GC thu gom).
- **Tối ưu hóa việc cấp phát và thu hồi bộ nhớ:**
 - Tránh tạo quá nhiều đối tượng mới liên tục, đặc biệt trong các vòng lặp.

5. Quản lý bộ nhớ trong các trường hợp đặc biệt

- **Native Memory Allocation:**
 - .NET có thể quản lý bộ nhớ gốc (native memory) khi làm việc với các thư viện C++ hoặc Win32 API.
 - Trong trường hợp này, bạn có thể cần sử dụng Interop và các kỹ thuật như `p/invoke` để truy cập và giải phóng bộ nhớ.
- **Memory Pooling:**

- Sử dụng các kỹ thuật như **Object Pooling** để tái sử dụng các đối tượng thay vì tạo mới và giải phóng chúng liên tục.

6. Best Practices

- **Giảm thiểu sự phân mảnh bộ nhớ** bằng cách tái sử dụng các đối tượng và hạn chế việc tạo ra quá nhiều đối tượng trong các đoạn mã yêu cầu hiệu suất cao.
- **Quản lý tài nguyên một cách cẩn thận** thông qua việc sử dụng các phương thức như `Dispose()` để tránh các rò rỉ bộ nhớ.

7. Công cụ hỗ trợ phân tích bộ nhớ

- **.NET Memory Profiler** và **Visual Studio Diagnostic Tools** giúp phân tích bộ nhớ, tìm kiếm các vấn đề như rò rỉ bộ nhớ hoặc phân mảnh bộ nhớ.

8. Tóm tắt

- Quản lý bộ nhớ trong .NET chủ yếu thông qua Garbage Collection (GC), giúp giảm bớt gánh nặng cho lập trình viên về việc giải phóng bộ nhớ.
- GC phân chia bộ nhớ heap thành nhiều thế hệ để tối ưu hóa việc thu gom rác.
- Các kỹ thuật tối ưu hóa và công cụ hỗ trợ có thể giúp cải thiện hiệu suất ứng dụng khi làm việc với bộ nhớ.

Dưới đây là một số ví dụ thực tế về cách quản lý bộ nhớ trong .NET, đặc biệt là về cách Garbage Collector (GC) hoạt động và cách tối ưu hóa bộ nhớ trong C#.

9. Ví dụ về Garbage Collection (GC)

C# tự động quản lý bộ nhớ thông qua Garbage Collector, do đó lập trình viên không cần phải giải phóng bộ nhớ thủ công. Dưới đây là một ví dụ minh họa:

```
using System;

public class MyClass
{
    public int Value;

    public MyClass(int value)
    {
        Value = value;
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        // Tạo các đối tượng MyClass
```

```
MyClass obj1 = new MyClass(10);
MyClass obj2 = new MyClass(20);

// Xóa tham chiếu đến đối tượng obj1
obj1 = null;

// Gọi GC để thu gom rác (không cần thiết, nhưng chỉ để minh họa)
GC.Collect();

// Xóa tham chiếu đến obj2, đối tượng này sẽ được GC thu gom khi không còn
tham chiếu
obj2 = null;

// Gọi GC lần nữa để thu gom các đối tượng không còn sử dụng
GC.Collect();

Console.WriteLine("Garbage Collection đã thực hiện.");
}
}
```

Trong ví dụ trên:

- Các đối tượng `obj1` và `obj2` được tạo ra và sau đó được gán giá trị `null` để đánh dấu chúng không còn được sử dụng.
- Sau khi gọi `GC.Collect()`, garbage collector sẽ tìm và thu gom những đối tượng không còn tham chiếu.

10. Sử dụng `Dispose` và `using` để quản lý tài nguyên

Khi làm việc với các tài nguyên ngoài bộ nhớ heap (ví dụ, các kết nối cơ sở dữ liệu hoặc tệp), chúng ta cần giải phóng chúng thủ công. `Dispose()` và `using` giúp việc này trở nên dễ dàng.

```
using System;
using System.IO;

public class Program
{
    public static void Main(string[] args)
    {
        // Sử dụng khối using để đảm bảo tài nguyên được giải phóng
        using (StreamWriter writer = new StreamWriter("example.txt"))
        {
            writer.WriteLine("Hello, World!");
        }

        Console.WriteLine("Tài nguyên đã được giải phóng sau khi sử dụng.");
    }
}
```

Trong ví dụ trên, `StreamWriter` là một đối tượng cần được giải phóng sau khi sử dụng, và khối `using` đảm bảo rằng tài nguyên được tự động giải phóng khi không còn sử dụng.

11. Sử dụng **WeakReference** để tránh giữ đối tượng trong bộ nhớ

Đôi khi bạn muốn giữ tham chiếu đến một đối tượng mà không ngăn Garbage Collector thu gom nó. Trong trường hợp này, bạn có thể sử dụng **WeakReference**.

```
using System;

public class MyClass
{
    public int Value;

    public MyClass(int value)
    {
        Value = value;
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        // Tạo đối tượng và sử dụng WeakReference
        MyClass obj = new MyClass(100);
        WeakReference weakRef = new WeakReference(obj);

        // Xóa tham chiếu chính thức
        obj = null;

        // Kiểm tra nếu đối tượng vẫn còn trong bộ nhớ
        if (weakRef.IsAlive)
        {
            MyClass retrievedObj = (MyClass)weakRef.Target;
            Console.WriteLine("Đối tượng vẫn còn sống với giá trị: " +
retrievedObj.Value);
        }
        else
        {
            Console.WriteLine("Đối tượng đã bị thu gom.");
        }

        // Gọi GC để thử thu gom rác
        GC.Collect();

        // Kiểm tra lại sau khi thu gom rác
        if (weakRef.IsAlive)
        {
            MyClass retrievedObj = (MyClass)weakRef.Target;
            Console.WriteLine("Đối tượng vẫn còn sống.");
        }
        else
        {
            Console.WriteLine("Đối tượng đã bị thu gom.");
        }
    }
}
```

Trong ví dụ này:

- Đối tượng `obj` được tham chiếu qua `WeakReference`. Điều này có nghĩa là garbage collector có thể thu gom đối tượng ngay cả khi `weakRef` vẫn tồn tại.

12. Tối ưu hóa với Object Pooling

Object pooling giúp tái sử dụng các đối tượng thay vì tạo mới liên tục, từ đó giảm thiểu việc cấp phát và giải phóng bộ nhớ, giúp cải thiện hiệu suất ứng dụng.

```
using System;
using System.Collections.Generic;

public class ObjectPool
{
    private Queue<MyClass> _pool = new Queue<MyClass>();

    public MyClass GetObject()
    {
        if (_pool.Count > 0)
        {
            return _pool.Dequeue();
        }
        return new MyClass();
    }

    public void ReturnObject(MyClass obj)
    {
        _pool.Enqueue(obj);
    }
}

public class MyClass
{
    public int Value { get; set; }

    public MyClass()
    {
        Console.WriteLine("Khởi tạo MyClass");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        ObjectPool pool = new ObjectPool();

        // Lấy đối tượng từ pool
        MyClass obj1 = pool.GetObject();
        obj1.Value = 10;

        // Trả lại đối tượng vào pool
        pool.ReturnObject(obj1);
    }
}
```

```
// Lấy đối tượng khác từ pool
MyClass obj2 = pool.GetObject();
Console.WriteLine("Giá trị của obj2: " + obj2.Value); // Sẽ in giá trị 10
}
}
```

Trong ví dụ này:

- **ObjectPool** giữ một hàng đợi (queue) các đối tượng có thể tái sử dụng, giúp giảm chi phí tạo mới đối tượng.

13. Memory Profiling và Debugging

Công cụ như **Visual Studio Diagnostic Tools** hoặc **.NET Memory Profiler** giúp bạn theo dõi bộ nhớ và phát hiện các vấn đề như rò rỉ bộ nhớ hoặc phân mảnh bộ nhớ. Bạn có thể sử dụng chúng để phân tích hiệu suất bộ nhớ trong ứng dụng thực tế của mình.

13. Tóm tắt

- **Garbage Collection** giúp tự động quản lý bộ nhớ trong .NET nhưng vẫn có thể bị ảnh hưởng bởi các yếu tố như phân mảnh bộ nhớ.
- **Dispose** và **using** giúp bạn giải phóng tài nguyên bên ngoài bộ nhớ heap.
- **WeakReference** cho phép tham chiếu đối tượng mà không giữ nó trong bộ nhớ.
- **Object Pooling** là một kỹ thuật tối ưu hóa giúp giảm thiểu chi phí tạo và giải phóng đối tượng.

3.2. Garbage collection and optimization tips.

1. Giới thiệu về Garbage Collection (GC)

- **Garbage Collection** trong C# là một cơ chế tự động quản lý bộ nhớ, giúp thu gom và giải phóng bộ nhớ của các đối tượng không còn được tham chiếu.
- **Lợi ích:** Giảm thiểu rủi ro rò rỉ bộ nhớ và giúp quản lý bộ nhớ dễ dàng hơn so với việc người lập trình phải giải phóng bộ nhớ thủ công (như trong C hoặc C++).
- **Chức năng:** GC theo dõi các đối tượng trong bộ nhớ heap và giải phóng những đối tượng không còn sử dụng nữa.

2. Các thế hệ trong Garbage Collection

- **Generation 0:** Đây là nơi chứa các đối tượng mới được tạo ra. GC sẽ kiểm tra Generation 0 đầu tiên vì các đối tượng này có thể không sống lâu.
- **Generation 1:** Các đối tượng sống lâu hơn sẽ được di chuyển từ Generation 0 sang Generation 1.

- **Generation 2:** Đây là nơi chứa các đối tượng "lâu dài", tức là các đối tượng sống qua nhiều lần thu gom.
- **Large Object Heap (LOH):** Đối với các đối tượng có kích thước lớn hơn 85,000 byte, chúng sẽ được lưu trữ tại LOH và được GC xử lý theo cách riêng biệt.

3. Các loại thu gom garbage collection

- **Full GC:** Thu gom toàn bộ bộ nhớ heap, mất nhiều thời gian và làm gián đoạn ứng dụng.
- **Minor GC:** Chỉ thu gom đối với Generation 0 và 1, ít tốn thời gian hơn so với Full GC.
- **Background GC:** Là hình thức thu gom trong nền, giúp giảm thiểu gián đoạn của ứng dụng.

4. Làm thế nào để tối ưu hóa Garbage Collection

- **Giảm thiểu số lượng đối tượng ngắn hạn:** Tạo ra ít đối tượng trong Generation 0 bằng cách giảm thiểu việc khởi tạo đối tượng.
- **Sử dụng cấu trúc dữ liệu hiệu quả:** Sử dụng các cấu trúc dữ liệu như `ArrayPool` hoặc `MemoryPool` để tái sử dụng bộ nhớ thay vì tạo mới đối tượng.
- **Giảm thiểu việc sử dụng các đối tượng có kích thước lớn:** Tránh sử dụng các đối tượng lớn trong LOH, vì chúng sẽ ảnh hưởng đến hiệu suất của GC.
- **Giảm độ phân mảnh của heap:** Sắp xếp lại bộ nhớ một cách hợp lý để GC dễ dàng giải phóng bộ nhớ không sử dụng.
- **Sử dụng `Dispose` đúng cách:** Đảm bảo đối tượng implement `IDisposable` để giải phóng tài nguyên ngoại vi như kết nối cơ sở dữ liệu, file stream ngay lập tức thay vì chờ đến khi GC thực hiện.

5. Tips tối ưu hóa khác

- **Sử dụng `WeakReference`:** Khi bạn muốn giữ tham chiếu đến đối tượng mà không làm tăng tuổi thọ của nó (tránh giữ đối tượng sống lâu trong bộ nhớ).
- **Tránh việc cấp phát bộ nhớ quá thường xuyên:** Sử dụng `StringBuilder` thay vì nối chuỗi liên tiếp, vì việc tạo mới chuỗi nhiều lần có thể làm tăng tần suất thu gom bộ nhớ.
- **Tối ưu hóa với `MemoryManagement API`:** Dùng `GC.Collect()` một cách hợp lý và chỉ khi cần thiết (ví dụ, khi bạn biết rằng sẽ có rất nhiều bộ nhớ không còn sử dụng nữa). Tuy nhiên, cần tránh gọi `GC.Collect()` thường xuyên vì nó có thể làm giảm hiệu suất.

6. Lập trình dựa trên cơ chế GC

- **Chờ đợi GC thu gom tự động:** Đảm bảo rằng mã của bạn không gây ra áp lực cho Garbage Collector quá lớn.

- **Quản lý bộ nhớ bằng cách phân bổ tài nguyên một cách cẩn thận:** Tránh sử dụng nhiều bộ nhớ đồng thời trong các luồng khác nhau.

7. Công cụ hỗ trợ GC trong C#

- **Visual Studio Diagnostic Tools:** Dùng công cụ này để theo dõi hoạt động của GC, xác định bộ nhớ bị rò rỉ và tối ưu hóa hiệu suất.
- **CLR Profiler:** Công cụ này cung cấp cái nhìn sâu sắc về các hoạt động thu gom rác và cách thức bộ nhớ được quản lý trong ứng dụng.

8. Thực hành và ví dụ

- **Ví dụ 1:** Tạo ra một ứng dụng đơn giản với nhiều đối tượng và theo dõi quá trình thu gom rác.
- **Ví dụ 2:** Tối ưu hóa bộ nhớ trong ứng dụng C# sử dụng `ArrayPool` và kiểm tra sự cải thiện về hiệu suất.

9. Kết luận

- Hiểu và tối ưu hóa Garbage Collection là một phần quan trọng trong việc phát triển ứng dụng C# hiệu quả, giảm thiểu bộ nhớ rò rỉ và tăng cường hiệu suất hệ thống.

Dưới đây là một ví dụ C# toàn diện để minh họa các tips tối ưu hóa Garbage Collection (GC) và quản lý bộ nhớ trong C#:

10. Giảm thiểu số lượng đối tượng ngắn hạn

Tạo ít đối tượng trong Generation 0 giúp giảm áp lực lên Garbage Collector. Ví dụ:

```
// Tránh tạo nhiều đối tượng ngắn hạn trong vòng lặp
using System;

for (int i = 0; i < 100000; i++)
{
    var person = new Person(); // Đối tượng ngắn hạn sẽ nhanh chóng bị GC thu gom
}

// Thay vào đó, tái sử dụng đối tượng
var person = new Person();
for (int i = 0; i < 100000; i++)
{
    person.Reset(); // Tái sử dụng đối tượng mà không tạo mới
}
```

11. Sử dụng cấu trúc dữ liệu hiệu quả (Memory Pooling)

Sử dụng `ArrayPool<T>` để tái sử dụng bộ nhớ thay vì tạo mới đối tượng mỗi lần.

```
using System.Buffers;

ArrayPool<byte> pool = ArrayPool<byte>.Shared;

// Lấy một mảng từ pool
byte[] buffer = pool.Rent(1024);

// Sử dụng mảng
buffer[0] = 1;
buffer[1] = 2;

// Trả lại mảng cho pool sau khi sử dụng
pool.Return(buffer);
```

12. Giảm thiểu việc sử dụng các đối tượng có kích thước lớn

Tránh sử dụng các đối tượng lớn trong Large Object Heap (LOH) vì chúng có thể gây ra vấn đề phân mảnh bộ nhớ.

```
// Tránh sử dụng các mảng lớn (ví dụ: kích thước lớn hơn 85,000 byte)
byte[] largeArray = new byte[100000]; // Đây sẽ nằm trong LOH

// Thay vào đó, chia nhỏ mảng hoặc dùng Object Pooling
List<byte[]> smallArrays = new List<byte[]>();
for (int i = 0; i < 100; i++)
{
    smallArrays.Add(new byte[1000]); // Tạo các mảng nhỏ hơn
}
```

13. Giảm độ phân mảnh của heap

Sắp xếp lại bộ nhớ một cách hợp lý để GC dễ dàng giải phóng bộ nhớ không sử dụng.

```
// Tránh tạo ra nhiều đối tượng ngắn hạn mà GC phải xử lý liên tục
for (int i = 0; i < 100000; i++)
{
    var temp = new MyClass();
}

// Thay vào đó, sử dụng Object Pooling hoặc tái sử dụng các đối tượng
```

14. Sử dụng **Dispose** đúng cách

Đảm bảo giải phóng tài nguyên khi không còn sử dụng đến, ví dụ như khi làm việc với tài nguyên ngoại vi (files, kết nối cơ sở dữ liệu, v.v).

```
public class MyResource : IDisposable
{
    private bool disposed = false;
    private FileStream stream;
```

```
public MyResource(string filePath)
{
    stream = new FileStream(filePath, FileMode.Open);
}

public void Dispose()
{
    if (!disposed)
    {
        stream?.Dispose();
        disposed = true;
    }
}

using (var resource = new MyResource("file.txt"))
{
    // Sử dụng tài nguyên
}
// Sau khi thoát khỏi khối using, Dispose được gọi tự động
```

15. Sử dụng WeakReference

Khi bạn muốn giữ tham chiếu đến đối tượng mà không làm tăng tuổi thọ của nó, **WeakReference** là lựa chọn tốt.

```
WeakReference<MyClass> weakRef = new WeakReference<MyClass>(new MyClass());

// Kiểm tra nếu đối tượng còn sống
if (weakRef.TryGetTarget(out var myClass))
{
    // Sử dụng đối tượng
}
else
{
    // Đối tượng đã bị GC thu gom
}
```

16. Tránh việc cấp phát bộ nhớ quá thường xuyên

Khi cần nối chuỗi, thay vì sử dụng **+**, bạn nên dùng **StringBuilder** để giảm thiểu việc tạo mới các đối tượng chuỗi liên tục.

```
using System.Text;

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++)
{
    sb.Append("Hello");
    sb.Append(" World");
}

string result = sb.ToString();
```

17. Tối ưu hóa với MemoryManagement API (GC.Collect)

Gọi `GC.Collect()` để thu gom bộ nhớ khi bạn chắc chắn rằng không còn đối tượng nào trong bộ nhớ, tuy nhiên cần sử dụng thận trọng vì nó có thể ảnh hưởng đến hiệu suất.

```
// Gọi GC.Collect chỉ khi thực sự cần thiết
GC.Collect(); // Dừng toàn bộ ứng dụng để thực hiện thu gom
```

18. Công cụ hỗ trợ GC

Dùng Visual Studio Diagnostic Tools để theo dõi việc thu gom và phát hiện bộ nhớ rò rỉ.

19. Lập trình dựa trên cơ chế GC

Cần hiểu cách GC hoạt động và tránh các hành vi gây áp lực cho GC.

```
// Chờ GC thu gom tự động sau khi các đối tượng không còn sử dụng
for (int i = 0; i < 1000000; i++)
{
    var temp = new MyClass();
    if (i % 10000 == 0)
    {
        // Chờ GC tự thu gom
        GC.Collect();
    }
}
```

20. Kết luận

Thông qua các ví dụ trên, bạn có thể tối ưu hóa việc quản lý bộ nhớ và giảm thiểu tác động tiêu cực đến hiệu suất của ứng dụng C#. Các kỹ thuật này sẽ giúp bạn giảm thiểu sự phân mảnh bộ nhớ, tối ưu hóa GC, và cải thiện hiệu suất tổng thể của ứng dụng.

3.3. Using IDisposable and the `using` statement.

Trong C#, việc quản lý tài nguyên là rất quan trọng để đảm bảo rằng các tài nguyên như bộ nhớ, kết nối mạng, kết nối cơ sở dữ liệu và các tài nguyên hệ thống khác được giải phóng đúng cách sau khi sử dụng. Để làm được điều này, C# cung cấp hai công cụ chính: **IDisposable** và câu lệnh **using**.

1. IDisposable Interface

IDisposable là một interface trong .NET được sử dụng để quản lý tài nguyên không phải là bộ nhớ (non-memory resources) như kết nối cơ sở dữ liệu, file, hoặc các đối tượng hệ thống khác. Interface này có một phương thức duy nhất:

```
public interface IDisposable
{
    void Dispose();
}
```

- **Phương thức `Dispose()`**: Phương thức này được gọi khi đối tượng không còn được sử dụng, nhằm giải phóng các tài nguyên không quản lý được bộ nhớ (ví dụ: tệp, kết nối cơ sở dữ liệu, vv).

2. Sử Dụng `Dispose()`

Khi một lớp triển khai **IDisposable**, phương thức `Dispose()` sẽ chứa logic để giải phóng tài nguyên.

Ví dụ:

```
public class FileReader : IDisposable
{
    private FileStream _fileStream;

    public FileReader(string filePath)
    {
        _fileStream = new FileStream(filePath, FileMode.Open);
    }

    public void ReadFile()
    {
        // Logic đọc tệp
    }

    public void Dispose()
    {
        // Giải phóng tài nguyên khi không còn sử dụng
        _fileStream?.Dispose();
    }
}
```

Trong ví dụ trên, **FileReader** sử dụng `Dispose` để giải phóng tài nguyên `_fileStream` khi đối tượng không còn được sử dụng.

3. Câu Lệnh `using`

Câu lệnh `using` là cách thuận tiện để đảm bảo tài nguyên được giải phóng tự động. Câu lệnh này yêu cầu đối tượng phải triển khai interface `IDisposable`. Câu lệnh `using` sẽ đảm bảo rằng phương thức `Dispose()` được gọi ngay cả khi xảy ra lỗi.

Cấu trúc cơ bản của câu lệnh `using`:

```
using (var resource = new FileReader("file.txt"))
{
    resource.ReadFile();
}
```

Khi ra khỏi khối `using`, phương thức `Dispose()` sẽ được gọi tự động, giúp giải phóng tài nguyên.

4. Ví Dụ Cụ Thể

```
using System;
using System.IO;
using System.IO.Pipelines;

class Program
{
    static void Main()
    {
        // Sử dụng câu lệnh using để quản lý tài nguyên
        using (var fileReader = new FileReader("example.txt"))
        {
            fileReader.ReadFile();
        } // Tài nguyên sẽ được giải phóng ở đây

        Console.WriteLine("File reading completed.");
    }
}
```

Trong ví dụ này, tài nguyên `fileReader` sẽ tự động được giải phóng khi ra khỏi khối `using`, đảm bảo rằng file không bị bỏ quên khi không còn sử dụng nữa.

5. Kết Hợp với try-catch-finally

Trong trường hợp bạn cần xử lý lỗi và vẫn đảm bảo rằng tài nguyên được giải phóng, bạn có thể kết hợp câu lệnh `using` với khối `try-catch-finally`:

```
FileReader resource = null;
try
{
    resource = new FileReader("file.txt");
    resource.ReadFile();
}
catch (Exception ex)
```

```
{
    Console.WriteLine("An error occurred: " + ex.Message);
}
finally
{
    if (resource != null)
    {
        resource.Dispose();
    }
}

// Dispose sẽ được gọi tự động ở đây, kể cả khi xảy ra lỗi
```

6. Ưu Điểm của `IDisposable` và `using`

- **Giảm Thiểu Lỗi Quản Lý Tài Nguyên:** Câu lệnh `using` giúp giảm thiểu lỗi khi quên gọi `Dispose()`.
- **Quản Lý Tài Nguyên Hiệu Quả:** Các tài nguyên như file, kết nối cơ sở dữ liệu được giải phóng đúng lúc, tránh rò rỉ bộ nhớ hoặc tài nguyên hệ thống.
- **Tự Động Giải Phóng:** Khi sử dụng `using`, việc giải phóng tài nguyên được thực hiện tự động mà không cần phải nhớ gọi `Dispose()` thủ công.

7. Kết Luận

Việc sử dụng `IDisposable` và câu lệnh `using` là rất quan trọng trong việc quản lý tài nguyên trong C#. Đây là một cách tiếp cận để đảm bảo rằng các tài nguyên không phải bộ nhớ được giải phóng đúng lúc, tránh gây ra lỗi hoặc tài nguyên không được giải phóng khi không còn cần thiết.

4. Multithreading and Parallel Programming

4.1. Basics of multithreading in C#.

1. Khái niệm cơ bản về Multithreading

Multithreading (đa luồng) là kỹ thuật trong lập trình cho phép một ứng dụng thực thi nhiều luồng (threads) đồng thời, giúp tối ưu hóa tài nguyên hệ thống và cải thiện hiệu suất khi xử lý các tác vụ tốn thời gian (như I/O, tính toán phức tạp). Mỗi luồng là một quá trình con có thể thực thi một phần của chương trình mà không làm gián đoạn các luồng khác.

2. Thread trong C#

Trong C#, bạn có thể tạo và quản lý các luồng bằng cách sử dụng lớp `Thread` trong không gian tên `System.Threading`. Một luồng có thể chạy song song với luồng chính (main thread), giúp thực thi các tác vụ đồng thời.

3. Tạo và chạy một Thread

Dưới đây là ví dụ đơn giản về cách tạo và chạy một luồng trong C#:

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        // Tạo một luồng mới
        Thread thread = new Thread(DoWork);
        thread.Start();

        // Tiếp tục thực hiện công việc trong luồng chính
        Console.WriteLine("Main thread is running...");

        // Đợi luồng mới kết thúc
        thread.Join();

        Console.WriteLine("Main thread finished.");
    }

    // Phương thức cho luồng phụ
    static void DoWork()
    {
        Console.WriteLine("Worker thread is running...");
        Thread.Sleep(2000); // Giả lập một công việc kéo dài 2 giây
        Console.WriteLine("Worker thread finished.");
    }
}
```

Giải thích:

- `Thread thread = new Thread(DoWork);` tạo một luồng mới, nơi `DoWork` là phương thức được thực thi trong luồng đó.
- `thread.Start();` bắt đầu thực thi luồng.
- `thread.Join();` đợi luồng phụ kết thúc trước khi chương trình tiếp tục.

4. Quản lý luồng bằng Task và async/await

C# hỗ trợ các kỹ thuật mới như Task và async/await để xử lý đa luồng hiệu quả hơn, giúp lập trình viên viết mã dễ dàng hơn và dễ bảo trì hơn.

Sử dụng Task:

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        // Tạo và chạy một task đồng thời
        Task task = Task.Run(() => DoWork());

        // Công việc trong luồng chính
        Console.WriteLine("Main task is running...");

        // Chờ task hoàn thành
        await task;

        Console.WriteLine("Main task finished.");
    }

    static void DoWork()
    {
        Console.WriteLine("Worker task is running...");
        Task.Delay(2000).Wait(); // Giả lập công việc kéo dài 2 giây
        Console.WriteLine("Worker task finished.");
    }
}
```

Giải thích:

- Task.Run() tạo và chạy một tác vụ không đồng bộ.
- await task; đảm bảo rằng phương thức Main sẽ đợi cho đến khi task hoàn thành.

5. Chạy nhiều tác vụ đồng thời

Bạn cũng có thể chạy nhiều tác vụ đồng thời với Task.WhenAll hoặc Task.WhenAny để đồng bộ hóa nhiều luồng.

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Task task1 = Task.Run(() => DoWork(1));
```

```
Task task2 = Task.Run(() => DoWork(2));

await Task.WhenAll(task1, task2);

Console.WriteLine("Both tasks completed.");
}

static void DoWork(int id)
{
    Console.WriteLine($"Worker {id} is running...");
    Task.Delay(2000).Wait(); // Giả lập công việc kéo dài 2 giây
    Console.WriteLine($"Worker {id} finished.");
}
}
```

6. Điều phối luồng với lock

Khi nhiều luồng truy cập cùng một tài nguyên, có thể xảy ra tình trạng "race condition", gây lỗi hoặc kết quả không mong muốn. Để tránh điều này, bạn có thể sử dụng từ khóa **lock** để bảo vệ các tài nguyên chia sẻ:

```
using System;
using System.Threading;

class Program
{
    static int counter = 0;
    static object lockObject = new object();

    static void Main()
    {
        Thread t1 = new Thread(IncrementCounter);
        Thread t2 = new Thread(IncrementCounter);

        t1.Start();
        t2.Start();

        t1.Join();
        t2.Join();

        Console.WriteLine($"Final counter value: {counter}");
    }

    static void IncrementCounter()
    {
        for (int i = 0; i < 1000; i++)
        {
            lock (lockObject)
            {
                counter++;
            }
        }
    }
}
```

Giải thích:

- **lock (lockObject)** đảm bảo chỉ một luồng có thể truy cập vào đoạn mã bên trong trong mỗi thời điểm, ngăn chặn tình trạng đồng thời truy cập tài nguyên chia sẻ.

7. Tóm tắt

- Đa luồng giúp xử lý các tác vụ đồng thời, tăng hiệu suất của ứng dụng.
- C# cung cấp các công cụ như **Thread**, **Task**, và **async/await** để làm việc với đa luồng.
- Sử dụng **lock** để đảm bảo an toàn khi nhiều luồng cùng truy cập tài nguyên chia sẻ.

Bài giảng này sẽ giúp bạn nắm bắt được các khái niệm và công cụ cơ bản để làm việc với đa luồng trong C#, từ đó tối ưu hóa hiệu suất của ứng dụng và cải thiện trải nghiệm người dùng.

4.2. Parallel programming and the Task Parallel Library (TPL).

1. Giới thiệu về Lập trình Song Song

- **Lập trình song song (Parallel Programming)** là một kỹ thuật cho phép thực thi đồng thời nhiều tác vụ hoặc công việc, giúp tăng hiệu suất và giảm thời gian xử lý cho các tác vụ tốn kém về tài nguyên tính toán.
- Trong C#, lập trình song song có thể thực hiện dễ dàng nhờ vào thư viện TPL.

2. Giới thiệu về Task Parallel Library (TPL)

- **TPL** là một thư viện mạnh mẽ được cung cấp trong .NET để dễ dàng thực thi các tác vụ song song và bất đồng bộ.
- TPL giúp xử lý các tác vụ song song thông qua lớp **Task**, giúp chia nhỏ công việc và tối ưu hóa hiệu suất.
- **Task** là một đại diện cho một đơn vị công việc mà bạn có thể chạy song song.

3. So sánh Multithreading vs Parallel Computing

Tiêu chí	Multithreading	Parallel Computing
Mục đích	Chạy nhiều luồng để cải thiện phản hồi	Tăng tốc xử lý bằng cách sử dụng nhiều CPU
Cách hoạt động	Luồng chạy xen kẽ, có thể bị tạm dừng	Các tác vụ chạy thực sự song song trên nhiều lõi CPU
Quản lý	Cần tạo và quản lý luồng thủ công	Tự động tối ưu hóa nhờ Task Parallel Library
Ứng dụng	UI, xử lý IO, tác vụ nền	Tính toán hiệu suất cao, xử lý dữ liệu lớn
Ví dụ API	<code>Thread</code> , <code>ThreadPool</code> , <code>Task</code>	<code>Parallel.For</code> , <code>Parallel.Invoke</code> , <code>PLINQ</code>

Khi nào dùng cái nào?

- **Dùng đa luồng (Multithreading) khi:**
 - Cần duy trì tính tương tác của ứng dụng (UI, xử lý nền).
 - Có các tác vụ chạy đồng thời nhưng không yêu cầu tối ưu CPU.
- **Dùng lập trình song song (Parallel Computing) khi:**
 - Cần xử lý khối lượng dữ liệu lớn, tính toán nặng.
 - Muốn tận dụng hết tài nguyên CPU.

4. Các lớp chính trong TPL

Task: Đại diện cho một công việc đơn lẻ sẽ thực thi song song.

Ví dụ:

```
Task task = Task.Run(() => { Console.WriteLine("Hello, Parallel World!"); });
task.Wait();
```

Task<T>: Tương tự như `Task`, nhưng có thể trả về một giá trị sau khi công việc hoàn thành.

Ví dụ:

```
Task<int> task = Task.Run(() => { return 42; });
int result = task.Result;
Console.WriteLine(result); // 42
```

Parallel: Cung cấp phương thức để thực thi các vòng lặp song song.

Parallel.For: Thực thi vòng lặp song song, xử lý nhiều phần tử cùng lúc

Tình huống: Bạn có một danh sách số nguyên và cần tính bình phương của từng số. Sử dụng `Parallel.For` giúp chạy song song trên nhiều CPU để tăng tốc xử lý.

```
int[] numbers = { 1, 2, 3, 4, 5 };

Parallel.For(0, numbers.Length, i =>
{
    int squared = numbers[i] * numbers[i];

    Console.WriteLine($"Số {numbers[i]} có bình phương: {squared} - Thread: {Task.CurrentId}");
});

Console.WriteLine("Tất cả số đã được tính toán.");
```

♦ **Khi nào dùng?** Khi bạn có một danh sách chỉ mục tuần tự và muốn xử lý từng phần tử song song.

Parallel.ForEach: Tương tự như `Parallel.For`, nhưng với một tập hợp (collection).

Tình huống: Bạn có một danh sách URLs và muốn tải nội dung của từng trang web song song để tiết kiệm thời gian.

```
List<string> urls = new List<string>
{
    "https://example.com",
    "https://dotnet.microsoft.com",
    "https://learn.microsoft.com"
};

HttpClient client = new HttpClient();

await Task.Run(() =>
{
    Parallel.ForEach(urls, async url =>
    {
        string content = await client.GetStringAsync(url);
    });
});
```

```
Console.WriteLine($"Tải xong: {url} - Kích thước: {content.Length}");

});

});
```

```
Console.WriteLine("Tất cả trang web đã tải xong.");
```

5. Quản lý Song song với `Task.WhenAll` và `Task.WhenAny`

Task.WhenAll: Dùng để đợi tất cả các tác vụ hoàn thành.

Ví dụ tình huống: Bạn có nhiều API cần gọi và muốn đợi tất cả hoàn thành trước khi tiếp tục.

```
HttpClient client = new HttpClient();

Task<string> task1 = client.GetStringAsync("https://example.com");

Task<string> task2 = client.GetStringAsync("https://dotnet.microsoft.com");

Task<string> task3 = client.GetStringAsync("https://learn.microsoft.com");

string[] results = await Task.WhenAll(task1, task2, task3);

Console.WriteLine($"Tất cả API đã phản hồi. Kết quả 1: {results[0].Length}, Kết quả 2: {results[1].Length}, Kết quả 3: {results[2].Length}");
```

Task.WhenAny: Xử lý ngay khi có tác vụ hoàn thành

Tình huống: Bạn có nhiều API cần gọi nhưng chỉ cần lấy kết quả của API nào trả về nhanh nhất.

```
HttpClient client = new HttpClient();

Task<string> task1 = client.GetStringAsync("https://example.com");

Task<string> task2 = client.GetStringAsync("https://dotnet.microsoft.com");

Task<string> task3 = client.GetStringAsync("https://learn.microsoft.com");

Task<string> firstCompletedTask = await Task.WhenAny(task1, task2, task3);

Console.WriteLine($"API phản hồi nhanh nhất có kết quả dài {firstCompletedTask.Result.Length} ký tự.");
```

Tóm tắt so sánh

Tên	Mô tả	Khi nào dùng?
-----	-------	---------------

<code>Parallel.For</code>	Lặp qua danh sách chỉ mục tuần tự một cách song song.	Khi xử lý dữ liệu theo chỉ mục (mảng, danh sách).
<code>Parallel.ForEach</code>	Duyệt qua danh sách dữ liệu song song.	Khi xử lý danh sách không theo thứ tự.
<code>Task.WhenAll</code>	Chờ tất cả tác vụ hoàn thành.	Khi cần lấy tất cả kết quả trước khi tiếp tục.
<code>Task.WhenAny</code>	Lấy kết quả của tác vụ nhanh nhất.	Khi chỉ cần một kết quả đầu tiên.

6. Quản lý Ngoại lệ trong TPL

- Trong TPL, nếu một tác vụ gặp lỗi, ngoại lệ đó sẽ không được ném ra ngoài ngay lập tức mà sẽ bị bao bọc trong một **AggregateException**.
- Để xử lý ngoại lệ, bạn cần sử dụng phương thức **Task.Wait()** hoặc **Task.WhenAll()**.

Ví dụ:

```
try
{
    Task task = Task.Run(() =>
    {
        throw new InvalidOperationException("Something went wrong.");
    });
    task.Wait();
}
catch (AggregateException ex)
{
    foreach (var e in ex.InnerExceptions)
    {
        Console.WriteLine(e.Message);
    }
}
```

7. Lợi ích và Thách thức khi sử dụng TPL

- Lợi ích:**
 - Tăng tốc độ xử lý với việc sử dụng đa lõi CPU.

- TPL giúp dễ dàng chia nhỏ công việc mà không cần phải tự quản lý thread.
- Các phương thức trong TPL giúp tăng tính dễ đọc và bảo trì cho mã nguồn.
- **Thách thức:**
 - Quản lý tài nguyên hạn chế (CPU, bộ nhớ).
 - Đồng bộ hóa dữ liệu khi nhiều tác vụ cùng truy cập vào tài nguyên chung.

8. Kết luận

- TPL là một công cụ mạnh mẽ trong C# giúp lập trình viên dễ dàng làm việc với các tác vụ song song và bất đồng bộ.
- Việc sử dụng TPL hợp lý có thể giúp cải thiện hiệu suất đáng kể trong các ứng dụng yêu cầu xử lý song song, nhưng cần phải cẩn trọng với việc đồng bộ hóa và quản lý ngoại lệ.

4.3. Thread safety and synchronization.

1. Thread Safety (Thread-Safety) là gì?

- **Thread safety** có nghĩa là khi một đối tượng hoặc tài nguyên có thể được truy cập và sử dụng bởi nhiều luồng cùng một lúc mà không gây ra lỗi hoặc sự không nhất quán trong dữ liệu.
- Khi một đối tượng không thread-safe, cần phải thực hiện các biện pháp bảo vệ như khóa (locks) để đảm bảo rằng chỉ một luồng có thể truy cập tài nguyên đó tại một thời điểm.

2. Synchronization (Đồng Bộ Hóa)

- **Synchronization** là quá trình đồng bộ các luồng để đảm bảo rằng tài nguyên chung (chẳng hạn như biến, mảng, hoặc đối tượng) không bị truy cập đồng thời bởi nhiều luồng, tránh tình trạng race condition.
- Có nhiều cách để đồng bộ hóa trong C#, ví dụ như sử dụng **lock**, **Monitor**, **Mutex**, **Semaphore**, hoặc **ReaderWriterLock**.

3. Các Cơ Chế Synchronization trong C#

lock: Đây là cách đơn giản và phổ biến nhất để đảm bảo tính thread-safe trong C#. Nó sử dụng một đối tượng để đồng bộ hóa các luồng truy cập vào mã nguồn.

```
using System.Threading;

private static readonly object _lock = new object();

public void SomeMethod()
{
    lock (_lock)
```

```
{  
    // Code được đồng bộ hóa  
}  
}
```

Monitor: Cung cấp các tính năng đồng bộ hóa giống như **lock**, nhưng cung cấp khả năng kiểm soát cao hơn, như là việc sử dụng **Monitor.Enter()** và **Monitor.Exit()**.

```
using System.Threading;  
  
public void SomeMethod()  
{  
    Monitor.Enter(_lock);  
    try  
    {  
        // Code được đồng bộ hóa  
    }  
    finally  
    {  
        Monitor.Exit(_lock);  
    }  
}
```

Mutex: Cung cấp đồng bộ hóa giữa các tiến trình (process), không chỉ giữa các luồng trong một tiến trình. Thường dùng cho các ứng dụng đa tiến trình.

```
using (Mutex mutex = new Mutex())  
{  
    mutex.WaitOne();  
    // Code được đồng bộ hóa  
    mutex.ReleaseMutex();  
}
```

Semaphore: Quản lý số lượng tối đa các luồng có thể truy cập một tài nguyên đồng thời.

```
Semaphore semaphore = new Semaphore(3, 3); // Cho phép tối đa 3 luồng truy cập  
semaphore.WaitOne();  
// Code được đồng bộ hóa  
semaphore.Release();
```

ReaderWriterLock: Cho phép nhiều luồng đọc tài nguyên đồng thời nhưng chỉ cho phép một luồng ghi tài nguyên.

```
ReaderWriterLockSlim rwLock = new ReaderWriterLockSlim();  
rwLock.EnterReadLock();  
// Đọc tài nguyên  
rwLock.ExitReadLock();
```

4. Race Condition và Deadlock

- **Race Condition** xảy ra khi hai hoặc nhiều luồng cùng truy cập và sửa đổi dữ liệu chung mà không có cơ chế bảo vệ, dẫn đến kết quả không thể đoán trước.
- **Deadlock** xảy ra khi hai hoặc nhiều luồng bị chặn lẫn nhau, không thể tiếp tục vì mỗi luồng đều đang chờ tài nguyên mà luồng kia giữ.

5. Ví Dụ Thực Tế

- **Ví dụ 1:** Cập nhật số dư tài khoản ngân hàng trong môi trường đa luồng.
Giả sử bạn có một tài khoản ngân hàng và nhiều luồng (threads) đang thực hiện giao dịch rút tiền. Nếu không đồng bộ hóa, có thể xảy ra tình trạng race condition dẫn đến số dư bị tính toán sai.

```
using System;

using System.Threading;

class BankAccount
{
    private int balance = 1000; // Số dư ban đầu

    private object lockObject = new object();

    public void Withdraw(int amount)
    {
        lock (lockObject) // Đảm bảo chỉ một luồng có thể thực hiện giao dịch tại một thời
        điểm
        {
            if (balance >= amount)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} đang rút {amount}...");

                Thread.Sleep(100); // Giả lập độ trễ xử lý

                balance -= amount;

                Console.WriteLine($"{Thread.CurrentThread.Name} đã rút thành công! Số dư còn lại:
                {balance}");
            }
        }
    }
}
```

```
        else
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} không thể rút {amount}. Số dư không
đủ!");
        }
    }
}

class Program
{
    static void Main()
    {
        BankAccount account = new BankAccount();

        Thread t1 = new Thread(() => account.Withdraw(700)) { Name = "Luồng 1" };
        Thread t2 = new Thread(() => account.Withdraw(500)) { Name = "Luồng 2" };
        Thread t3 = new Thread(() => account.Withdraw(300)) { Name = "Luồng 3" };

        t1.Start();
        t2.Start();
        t3.Start();

        t1.Join();
        t2.Join();
        t3.Join();

        Console.WriteLine("Giao dịch hoàn tất.");
    }
}
```

- **Ví dụ 2:** Quản lý các tài nguyên chia sẻ giữa các luồng bằng cách sử dụng **Monitor**. **Monitor** tương tự như **lock** nhưng cho phép kiểm soát linh hoạt hơn.

```
using System;

using System.Threading;

class SharedResource
{
    private static object lockObject = new object();

    public static void AccessResource(string threadName)
    {
        bool lockTaken = false;

        try
        {
            Monitor.Enter(lockObject, ref lockTaken);

            Console.WriteLine($"{threadName} đang truy cập tài nguyên...");

            Thread.Sleep(500);

            Console.WriteLine($"{threadName} đã hoàn thành.");
        }
        finally
        {
            if (lockTaken)
                Monitor.Exit(lockObject);
        }
    }
}

class Program
{
    static void Main()
```



```
{  
  
    Thread t1 = new Thread(() => SharedResource.AccessResource("Luồng 1"));  
  
    Thread t2 = new Thread(() => SharedResource.AccessResource("Luồng 2"));  
  
    Thread t3 = new Thread(() => SharedResource.AccessResource("Luồng 3"));  
  
    t1.Start();  
  
    t2.Start();  
  
    t3.Start();  
  
    t1.Join();  
  
    t2.Join();  
  
    t3.Join();  
  
    Console.WriteLine("Truy cập tài nguyên hoàn tất.");  
  
}
```

So sánh **lock** và **Monitor**

Tiêu chí	lock	Monitor
Dễ sử dụng	Dễ	Cần dùng Monitor.Enter và Monitor.Exit
Kiểm soát chi tiết	Ít linh hoạt	Linh hoạt hơn, có thể kiểm tra trước khi khóa
Hiệu suất	Tương tự Monitor	Tương tự lock , nhưng hữu ích hơn trong xử lý phức tạp

6. Lời khuyên khi làm việc với Thread Safety và Synchronization

- Hạn chế sử dụng **lock** và các cơ chế đồng bộ hóa không cần thiết vì chúng có thể ảnh hưởng đến hiệu suất.

- Cẩn thận với việc sử dụng quá nhiều cơ chế đồng bộ hóa, vì chúng có thể dẫn đến tình trạng deadlock hoặc giảm hiệu suất.
- Sử dụng các công cụ như `ConcurrentQueue`, `ConcurrentDictionary` nếu có thể, vì chúng đã được tối ưu cho việc làm việc với luồng.

7. Tóm tắt

- Đảm bảo tính thread-safe trong C# rất quan trọng khi làm việc với ứng dụng đa luồng.
- C# cung cấp nhiều cơ chế đồng bộ hóa như `lock`, `Monitor`, `Mutex`, và `Semaphore` để giúp quản lý các luồng.
- Cẩn thận với race condition và deadlock khi làm việc với các luồng đồng thời.

Hy vọng bài giảng này sẽ giúp học viên nắm vững cách quản lý và đồng bộ hóa các luồng trong C# để viết các ứng dụng mạnh mẽ và an toàn.

5. Design Patterns in C#

5.1. Common design patterns (Singleton, Factory, Observer, etc.).

Mục tiêu bài học

1. Hiểu các mẫu thiết kế phổ biến và lý do sử dụng chúng.
2. Biết cách triển khai các mẫu thiết kế trong C#.
3. Áp dụng các mẫu thiết kế vào các tình huống thực tế.

Nội dung bài học

I. Giới thiệu về Design Patterns

- Design Patterns là gì?

Design Pattern là các giải pháp thiết kế đã được kiểm chứng để giải quyết các vấn đề phổ biến trong phát triển phần mềm. Chúng giúp viết mã dễ bảo trì, tái sử dụng và mở rộng bằng cách cung cấp các mẫu thiết kế có cấu trúc rõ ràng.

Tại sao cần dùng Design Pattern?

- **Tái sử dụng:** Tránh phát minh lại cách giải quyết một vấn đề đã có giải pháp tối ưu.
- **Dễ bảo trì:** Giảm sự phụ thuộc giữa các thành phần trong hệ thống.

- **Mở rộng dễ dàng:** Cho phép bổ sung tính năng mới mà không ảnh hưởng đến các phần khác.
- Phân loại các nhóm Design Patterns:

1. Creational Patterns (Nhóm khởi tạo) – Kiểm soát quá trình tạo đối tượng.

- **Factory Method:** Cung cấp một phương thức để tạo đối tượng mà không cần chỉ rõ lớp cụ thể.
- **Abstract Factory:** Tạo ra một nhóm đối tượng liên quan mà không cần xác định lớp cụ thể.
- **Builder:** Tách quá trình tạo đối tượng phức tạp thành các bước nhỏ có thể tùy chỉnh.
- **Prototype:** Tạo đối tượng mới bằng cách sao chép một đối tượng hiện có.
- **Singleton:** Đảm bảo chỉ có một thể hiện (instance) duy nhất của lớp trong suốt vòng đời chương trình.

2. Structural Patterns (Nhóm cấu trúc) – Xây dựng mối quan hệ giữa các lớp và đối tượng.

- **Adapter (Wrapper):** Chuyển đổi giao diện của một lớp thành giao diện khác mà client mong đợi.
- **Bridge:** Tách phần giao diện và phần triển khai để có thể thay đổi độc lập.
- **Composite:** Nhóm các đối tượng theo cấu trúc cây để xử lý chúng như một thể thống nhất.
- **Decorator:** Mở rộng chức năng của đối tượng mà không cần thay đổi cấu trúc bên trong.
- **Facade:** Cung cấp một giao diện đơn giản để truy cập vào một hệ thống phức tạp.
- **Flyweight:** Giảm sử dụng bộ nhớ bằng cách chia sẻ trạng thái chung giữa nhiều đối tượng.
- **Proxy:** Cung cấp một đại diện (proxy) để kiểm soát truy cập vào một đối tượng khác.

3. Behavioral Patterns (Nhóm hành vi) – Định nghĩa cách đối tượng giao tiếp và phân chia trách nhiệm.

- **Chain of Responsibility:** Chuyển yêu cầu qua một chuỗi các đối tượng xử lý cho đến khi có một đối tượng xử lý được.
- **Command:** Biến một yêu cầu thành một đối tượng độc lập, giúp dễ dàng lưu trữ, xếp hàng, và hoàn tác.
- **Interpreter:** Xây dựng trình thông dịch để xử lý cú pháp ngôn ngữ đặc biệt.

- **Iterator**: Cung cấp cách để duyệt qua các phần tử của một tập hợp mà không lộ ra chi tiết bên trong.
 - **Mediator**: Điều phối giao tiếp giữa các đối tượng để giảm sự phụ thuộc trực tiếp giữa chúng.
 - **Memento**: Lưu trạng thái của đối tượng để có thể khôi phục sau này mà không vi phạm tính đóng gói.
 - **Observer**: Một đối tượng tự động thông báo cho các đối tượng khác khi có thay đổi trạng thái.
 - **State**: Cho phép đối tượng thay đổi hành vi khi trạng thái nội bộ của nó thay đổi.
 - **Strategy**: Định nghĩa một họ thuật toán và cho phép thay đổi thuật toán khi chạy.
 - **Template Method**: Xác định bộ khung thuật toán và cho phép các bước cụ thể được ghi đè trong lớp con.
 - **Visitor**: Cho phép thêm chức năng vào đối tượng mà không cần thay đổi lớp của nó.
 - **Null Object**: Tránh xử lý giá trị null bằng cách cung cấp một đối tượng mặc định có hành vi rỗng.
- Tại sao nên sử dụng Design Patterns?
 - Tăng khả năng tái sử dụng mã nguồn.
 - Dễ bảo trì và mở rộng.
 - Giúp thiết kế phần mềm dễ đọc và hiểu.
-

II. Các Design Patterns phổ biến

1. Singleton Pattern

- **Mục đích**: Đảm bảo một lớp chỉ có duy nhất một thể hiện (instance) và cung cấp điểm truy cập toàn cục.
- **Ví dụ thực tế**: Quản lý cấu hình ứng dụng, logging.

Triển khai trong C#:

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> instance =
        new Lazy<Singleton>(() => new Singleton());

    private Singleton() { }

    public static Singleton Instance => instance.Value;
}
```

- **Ưu điểm**:
 - Kiểm soát được số lượng instance.

- Dễ dàng truy cập toàn cục.
 - **Nhược điểm:**
 - Khó kiểm tra (unit test).
 - Cần cẩn thận trong môi trường đa luồng.
-

2. Factory Pattern

- **Mục đích:** Tạo đối tượng mà không để lộ logic khởi tạo.
- **Ví dụ thực tế:** Tạo đối tượng theo các điều kiện khác nhau (như loại tài liệu, loại báo cáo).

Triển khai trong C#:

```
public interface IProduct
{
    string GetDetails();
}

public class ProductA : IProduct
{
    public string GetDetails() => "Product A";
}

public class ProductB : IProduct
{
    public string GetDetails() => "Product B";
}

public class Factory
{
    public static IProduct CreateProduct(string type)
    {
        return type switch
        {
            "A" => new ProductA(),
            "B" => new ProductB(),
            _ => throw new ArgumentException("Invalid type")
        };
    }
}
```

- **Ưu điểm:**
 - Giảm sự phụ thuộc vào các lớp cụ thể.
 - Dễ dàng thêm các loại sản phẩm mới.
 - **Nhược điểm:**
 - Có thể dẫn đến quá tải lớp Factory nếu quá nhiều loại sản phẩm.
-

3. Observer Pattern

- **Mục đích:** Định nghĩa mối quan hệ phụ thuộc một-nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc được thông báo.
- **Ví dụ thực tế:** Hệ thống sự kiện (event), hệ thống thông báo (notification).

Triển khai trong C#:

```
public class Subject
{
    private readonly List<IObserver> observers = new();

    public void Attach(IObserver observer) => observers.Add(observer);
    public void Detach(IObserver observer) => observers.Remove(observer);

    public void Notify()
    {
        foreach (var observer in observers)
        {
            observer.Update();
        }
    }
}

public interface IObserver
{
    void Update();
}

public class ConcreteObserver : IObserver
{
    private readonly string name;

    public ConcreteObserver(string name) => this.name = name;

    public void Update() => Console.WriteLine($"{name} has been notified!");
}
```

- **Ưu điểm:**
 - Giảm sự liên kết giữa các đối tượng.
 - Dễ dàng thêm các observer mới.
- **Nhược điểm:**
 - Có thể gây vấn đề về hiệu suất nếu có quá nhiều observer.

III. So sánh và áp dụng

1. **Singleton vs. Factory:**
 - Singleton dùng để quản lý một đối tượng duy nhất, Factory dùng để tạo nhiều loại đối tượng khác nhau.
2. **Khi nào sử dụng Observer?:**

- Khi có nhiều thành phần cần phản ứng lại sự thay đổi từ một nguồn duy nhất.
-

IV. Bài tập thực hành

1. **Singleton:**
 - Tạo một lớp quản lý cấu hình ứng dụng bằng Singleton.
 2. **Factory:**
 - Viết chương trình tạo các loại tài liệu như PDF, Word bằng Factory Pattern.
 3. **Observer:**
 - Tạo ứng dụng đơn giản: một Subject thông báo khi có thay đổi dữ liệu, các Observer nhận thông báo.
-

V. Kết luận

- Các Design Patterns giúp tăng hiệu quả phát triển phần mềm.
- Nắm vững và áp dụng đúng Design Patterns vào các tình huống phù hợp sẽ cải thiện chất lượng phần mềm.

5.2. Practical applications and benefits.

Mục tiêu bài học

1. Hiểu rõ cách ứng dụng các design pattern phổ biến trong các tình huống thực tế.
 2. Nắm được các lợi ích mà design pattern mang lại, bao gồm:
 - Tăng khả năng tái sử dụng mã nguồn.
 - Tăng cường khả năng bảo trì và mở rộng hệ thống.
 - Giảm thiểu rủi ro và lỗi phát sinh khi phát triển phần mềm.
 3. Thực hành với một số ví dụ minh họa thực tế.
-

Nội dung chính

1. **Tổng quan về Design Pattern**
 - Nhắc lại khái niệm về design pattern.
 - Phân loại: Creational, Structural, Behavioral.
2. **Ứng dụng thực tiễn**

- **Singleton Pattern**

Ứng dụng: Quản lý kết nối cơ sở dữ liệu hoặc cấu hình hệ thống.

- **Ví dụ:** Tạo một lớp quản lý cấu hình toàn cục, đảm bảo chỉ có một thể hiện duy nhất.

Ví dụ: Trong một hệ thống, có một class quản lý cấu hình hệ thống (System Configuration) và đảm bảo chỉ có một instance duy nhất để truy xuất.

```
public class ConfigurationManager
{
    private static ConfigurationManager? _instance;

    private static readonly object _lock = new object();

    public string DatabaseConnectionString { get; private set; }

    private ConfigurationManager()
    {
        // Giả sử đây là nơi tải cấu hình từ file hoặc biến môi trường

        DatabaseConnectionString =
            "Server=myServer;Database=myDB;User=myUser;Password=myPassword;";
    }

    public static ConfigurationManager Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new ConfigurationManager();
                }

                return _instance;
            }
        }
    }
}
```



```
        }  
    }  
}  
  
internal class Program  
{  
  
    private static void Main(string[] args)  
    {  
  
        var config = ConfigurationManager.Instance;  
  
        Console.WriteLine(config.DatabaseConnectionString);  
  
    }  
}
```

Factory Method Pattern

Ứng dụng: Một hệ thống cần tạo tài khoản người dùng với nhiều loại tài khoản khác nhau (Admin, Guest, Customer).

Cách triển khai:

- **User** là lớp cơ sở.
- Các lớp **AdminUser**, **GuestUser**, **CustomerUser** kế thừa từ **User**.
- **UserFactory** quyết định loại tài khoản cần tạo.

```
public abstract class User  
{  
  
    public abstract void GetRole();  
  
}  
  
public class AdminUser : User  
{  
  
    public override void GetRole() => Console.WriteLine("Admin User - Full Access");  
  
}
```

```
}

public class GuestUser : User
{
    public override void GetRole() => Console.WriteLine("Guest User - Limited Access");
}

public class CustomerUser : User
{
    public override void GetRole() => Console.WriteLine("Customer User - Standard Access");
}

public class UserFactory
{
    public static User CreateUser(string userType)
    {
        return userType switch
        {
            "Admin" => new AdminUser(),
            "Guest" => new GuestUser(),
            "Customer" => new CustomerUser(),
            _ => throw new ArgumentException("Invalid user type"),
        };
    }
}

User admin = UserFactory.CreateUser("Admin");
admin.GetRole(); // Output: Admin User - Full Access

User guest = UserFactory.CreateUser("Guest");
guest.GetRole(); // Output: Guest User - Limited Access
```

Observer Pattern

Một trang thương mại điện tử muốn thông báo cho khách hàng khi có sản phẩm mới trong danh mục mà họ quan tâm.

Cách triển khai:

- **ProductCategory** đóng vai trò **Subject**.
- **Customer** là **Observer** đăng ký nhận thông báo khi có sản phẩm mới.

```
public interface IObservable
{
    void Update(string productName);
}

public class Customer : IObservable
{
    public string Name { get; }

    public Customer(string name)
    {
        Name = name;
    }

    public void Update(string productName)
    {
        Console.WriteLine($"{Name} nhận được thông báo: Sản phẩm mới - {productName}");
    }
}
```

```
public class ProductCategory
{
    private List<IObserver> observers = new List<IObserver>();

    public void AddObserver(IObserver observer) => observers.Add(observer);

    public void RemoveObserver(IObserver observer) => observers.Remove(observer);

    public void NotifyObservers(string productName)
    {
        foreach (var observer in observers)
        {
            observer.Update(productName);
        }
    }

    public void AddNewProduct(string productName)
    {
        Console.WriteLine($"Sản phẩm mới: {productName} vừa được thêm!");
        NotifyObservers(productName);
    }
}
```

```
ProductCategory electronics = new ProductCategory();
```

```
Customer customer1 = new Customer("Alice");
```

```
Customer customer2 = new Customer("Bob");
```

```
electronics.AddObserver(customer1);
```

```
electronics.AddObserver(customer2);
```

```
electronics.AddNewProduct("iPhone 15 Pro");
```

// Output:

// Sản phẩm mới: iPhone 15 Pro vừa được thêm!

// Alice nhận được thông báo: Sản phẩm mới - iPhone 15 Pro

// Bob nhận được thông báo: Sản phẩm mới - iPhone 15 Pro

3. Lợi ích của việc sử dụng Design Pattern

- **Tăng tính modular:** Dễ dàng thay đổi hoặc mở rộng từng phần của hệ thống.
- **Tăng tính nhất quán:** Các nhà phát triển khác dễ dàng hiểu cấu trúc hệ thống hơn.
- **Giảm rủi ro và lỗi logic:** Áp dụng các giải pháp đã được kiểm chứng.
- **Cải thiện hiệu suất nhóm phát triển:** Chuẩn hóa cách giải quyết vấn đề.

4. Thực hành minh họa

- Tạo một ứng dụng nhỏ sử dụng các pattern đã học.
- **Đề xuất bài tập:** Xây dựng một hệ thống quản lý giao hàng:
 - Sử dụng Singleton để quản lý cấu hình.
 - Sử dụng Factory Method để tạo các đối tượng giao hàng khác nhau (nhẹ, tiêu chuẩn).
 - Sử dụng Observer để cập nhật trạng thái đơn hàng cho khách hàng.

Tổng kết

- Design pattern là công cụ mạnh mẽ giúp cải thiện chất lượng phần mềm.
- Việc hiểu và áp dụng chúng một cách phù hợp sẽ giúp dự án phần mềm trở nên dễ bảo trì, linh hoạt và giảm rủi ro.

Bài tập

1. Xây dựng một ứng dụng console sử dụng ít nhất 2 design pattern đã học.
2. Đánh giá lợi ích của từng pattern trong ứng dụng vừa xây dựng.

5.3. Choosing the right pattern for different scenarios.

Mục tiêu bài học:

- Hiểu cách phân tích và lựa chọn mẫu thiết kế (Design Pattern) phù hợp dựa trên yêu cầu thực tế.
 - Hiểu cách áp dụng các mẫu thiết kế phổ biến trong các kịch bản cụ thể.
 - Nắm được ưu điểm và nhược điểm của từng mẫu thiết kế để sử dụng hiệu quả.
-

Nội dung chi tiết

1. Tổng quan về Design Patterns

- **Design Pattern là gì?**
Một giải pháp tái sử dụng cho các vấn đề phổ biến trong thiết kế phần mềm.
 - **Phân loại Design Patterns:**
 - **Creational Patterns:** Tạo đối tượng (Singleton, Factory, Builder).
 - **Structural Patterns:** Cấu trúc đối tượng (Adapter, Composite, Decorator).
 - **Behavioral Patterns:** Hành vi đối tượng (Strategy, Observer, State).
-

2. Phân tích yêu cầu và xác định mẫu phù hợp

- **Kịch bản 1:** Cần quản lý phiên bản duy nhất của một đối tượng.
 - **Giải pháp: Singleton Pattern.**
 - **Ví dụ thực tế:**
 - Quản lý kết nối cơ sở dữ liệu (DB Connection Pool).
 - Quản lý máy in trong một hệ thống in ấn tập trung.
 - Trình quản lý cấu hình ứng dụng (Application Configuration Manager).
- **Kịch bản 2:** Cần tạo các đối tượng linh hoạt mà không phụ thuộc trực tiếp vào lớp cụ thể.
 - **Giải pháp: Factory Method Pattern hoặc Abstract Factory Pattern.**
 - **Ví dụ thực tế:**
 - Hệ thống thanh toán hỗ trợ nhiều phương thức như PayPal, Stripe, và Credit Card.
 - Hệ thống gửi thông báo (Email, SMS, Push Notification).
 - Hệ thống quản lý tài khoản ngân hàng với nhiều loại tài khoản khác nhau.
- **Kịch bản 3:** Cần thay đổi hành vi của đối tượng mà không thay đổi cấu trúc của nó.

- **Giải pháp: Strategy Pattern** hoặc **State Pattern**.
 - **Ví dụ thực tế:** Cách tính giá cước vận chuyển trong các khu vực khác nhau.
 - **Kịch bản 4:** Cần tích hợp với hệ thống bên ngoài có giao diện khác biệt.
 - **Giải pháp: Adapter Pattern**.
 - **Ví dụ thực tế:**
 - Kết nối hệ thống kế toán cũ với hệ thống ERP mới.
 - Tích hợp cổng thanh toán mới vào một hệ thống thương mại điện tử cũ.
 - Chuyển đổi API từ REST sang GraphQL để tương thích với client mới.
-

3. Ưu điểm và nhược điểm của các mẫu thiết kế

- **Singleton:**
 - *Ưu điểm:* Dễ triển khai, đảm bảo một đối tượng duy nhất.
 - *Nhược điểm:* Khó kiểm tra đơn vị (UT), có thể gây vấn đề trong môi trường đa luồng.
 - **Factory:**
 - *Ưu điểm:* Tăng tính linh hoạt, dễ mở rộng.
 - *Nhược điểm:* Tăng phức tạp trong việc triển khai.
 - **Strategy:**
 - *Ưu điểm:* Dễ mở rộng hành vi, giảm sự phụ thuộc giữa các lớp.
 - *Nhược điểm:* Tăng số lượng lớp, cần quản lý các đối tượng chiến lược.
-

4. Thực hành áp dụng Design Pattern

- **Bài tập thực hành:**

Viết chương trình áp dụng **Strategy Pattern** để tính thuế VAT theo các quốc gia khác nhau.

 - Quốc gia 1: VAT = 10%.
 - Quốc gia 2: VAT = 15%.
 - Quốc gia 3: VAT = 20%.
-

5. Kết luận

- Không có mẫu thiết kế nào là "hoàn hảo" cho mọi trường hợp. Hãy dựa vào yêu cầu cụ thể để lựa chọn mẫu phù hợp.

- Thực hành nhiều tình huống thực tế sẽ giúp bạn thành thạo trong việc áp dụng Design Patterns.

Tài liệu tham khảo:

- Sách *"Design Patterns: Elements of Reusable Object-Oriented Software"*.
- Các tài nguyên từ Microsoft Docs và Refactoring Guru.

5.4. Nguyên lý SOLID

Nguyên lý SOLID là tập hợp 5 nguyên tắc thiết kế phần mềm, giúp mã dễ bảo trì, mở rộng và ít xảy ra lỗi hơn. Chúng đặc biệt hữu ích trong lập trình hướng đối tượng. Hãy cùng tìm hiểu từng nguyên lý và ví dụ minh họa trong **C#**.

1. Single Responsibility Principle (SRP) - Nguyên lý đơn trách nhiệm

Mỗi lớp chỉ nên có một lý do để thay đổi, tức là mỗi lớp chỉ đảm nhiệm một trách nhiệm duy nhất.

Ví dụ:

Sai phạm SRP:

```
public class Invoice
{
    public void CalculateTotal() { /* Tính tổng hóa đơn */ }
    public void PrintInvoice() { /* In hóa đơn */ }
    public void SaveToDatabase() { /* Lưu vào cơ sở dữ liệu */ }
}
```

Cách đúng: Tách trách nhiệm.

```
public class Invoice
{
    public void CalculateTotal() { /* Tính tổng hóa đơn */ }
}

public class InvoicePrinter
{
    public void PrintInvoice(Invoice invoice) { /* In hóa đơn */ }
}

public class InvoiceRepository
{
    public void SaveToDatabase(Invoice invoice) { /* Lưu hóa đơn vào cơ sở dữ liệu */ }
}
```



```
}
```

2. Open/Closed Principle (OCP) - Nguyên lý mở/đóng

Lớp nên mở để mở rộng nhưng đóng để sửa đổi. Nghĩa là bạn có thể thêm chức năng mới mà không cần chỉnh sửa mã hiện có.

Ví dụ:

Sai phạm OCP:

```
public class DiscountCalculator
{
    public double CalculateDiscount(string customerType)
    {
        if (customerType == "Regular") return 0.1;
        if (customerType == "VIP") return 0.2;
        return 0.0;
    }
}
```

Cách đúng: Sử dụng **kế thừa** hoặc **interface** để mở rộng.

```
public interface IDiscount
{
    double GetDiscount();
}

public class RegularCustomerDiscount : IDiscount
{
    public double GetDiscount() => 0.1;
}

public class VIPCustomerDiscount : IDiscount
{
    public double GetDiscount() => 0.2;
}

public class DiscountCalculator
{
    public double CalculateDiscount(IDiscount discount) => discount.GetDiscount();
}
```

3. Liskov Substitution Principle (LSP) - Nguyên lý thay thế Liskov

Các lớp con có thể thay thế lớp cha mà không làm thay đổi tính đúng đắn của chương trình.

Ví dụ:

Sai phạm LSP:

```
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
    public int Area => Width * Height;
}

public class Square : Rectangle
{
    public override int Width
    {
        set { base.Width = base.Height = value; }
    }
    public override int Height
    {
        set { base.Width = base.Height = value; }
    }
}
```

Cách đúng: Không nên kế thừa khi đặc tính không giống nhau.

```
public interface IShape
{
    int Area { get; }
}

public class Rectangle : IShape
{
    public int Width { get; set; }
    public int Height { get; set; }
    public int Area => Width * Height;
}

public class Square : IShape
{
    public int Side { get; set; }
    public int Area => Side * Side;
}
```

4. Interface Segregation Principle (ISP) - Nguyên lý phân tách Interface

Không nên ép buộc một lớp thực thi các phương thức mà nó không sử dụng. Hãy chia nhỏ interface.

Ví dụ:

Sai phạm ISP:

```
public interface IWorker
{
    void Work();
    void Eat();
}

public class Robot : IWorker
{
    public void Work() { /* Làm việc */ }
    public void Eat() { throw new NotImplementedException(); }
}
```

Cách đúng: Tách interface.

```
public interface IWorker
{
    void Work();
}

public interface IEater
{
    void Eat();
}

public class Human : IWorker, IEater
{
    public void Work() { /* Làm việc */ }
    public void Eat() { /* Ăn uống */ }
}

public class Robot : IWorker
{
    public void Work() { /* Làm việc */ }
}
```

5. Dependency Inversion Principle (DIP) - Nguyên lý đảo ngược phụ thuộc

Các module cấp cao không nên phụ thuộc vào module cấp thấp, mà cả hai nên phụ thuộc vào abstraction (trừu tượng).

Ví dụ:

Sai phạm DIP:

```
public class EmailService
{
```

```
        public void SendEmail(string message) { /* Gửi email */ }
    }

    public class Notification
    {
        private EmailService _emailService = new EmailService();
        public void Send(string message)
        {
            _emailService.SendEmail(message);
        }
    }
}
```

Cách đúng: Sử dụng **interface** để đảo ngược phụ thuộc.

```
public interface IMessageService
{
    void SendMessage(string message);
}

public class EmailService : IMessageService
{
    public void SendMessage(string message) { /* Gửi email */ }
}

public class SmsService : IMessageService
{
    public void SendMessage(string message) { /* Gửi SMS */ }
}

public class Notification
{
    private readonly IMessageService _messageService;
    public Notification(IMessageService messageService)
    {
        _messageService = messageService;
    }

    public void Send(string message)
    {
        _messageService.SendMessage(message);
    }
}
```

Kết luận:

Áp dụng nguyên lý SOLID giúp mã dễ đọc, mở rộng và bảo trì. Trong C#, các nguyên lý này thường được kết hợp với các công cụ như Dependency Injection (DI), Interface, và Design Patterns (như Strategy, Factory, Adapter). Hãy áp dụng chúng linh hoạt để cải thiện chất lượng mã của bạn!

6. Unit Testing and Test-Driven Development (TDD)

6.1. Introduction to unit testing frameworks (e.g., MSTest, NUnit, xUnit).

1. Kiểm thử đơn vị là gì?

- **Định nghĩa:**
Kiểm thử đơn vị (Unit Testing) là một kỹ thuật trong phát triển phần mềm dùng để kiểm tra các đơn vị nhỏ nhất của chương trình (thường là các hàm hoặc phương thức).
- **Mục tiêu:**
Xác minh rằng một đoạn mã cụ thể hoạt động đúng theo yêu cầu.
- **Lợi ích:**
 - Phát hiện lỗi sớm trong quá trình phát triển.
 - Giúp mã dễ bảo trì hơn (refactor mà không sợ phá vỡ logic).
 - Tăng độ tin cậy khi triển khai phần mềm.

2. Tổng quan về các framework kiểm thử đơn vị

Các framework kiểm thử đơn vị giúp tự động hóa việc kiểm tra mã. Một số framework phổ biến trong hệ sinh thái .NET:

- **MSTest:**
 - Framework kiểm thử đơn vị chính thức của Microsoft.
 - Tích hợp sâu với Visual Studio.
 - Cấu trúc rõ ràng, phù hợp với các dự án doanh nghiệp.
- **NUnit:**
 - Framework độc lập, linh hoạt, và mạnh mẽ.
 - Cộng đồng hỗ trợ rộng rãi.
 - Hỗ trợ nhiều tính năng như Data-Driven Testing.
- **xUnit:**
 - Một framework hiện đại và được sử dụng rộng rãi.
 - Đơn giản, dễ sử dụng, và tối ưu cho các dự án mới.
 - Được sử dụng bởi .NET Core và các dự án mã nguồn mở.

3. So sánh MSTest, NUnit và xUnit

Đặc điểm	MSTest	NUnit	xUnit
Nguồn gốc	Microsoft	Độc lập	Độc lập
Hỗ trợ .NET Core	Có	Có	Có
Tính năng chính	Tích hợp Visual Studio	Data-Driven, Assert đa dạng	Hiện đại, tối ưu, khả năng tùy chỉnh cao
Phổ biến	Trung bình	Cao	Cao

4. Kiến trúc cơ bản của một bài kiểm thử đơn vị

- Một bài kiểm thử đơn vị gồm:
 - Setup (Arrange):** Chuẩn bị dữ liệu hoặc cấu hình cần thiết.
 - Action (Act):** Thực thi phương thức cần kiểm tra.
 - Assert:** Kiểm tra kết quả có đúng như mong đợi.

Ví dụ:

```
[TestMethod] // MSTest
public void Add_ShouldReturnSum_WhenTwoNumbersAreProvided()
{
    // Arrange
    var calculator = new Calculator();
    int a = 2, b = 3;

    // Act
    int result = calculator.Add(a, b);

    // Assert
    Assert.AreEqual(5, result);
}
```

5. Cài đặt và cấu hình

- MSTest:**
 - Tích hợp sẵn trong Visual Studio.

Thêm package nếu cần:

```
dotnet add package MSTest.TestFramework
```

- NUnit:**

Cài đặt qua NuGet:

```
dotnet add package NUnit
```

- Thêm **NUnit3TestAdapter** để tích hợp với Visual Studio.

- **xUnit:**

Cài đặt qua NuGet:

```
dotnet add package xunit
```

- Tích hợp tốt với .NET Core bằng lệnh `dotnet test`.

6. Demo cơ bản với xUnit

Tạo một project kiểm thử:

```
dotnet new xunit -n UnitTestProject
```

Viết một bài kiểm thử:

```
public class CalculatorTests
{
    [Fact]
    public void Add_ShouldReturnCorrectSum()
    {
        // Arrange
        var calculator = new Calculator();
        int a = 1, b = 2;

        // Act
        var result = calculator.Add(a, b);

        // Assert
        Assert.Equal(3, result);
    }
}
```

7. Kết luận

- Lựa chọn framework kiểm thử phù hợp dựa trên yêu cầu dự án và sự quen thuộc của nhóm phát triển.
- Kiểm thử đơn vị là một kỹ năng quan trọng giúp tăng chất lượng phần mềm, và sử dụng framework phù hợp sẽ tối ưu hóa quá trình kiểm thử.

Bài tập

1. Cài đặt một trong ba framework (MSTest, NUnit hoặc xUnit) và viết bài kiểm thử cho một phương thức đơn giản như `Add`, `Subtract`.

2. Tìm hiểu thêm về các thuộc tính đặc biệt như `[Theory]` trong xUnit hoặc `[TestCase]` trong NUnit.

6.2. Writing and running tests.

Mục tiêu

- Hiểu cách viết và chạy unit test trong C#.
 - Sử dụng framework phổ biến **xUnit** để kiểm thử.
 - Thực hành viết unit test cho một hàm cụ thể.
-

1. Giới thiệu về Unit Test

- **Unit Test:** Kiểm tra một đơn vị nhỏ nhất của phần mềm (thường là một hàm hoặc phương thức).
 - Mục đích:
 - Đảm bảo chức năng hoạt động đúng như mong đợi.
 - Phát hiện lỗi sớm trong quá trình phát triển.
 - Cải thiện chất lượng mã nguồn.
-

2. Cài đặt xUnit

2.1. Thêm xUnit vào dự án

1. Mở **Visual Studio**.
2. Tạo một **Test Project** hoặc thêm thư viện kiểm thử riêng biệt:
 - Chọn **File > New > Project > xUnit Test Project (.NET Core)**.
3. Nếu dự án chính cần thêm thư viện xUnit:
 - Mở **NuGet Package Manager**.

Cài đặt các package:

`Install-Package xUnit`

`Install-Package xUnit.runner.visualstudio`

- **Microsoft.Extensions.DependencyInjection** (nếu cần mô phỏng DI - Dependency Injection).

2.2. Cấu trúc thư mục

- Tạo thư mục **Tests** cùng cấp với dự án chính.
 - Đặt tên cho test project theo cấu trúc: **[TênDựÁn].Tests**.
-

3. Viết Unit Test

3.1. Một ví dụ đơn giản

Hàm cần kiểm thử: Tính tổng hai số nguyên.

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

Test case:

```
using Xunit;

public class CalculatorTests
{
    [Fact]
    public void Add_ReturnsCorrectSum()
    {
        // Arrange
        var calculator = new Calculator();
        int a = 2, b = 3;

        // Act
        var result = calculator.Add(a, b);

        // Assert
        Assert.Equal(5, result);
    }
}
```

3.2. Tạo nhiều trường hợp kiểm thử

Sử dụng **[Theory] và **[InlineData]**:**

```
public class CalculatorTests
{
    [Theory]
```

```
[InlineData(2, 3, 5)]
[InlineData(-1, -1, -2)]
[InlineData(0, 0, 0)]
public void Add_ReturnsCorrectSum_MultipleCases(int a, int b, int expected)
{
    // Arrange
    var calculator = new Calculator();

    // Act
    var result = calculator.Add(a, b);

    // Assert
    Assert.Equal(expected, result);
}
}
```

4. Chạy Unit Test

4.1. Dùng Test Explorer

1. Mở **Test Explorer** từ Visual Studio:
Test > Windows > Test Explorer.
2. Nhấn nút **Run All** để chạy toàn bộ kiểm thử.

4.2. Dùng lệnh `dotnet test`

- Mở **Command Prompt** hoặc **Terminal** tại thư mục gốc dự án.

Chạy lệnh:

```
dotnet test
```

- Kết quả hiển thị danh sách test thành công hoặc thất bại.
-

5. Phương pháp hay khi viết Unit Test

- **AAA Pattern** (Arrange, Act, Assert):
 - **Arrange**: Chuẩn bị dữ liệu đầu vào.
 - **Act**: Gọi phương thức cần kiểm thử.
 - **Assert**: Kiểm tra kết quả đầu ra.
 - Kiểm tra từng trường hợp riêng biệt (Test độc lập).
 - Đặt tên hàm test mô tả rõ ràng:
`MethodName_Condition_ExpectedResult`.
-

6. Nâng cao

6.1. Mock và Stub

Sử dụng **Moq** để giả lập đối tượng phụ thuộc:

```
var mockDependency = new Mock<IMyDependency>();  
mockDependency.Setup(m => m.SomeMethod()).Returns("Mocked Result");
```

6.2. Test Asynchronous Methods

```
[Fact]  
public async Task AsyncMethod_ReturnsExpectedResult()  
{  
    // Arrange  
    var service = new MyService();  
  
    // Act  
    var result = await service.GetDataAsync();  
  
    // Assert  
    Assert.NotNull(result);  
}
```

Bài tập thực hành

- Viết unit test kiểm tra hàm tính **factorial** (giai thừa) với các trường hợp:
 - $n = 0$, $n = 1$, và $n > 1$.
- Sử dụng **[Theory]** để kiểm tra hàm kiểm tra số nguyên tố.

6.3. Fundamentals of TDD in C# projects.

1. Giới thiệu về Test-Driven Development (TDD)

- **TDD là gì?**
 - Là một phương pháp phát triển phần mềm dựa trên việc viết test trước khi viết code chính.
 - Chu trình phổ biến: **Red -> Green -> Refactor**.
 - **Lợi ích của TDD:**
 - Tăng độ tin cậy của code.
 - Giảm thiểu bug trong quá trình phát triển.
 - Hỗ trợ thiết kế code tốt hơn (clean code).
-

2. Chu trình TDD: Red -> Green -> Refactor

- **Bước 1: Red (Viết Test không chạy được)**
 - Viết một test đơn giản nhất đại diện cho yêu cầu.
 - Test sẽ thất bại vì chưa có logic thực thi.
 - **Bước 2: Green (Viết đủ code để test chạy được)**
 - Viết code tối thiểu cần thiết để pass test.
 - Không quan tâm tới tối ưu hoá ở bước này.
 - **Bước 3: Refactor (Tối ưu hóa code và test)**
 - Làm sạch code, cải tiến mà không làm fail test.
-

3. Cấu trúc một bài test trong C#: AAA

- **Arrange:** Chuẩn bị dữ liệu cần thiết cho test.
- **Act:** Thực thi hành động hoặc logic cần kiểm tra.
- **Assert:** Xác minh kết quả đầu ra có khớp với kỳ vọng hay không.

Ví dụ:

```
[TestMethod]
public void Add_WhenGivenTwoNumbers_ReturnsSum()
{
    // Arrange
    var calculator = new Calculator();
    var num1 = 3;
    var num2 = 5;

    // Act
    var result = calculator.Add(num1, num2);

    // Assert
    Assert.AreEqual(8, result);
}
```

4. Các công cụ TDD phổ biến trong C#

- **xUnit:**
 - Framework hiện đại, nhanh chóng, và phổ biến.
 - **NUnit:**
 - Hỗ trợ linh hoạt với nhiều tính năng mở rộng.
 - **MSTest:**
 - Framework tích hợp sẵn trong Visual Studio.
-

5. Áp dụng TDD trong dự án C#

- **Bắt đầu với một bài toán nhỏ:**
 - Phát triển từng phần nhỏ của logic, không viết cả chức năng phức tạp ngay từ đầu.
 - **Công cụ hỗ trợ:**
 - Visual Studio hoặc JetBrains Rider.
 - **Mocking Frameworks:** Moq hoặc NSubstitute để giả lập dữ liệu hoặc hành vi.
-

6. Lưu ý quan trọng khi áp dụng TDD

- Viết test đơn giản, rõ ràng, có thể đọc hiểu dễ dàng.
 - Không nên viết quá nhiều test phức tạp cho một chức năng nhỏ.
 - Sử dụng Continuous Integration (CI) để đảm bảo tất cả test luôn pass trước khi deploy.
-

7. Bài tập thực hành

1. **Tạo một class `StringCalculator`:**
 - Tính tổng của các số trong chuỗi đầu vào, ví dụ: "`1,2,3`" => `6`.
 2. **Viết các test case với TDD:**
 - Chuỗi rỗng trả về `0`.
 - Một số đơn lẻ trả về chính số đó.
 - Nhiều số cách nhau bởi dấu `,` trả về tổng của chúng.
 - Chấp nhận các ký tự không phải số, bỏ qua chúng.
-

8. Kết luận

- TDD không chỉ giúp cải thiện chất lượng phần mềm mà còn định hình cách viết code tốt hơn.
- TDD cần thời gian làm quen nhưng sẽ mang lại hiệu quả dài hạn khi áp dụng đúng cách.

7. Advanced .NET Features

7.1. Dependency Injection (DI) and Inversion of Control (IoC).

1. Mục tiêu bài học

- Hiểu rõ khái niệm **Dependency Inversion Principle (DIP)**, **Dependency Injection** và **Inversion of Control**.
 - Biết cách sử dụng DI trong .NET để giảm sự phụ thuộc và cải thiện tính linh hoạt, bảo trì của ứng dụng.
 - Tìm hiểu cách cấu hình IoC Container trong ASP.NET Core.
 - Áp dụng DI trong thực tế với ví dụ cụ thể.
-

2. Nội dung bài học

2.1. Dependency Inversion Principle (DIP) - Nguyên tắc đảo ngược sự phụ thuộc trong SOLID

DIP là nguyên tắc **D** trong SOLID, và nó định nghĩa rằng:

1. **Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào abstraction (giao diện hoặc lớp trừu tượng).**
2. **Abstraction không nên phụ thuộc vào chi tiết. Chi tiết nên phụ thuộc vào abstraction.**

DIP nhằm **tách biệt các module cấp cao và cấp thấp** bằng cách sử dụng abstraction (interface hoặc abstract class), giúp mã dễ mở rộng và bảo trì hơn.

Kết luận: Nó chỉ là **một nguyên tắc thiết kế** trong SOLID, không liên quan trực tiếp đến cách quản lý dependencies.

2.2. Inversion of Control (IoC)

- **Định nghĩa:**
 - IoC là **một nguyên tắc rộng hơn** nhằm đảo ngược quyền kiểm soát việc khởi tạo và quản lý dependencies.
 - Thay vì một class tự tạo dependencies của nó, một container hoặc framework sẽ làm điều đó.
 - IoC thường được thực hiện bằng **Dependency Injection (DI)**.

Ví dụ về IoC:Trong ASP.NET Core, IoC Container quản lý vòng đời của các đối tượng (services):

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddScoped<ILogger, FileLogger>(); // IoC Container quản lý dependency injection
```

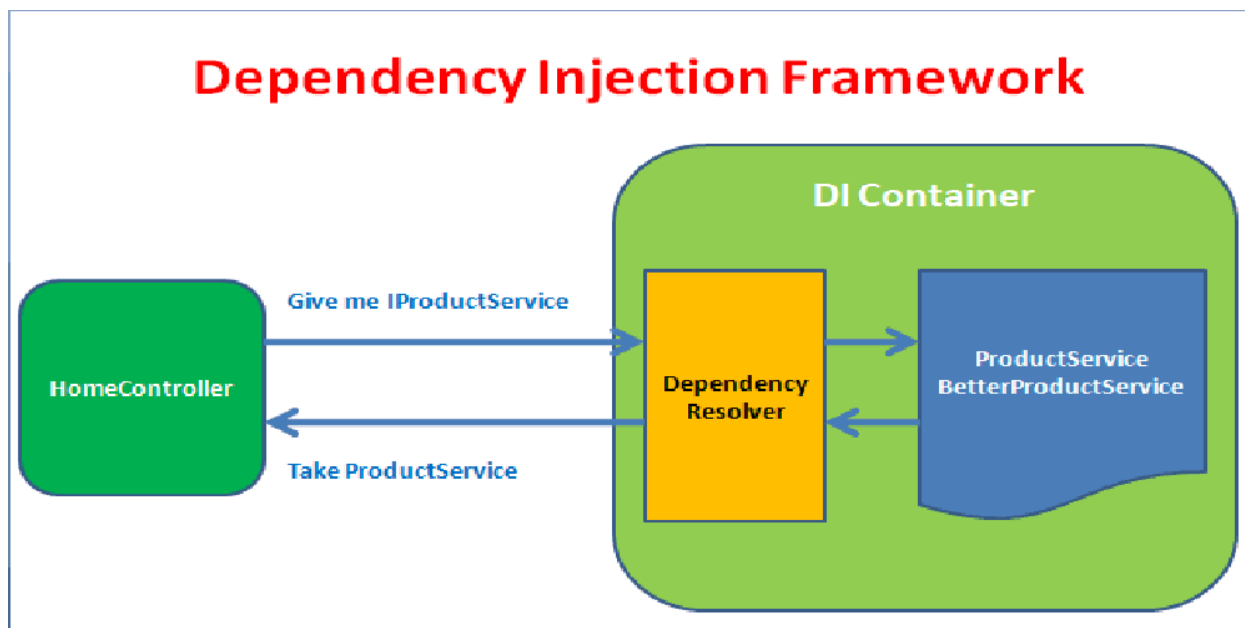
```
builder.Services.AddScoped<OrderService>();
```

```
var app = builder.Build();
```

Sự khác nhau giữa DIP và IoC

Tiêu chí	Dependency Inversion Principle (DIP)	Inversion of Control (IoC)
Bản chất	Một nguyên tắc trong SOLID giúp giảm sự phụ thuộc giữa các module bằng abstraction.	Một nguyên tắc thiết kế giúp kiểm soát dependencies bằng cách đảo ngược luồng điều khiển.
Mục tiêu	Giảm sự phụ thuộc trực tiếp giữa các module cấp cao và cấp thấp.	Chuyển quyền kiểm soát việc tạo và quản lý dependencies ra khỏi class.
Cách thực hiện	Sử dụng interface hoặc abstract class để tách biệt các module.	Thường được thực hiện bằng Dependency Injection (DI) và IoC Container.
Liên quan đến DI?	Không trực tiếp, nhưng thường đi kèm với DI.	DI là một cách để thực hiện IoC.

2.3. Dependency Injection là gì?



- **Định nghĩa:** DI là một design pattern trong lập trình mà ở đó các dependency (phụ thuộc) của một đối tượng được cung cấp từ bên ngoài thay vì đối tượng tự tạo hoặc quản lý chúng.
- **Lợi ích:**
 - Giảm sự phụ thuộc chặt chẽ (tight coupling) giữa các lớp.
 - Dễ dàng thay thế hoặc mở rộng module.
 - Hỗ trợ unit testing dễ dàng hơn.

✅ **DI (Dependency Injection) → Mẫu thiết kế (design pattern)** giúp triển khai cả **DIP** (bằng abstraction) và **IoC** (bằng cách inject dependencies thay vì để class tự tạo).

Nói cách khác:

- **DIP** là mục tiêu (giảm phụ thuộc cứng).
- **IoC** là cách tiếp cận (đảo ngược quyền kiểm soát dependencies).
- **DI** là công cụ để thực hiện cả hai (inject dependencies thay vì tự tạo).

Ví dụ:

```
public class Car
{
    private Engine _engine;

    public Car(Engine engine) // Dependency Injection qua constructor
    {
        _engine = engine;
    }

    public void Start()
```



```
{  
    _engine.Run();  
}  
}
```

2.4. DI Containers

- DI Container giúp:
 - Quản lý vòng đời của dependency (Transient, Scoped, Singleton).
 - Tự động resolve các dependency khi cần.

Các DI Container phổ biến trong .NET:

- Built-in container trong ASP.NET Core.
 - Autofac, Ninject, Unity.
-

3. Các loại Dependency Injection

1. **Constructor Injection** (Phổ biến nhất)
 - Cung cấp dependency qua constructor.

```
public class Car  
{  
    private readonly IEngine _engine;  
  
    public Car(IEngine engine)  
    {  
        _engine = engine;  
    }  
  
    public void Start()  
    {  
        _engine.Run();  
    }  
}
```

2. **Property Injection**
 - Dependency được inject qua property.

```
public class Car  
{  
    public IEngine Engine { get; set; }  
  
    public void Start()  
    {  
        Engine.Run();  
    }  
}
```

3. Method Injection

- Dependency được truyền vào qua method.

```
public class Car
{
    public void Start(IEngine engine)
    {
        engine.Run();
    }
}
```

4. DI trong ASP.NET Core

Đăng ký dịch vụ trong **Program.cs**:

```
var builder = WebApplication.CreateBuilder(args);

// Đăng ký service
builder.Services.AddTransient<IEngine, PetrolEngine>();

var app = builder.Build();

app.Run();
```

Inject dịch vụ vào Controller:

```
public class CarController : ControllerBase
{
    private readonly IEngine _engine1;
    private readonly IEngine _engine2;

    public CarController(IEngine engine1, IEngine engine2)
    {
        _engine1 = engine1;
        _engine2 = engine2;
    }

    [HttpGet]
    public IActionResult StartCar()
    {
        _engine.Run();
        return Ok("Car started!");
    }
}
```

Vòng đời của Service:

1. **Transient**: Mỗi lần yêu cầu tạo một instance mới.
2. **Scoped**: Instance được chia sẻ trong suốt một request.
3. **Singleton**: Instance được tạo một lần duy nhất trong suốt vòng đời ứng dụng.

5. Bài tập thực hành

1. Tạo một ứng dụng ASP.NET Core Web API.
 2. Cấu hình các service như:
 - `ILogger` để log thông tin.
 - Một dịch vụ custom như `IService` và `Service`.
 3. Áp dụng DI để inject các service này vào Controller.
 4. Chạy và kiểm tra các vòng đời service khác nhau.
-

6. Kết luận

- DI và IoC là những kỹ thuật quan trọng trong phát triển phần mềm hiện đại.
- Áp dụng DI giúp cải thiện thiết kế code, giảm phụ thuộc và tăng tính bảo trì.
- ASP.NET Core cung cấp công cụ mạnh mẽ để quản lý DI một cách dễ dàng và hiệu quả.

7.2. Using attributes and annotations.

1. Tổng quan về Attributes trong C#

- **Attributes** là metadata được gắn vào code (classes, methods, properties, etc.) để cung cấp thông tin bổ sung cho runtime hoặc các công cụ khác.
- Các use case phổ biến:
 - **Đánh dấu trạng thái** (Obsolete, Serializable)
 - **Cấu hình** (Data Annotations trong Entity Framework)
 - **Kiểm tra runtime** (Custom Attributes).

Ví dụ:

```
[Obsolete("This method is deprecated. Use NewMethod instead.")]
public void OldMethod()
{
    Console.WriteLine("This is the old method.");
}
```

2. Các Attribute sẵn có trong C#

- **[Obsolete]**: Thông báo một method hoặc class đã lỗi thời.
- **[Serializable]**: Đánh dấu một class có thể được serialize.

- **[DebuggerStepThrough]**: Bỏ qua method/class khi debug.
- **[Conditional]**: Thực thi code dựa trên điều kiện.

Ví dụ:

```
[Serializable]
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

3. Annotations trong Entity Framework

- **Annotations** cung cấp cách định nghĩa validation hoặc cấu hình cho database thông qua code.
 - **[Required]**: Bắt buộc trường phải có giá trị.
 - **[StringLength]**: Giới hạn độ dài của chuỗi.
 - **[Key]**: Định nghĩa primary key.
 - **[ForeignKey]**: Định nghĩa quan hệ với bảng khác.

Ví dụ:

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;

public class Product
{
    [Key]
    public int Id { get; set; }

    [Required]
    [StringLength(100)]
    public string Name { get; set; }

    [ForeignKey("Category")]
    public int CategoryId { get; set; }

    public Category Category { get; set; }
}
```

4. Tạo Custom Attribute

- Bạn có thể tạo ra **Custom Attribute** để mở rộng chức năng theo nhu cầu.
- Tạo một class kế thừa từ **System.Attribute**.

Ví dụ:

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
public class MyCustomAttribute : Attribute
{
    public string Description { get; }

    public MyCustomAttribute(string description)
    {
        Description = description;
    }
}

[MyCustomAttribute("This is a custom attribute example.")]
public class ExampleClass
{
    public void Display()
    {
        Console.WriteLine("Example method with custom attribute.");
    }
}
```

5. Sử dụng Reflection để đọc Attributes

- Reflection cho phép bạn lấy thông tin từ **Attributes** trong runtime.

Ví dụ:

```
using System.Reflection.Metadata;

var type = typeof(ExampleClass);
var attributes = type.GetCustomAttributes(typeof(MyCustomAttribute), false);

foreach (MyCustomAttribute attr in attributes)
{
    Console.WriteLine($"Description: {attr.Description}");
}
```

6. Thực hành

1. Tạo một class có sử dụng **Obsolete** và **Serializable**.
 2. Tạo một bảng trong database với các Data Annotations (EF Core).
 3. Tự tạo một Custom Attribute và đọc nó bằng Reflection.
-

7. Kết luận

- Attributes và Annotations giúp làm cho code gọn gàng hơn và giảm bớt việc viết lặp lại logic.
- Hiểu rõ cách sử dụng Attributes sẽ giúp bạn tận dụng tối đa tiềm năng của C#.

7.3. Working with Expression Trees.

1. Giới thiệu Expression Trees

- **Expression Trees là gì?**
 - Là cấu trúc dữ liệu dạng cây đại diện cho code trong C# ở dạng biểu thức.
 - Hữu ích trong việc phân tích và xử lý biểu thức tại runtime.
 - Được sử dụng trong LINQ-to-SQL, Entity Framework, Dynamic LINQ, v.v.
 - **Namespace:** `System.Linq.Expressions`.
-

2. Lợi ích của Expression Trees

- Phân tích hoặc biên dịch lại mã tại runtime.
 - Tạo các biểu thức động.
 - Hiệu quả khi làm việc với các thư viện sử dụng LINQ-to-SQL hoặc ORM.
-

3. Cấu trúc Expression Trees

- **Các node trong cây biểu thức:**
 - **ParameterExpression:** Đại diện biến hoặc tham số.
 - **ConstantExpression:** Đại diện giá trị hằng số.
 - **BinaryExpression:** Đại diện các toán tử nhị phân (+, -, *, /).
 - **MethodCallExpression:** Gọi phương thức.
-

4. Tạo một Expression Tree

Ví dụ 1: Tạo biểu thức đơn giản `x + y`

```
using System;  
using System.Linq.Expressions;
```

```
class Program
{
    static void Main()
    {
        // Tạo parameter x và y
        ParameterExpression x = Expression.Parameter(typeof(int), "x");
        ParameterExpression y = Expression.Parameter(typeof(int), "y");

        // Tạo biểu thức x + y
        BinaryExpression body = Expression.Add(x, y);

        // Tạo Expression Tree
        Expression<Func<int, int, int>> expression = Expression.Lambda<Func<int,
int, int>>(body, x, y);

        // In ra biểu thức
        Console.WriteLine(expression);

        // Compile và chạy biểu thức
        var compiledExpression = expression.Compile();
        Console.WriteLine("Kết quả của 2 + 3: " + compiledExpression(2, 3));
    }
}
```

Kết quả:

```
(x, y) => (x + y)
Kết quả của 2 + 3: 5
```

5. Phân tích Expression Tree

- **Expression Visitors:**
 - Duyệt cây biểu thức để phân tích hoặc thay đổi.

Ví dụ:

```
using System.Linq.Expressions;

class CustomVisitor : ExpressionVisitor
{
    protected override Expression VisitBinary(BinaryExpression node)
    {
        Console.WriteLine($"Thăm toán tử: {node.NodeType}");
        return base.VisitBinary(node);
    }
}

var visitor = new CustomVisitor();
visitor.Visit(expression);
```

6. Ứng dụng thực tế

- **LINQ-to-SQL:**
 - Dịch Expression Tree thành câu lệnh SQL.
 - **Dynamic Query:**
 - Tạo các truy vấn động dựa trên điều kiện runtime.
 - **Xây dựng DSL (Domain Specific Language):**
 - Tạo ngôn ngữ tùy chỉnh để mô tả logic kinh doanh.
-

7. Bài tập thực hành

Ví dụ: Tạo bộ lọc động bằng Expression Tree

Giả sử bạn có một danh sách người dùng (**User**) và muốn xây dựng một bộ lọc động mà không cần viết sẵn điều kiện.

8. Tổng kết

- Expression Trees là một công cụ mạnh mẽ trong C#.
- Cần thiết cho các ứng dụng yêu cầu phân tích hoặc biên dịch lại mã tại runtime.
- Kết hợp hiệu quả với LINQ và các ORM framework.

8. Performance Optimization and Best Practices

8.1. Profiling and optimizing C# code.

Mục tiêu

- Hiểu khái niệm **profiling** và lý do cần thực hiện trong phát triển phần mềm.
 - Sử dụng các công cụ profiling để phân tích hiệu năng mã C#.
 - Tìm hiểu các phương pháp tối ưu hóa mã nguồn.
 - Thực hành áp dụng profiling và tối ưu hóa vào các bài toán thực tế.
-

Nội dung chi tiết

1. Giới thiệu về Profiling

- **Profiling là gì?**
 - Profiling là quá trình đo lường và phân tích hiệu năng ứng dụng, giúp xác định các phần của mã gây ra vấn đề về tốc độ hoặc bộ nhớ.
 - **Lợi ích của Profiling**
 - Tăng hiệu năng ứng dụng.
 - Giảm thiểu tài nguyên tiêu tốn.
 - Xác định các vấn đề tiềm ẩn như **memory leaks**, **CPU bottlenecks**, hoặc **I/O overheads**.
-

2. Các công cụ Profiling phổ biến

- **Visual Studio Profiler**
 - Hướng dẫn sử dụng Visual Studio Diagnostics Tools.
 - Phân tích CPU Usage và Memory Usage.
 - **dotTrace (JetBrains)**
 - Theo dõi hiệu năng ứng dụng .NET.
 - Cách xác định "hot path" trong mã.
 - **PerfView**
 - Công cụ nhẹ để thu thập và phân tích logs hiệu năng.
 - Cách sử dụng để phát hiện **GC (Garbage Collection)** và **thread contention**.
 - **BenchmarkDotNet**
 - Framework để thực hiện benchmark hiệu quả trên C#.
 - Cách tích hợp và đo lường mã.
-

3. Các kỹ thuật tối ưu hóa mã C#

- **Tối ưu hóa thuật toán và cấu trúc dữ liệu**
 - So sánh hiệu năng giữa List, Dictionary, và HashSet.
 - Cách giảm độ phức tạp thời gian từ $O(n^2)$ xuống $O(n \log n)$ hoặc thấp hơn.
- **Quản lý bộ nhớ**
 - Tối ưu hóa việc sử dụng bộ nhớ bằng cách:
 - Giảm thiểu việc tạo các object mới.
 - Sử dụng **struct** thay vì **class** trong một số trường hợp.
 - Kiểm soát và tối ưu hóa **Garbage Collector (GC)**.
- **Tối ưu hóa LINQ**

- Tránh sử dụng các phương thức tốn kém như `.ToList()` không cần thiết.
 - Tận dụng **deferred execution** trong LINQ.
 - **Parallel Programming và Asynchronous Code**
 - Tối ưu hiệu năng thông qua **Task Parallel Library (TPL)** hoặc **async/await**.
 - Giảm thiểu "thread blocking" với **I/O-bound operations**.
 - **Caching**
 - Áp dụng caching tại client-side và server-side.
 - Sử dụng **MemoryCache** hoặc các công cụ caching như **Redis**.
-

4. Thực hành: Profiling và tối ưu hóa mã

- **Bài toán mẫu 1: Phân tích hiệu năng**
 - Tạo ứng dụng tính tổng các số nguyên lớn.
 - Sử dụng Visual Studio Profiler để xác định các điểm nghẽn.
 - **Bài toán mẫu 2: Tối ưu hóa LINQ**
 - Tối ưu hóa các truy vấn LINQ phức tạp trên tập dữ liệu lớn.
 - **Bài toán mẫu 3: Sử dụng BenchmarkDotNet**
 - Viết benchmark để so sánh hiệu năng của **List** và **Dictionary**.
-

5. Tổng kết

- Tầm quan trọng của việc kết hợp profiling và tối ưu hóa mã nguồn.
- Các công cụ và kỹ thuật cần có trong việc phát triển phần mềm hiệu quả.
- Tích hợp profiling vào quy trình CI/CD để phát hiện sớm các vấn đề.

8.2. Tips for memory and performance optimization.

1. Hiểu về bộ nhớ trong C#

- **Heap vs Stack:** Giải thích sự khác biệt giữa heap (bộ nhớ động) và stack (bộ nhớ tĩnh). Các đối tượng được tạo ra trên heap, trong khi các giá trị cơ bản (như int, bool) được lưu trữ trên stack.
- **GC (Garbage Collection):** Cách Garbage Collector hoạt động trong C# và ảnh hưởng của nó đến hiệu suất của ứng dụng.
- **Reference Types vs Value Types:** So sánh sự khác biệt về cách thức lưu trữ các kiểu tham chiếu và kiểu giá trị trong bộ nhớ.

2. Quản lý bộ nhớ và tài nguyên

- **Dispose Pattern:** Sử dụng `IDisposable` và `Dispose` để giải phóng tài nguyên không còn sử dụng.
- **Weak References:** Giới thiệu về Weak References và cách sử dụng chúng để tránh giữ các đối tượng trong bộ nhớ khi không còn cần thiết.
- **Memory Pools:** Sử dụng các bộ nhớ tái sử dụng như `ArrayPool<T>` và `MemoryPool<T>` để giảm thiểu việc cấp phát và giải phóng bộ nhớ.

3. Tối ưu hóa bộ nhớ

- **Giảm thiểu việc cấp phát bộ nhớ thường xuyên:** Các kỹ thuật để giảm thiểu việc cấp phát bộ nhớ (memory allocations), ví dụ như tránh việc tạo các đối tượng không cần thiết trong vòng lặp.
- **Sử dụng Structs thay vì Classes:** Cách dùng structs để tiết kiệm bộ nhớ, vì structs lưu trữ trực tiếp trong stack thay vì heap.
- **String Interning:** Sử dụng kỹ thuật string interning để tránh việc tạo ra nhiều bản sao của cùng một chuỗi trong bộ nhớ.

4. Tối ưu hóa hiệu suất

- **Parallelism và Concurrency:** Tối ưu hóa bằng cách sử dụng `Task Parallel Library (TPL)`, `async/await` để cải thiện hiệu suất trong các tác vụ bất đồng bộ và đa luồng.
- **Data Locality và Caching:** Sử dụng caching và tối ưu hóa việc truy xuất dữ liệu để giảm độ trễ và tăng hiệu suất ứng dụng.
- **Nguyên tắc SOLID và Clean Code:** Cách thiết kế mã nguồn sạch sẽ, dễ bảo trì và tối ưu hiệu suất với các nguyên tắc như SOLID, tránh lặp lại mã và bảo vệ tính tái sử dụng.

5. Sử dụng các công cụ phân tích hiệu suất

- **Profiling với Visual Studio:** Hướng dẫn sử dụng công cụ profiler của Visual Studio để theo dõi và phân tích hiệu suất ứng dụng.
- **BenchmarkDotNet:** Sử dụng thư viện BenchmarkDotNet để benchmark và so sánh các phương thức, thuật toán khác nhau về hiệu suất.
- **Memory Diagnostics:** Cách sử dụng các công cụ như `dotMemory` hoặc `Visual Studio Diagnostic Tools` để phát hiện các vấn đề về bộ nhớ trong ứng dụng.

6. Thực hành và ví dụ

- **Ví dụ thực tế:** Cung cấp các ví dụ cụ thể về cách áp dụng các kỹ thuật tối ưu bộ nhớ và hiệu suất trong các dự án C# thực tế.

- **Thực hành tối ưu hóa:** Cung cấp bài tập thực hành về tối ưu hóa bộ nhớ và hiệu suất cho học viên.

7. Kết luận

- Tóm tắt các kỹ thuật chính để tối ưu hóa bộ nhớ và hiệu suất.
- Khuyến khích học viên áp dụng những kỹ thuật này vào dự án thực tế của mình để cải thiện hiệu suất ứng dụng C#.

8.3. 10 thủ thuật tối ưu Code trong C#

1. Sử dụng `StringBuilder` thay cho việc nối chuỗi (+)

Khi cần ghép nhiều chuỗi, sử dụng `StringBuilder` giúp giảm thiểu việc cấp phát bộ nhớ mới cho từng chuỗi.

Không tối ưu:

```
string result = "";
for (int i = 0; i < 1000; i++)
{
    result += i.ToString(); // Tạo mới đối tượng chuỗi ở mỗi lần nối
}
```

Tối ưu:

```
using System.Text;

var stringBuilder = new StringBuilder();
for (int i = 0; i < 1000; i++)
{
    stringBuilder.Append(i);
}
string result = stringBuilder.ToString(); // Ít cấp phát bộ nhớ hơn
```

2. Sử dụng `readonly` khi có thể

Biến được khai báo `readonly` giúp tối ưu hóa hiệu suất bằng cách đảm bảo rằng giá trị không bị thay đổi, đặc biệt khi làm việc với struct.

Không tối ưu:

```
struct Point
{
    public int X;
    public int Y;
```

```
}
```

Tối ưu:

```
readonly struct Point
{
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

3. Tránh sử dụng LINQ khi hiệu năng là ưu tiên

LINQ dễ đọc nhưng có thể gây tốn bộ nhớ và thời gian nếu không sử dụng cẩn thận.

Không tối ưu:

```
var sum = numbers.Where(x => x % 2 == 0).Sum(); // Tạo nhiều đối tượng trung gian
```

Tối ưu:

```
int sum = 0;
foreach (var number in numbers)
{
    if (number % 2 == 0)
    {
        sum += number;
    }
}
```

4. Sử dụng ArrayPool<T> thay cho việc cấp phát mảng liên tục

ArrayPool<T> giúp tái sử dụng mảng thay vì tạo mới liên tục.

Không tối ưu:

```
byte[] buffer = new byte[1024];
// Sử dụng buffer và hủy sau đó
```

Tối ưu:

```
var pool = System.Buffers.ArrayPool<byte>.Shared;
```

```
byte[] buffer = pool.Rent(1024); // Thuê mảng từ pool
try
{
    // Sử dụng buffer
}
finally
{
    pool.Return(buffer); // Trả lại mảng về pool
}
```

Ví dụ chi tiết hơn:

Giả sử bạn đọc dữ liệu từ một tệp lớn và cần xử lý từng phần của dữ liệu trong bộ đệm.

Không sử dụng `ArrayPool<T>`:

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        const int bufferSize = 1024;

        using var fileStream = new FileStream("largefile.txt", FileMode.Open,
        FileAccess.Read);
        while (true)
        {
            byte[] buffer = new byte[bufferSize]; // Tạo mới mảng buffer
            int bytesRead = fileStream.Read(buffer, 0, buffer.Length);

            if (bytesRead == 0) break; // Đọc xong
            ProcessData(buffer, bytesRead);
        }

        static void ProcessData(byte[] buffer, int length)
        {
            // Xử lý dữ liệu trong buffer
            Console.WriteLine($"Processed {length} bytes.");
        }
    }
}
```

Sử dụng `ArrayPool<T>`:

```
using System;
using System.Buffers;
using System.IO;

class Program
{
    static void Main()
    {
        const int bufferSize = 1024;

        // Lấy ArrayPool instance (mặc định)
        var arrayPool = ArrayPool<byte>.Shared;
```

```
// Thuê một buffer từ ArrayPool
byte[] buffer = arrayPool.Rent(bufferSize);

try
{
    using var fileStream = new FileStream("largefile.txt", FileMode.Open,
    FileAccess.Read);
    while (true)
    {
        int bytesRead = fileStream.Read(buffer, 0, buffer.Length);

        if (bytesRead == 0) break; // Đọc xong
        ProcessData(buffer, bytesRead);
    }
}
finally
{
    // Trả lại buffer cho ArrayPool
    arrayPool.Return(buffer);
}

static void ProcessData(byte[] buffer, int length)
{
    // Xử lý dữ liệu trong buffer
    Console.WriteLine($"Processed {length} bytes.");
}
}
```

So sánh:

1. **Không sử dụng ArrayPool<T>:**

- o Mỗi lần đọc dữ liệu, một mảng mới được tạo trong bộ nhớ (new byte[bufferSize]).
- o Khi không còn sử dụng, mảng sẽ bị GC thu hồi, gây áp lực lên bộ thu gom rác (GC).

2. **Sử dụng ArrayPool<T>:**

- o Tái sử dụng mảng từ ArrayPool<T> mà không cần liên tục tạo và hủy.
- o Giảm áp lực lên bộ thu gom rác (GC), tăng hiệu suất trong các ứng dụng xử lý dữ liệu lớn hoặc yêu cầu hiệu suất cao.

Lưu ý:

- **Rent:** Thuê một mảng từ ArrayPool<T>. Nếu mảng sẵn có nhỏ hơn kích thước yêu cầu, một mảng mới sẽ được tạo.
- **Return:** Trả lại mảng cho ArrayPool<T> sau khi sử dụng. Đảm bảo không giữ tham chiếu hoặc sử dụng mảng sau khi trả lại.
- **Tùy chọn clearArray khi Return:** Bạn có thể gọi arrayPool.Return(buffer, clearArray: true) để xóa dữ liệu nhạy cảm trong mảng trước khi trả lại.

5. Sử dụng `Span<T>` để xử lý dữ liệu không cần cấp phát mới

`Span<T>` giảm thiểu việc cấp phát bộ nhớ mới khi thao tác với các phần của mảng hoặc chuỗi.

Không tối ưu:

```
string input = "Hello, World!";  
string substring = input.Substring(7); // Tạo chuỗi mới
```

Tối ưu:

```
string input = "Hello, World!";  
ReadOnlySpan<char> substring = input.AsSpan(7); // Không tạo đối tượng mới
```

6. Sử dụng `ValueTask` thay cho `Task` trong trường hợp đơn giản

`ValueTask` tối ưu hơn `Task` khi không cần tạo đối tượng `Task` mới.

Không tối ưu:

```
public async Task<int> GetNumberAsync()  
{  
    return 42; // Tạo một Task không cần thiết  
}
```

Tối ưu:

```
public ValueTask<int> GetNumberAsync()  
{  
    return new ValueTask<int>(42); // Không cần tạo Task mới  
}
```

7. Sử dụng `Dictionary` thay vì `List` khi tìm kiếm

Sử dụng `Dictionary` khi cần tìm kiếm dữ liệu nhiều lần, thay vì duyệt qua danh sách.

Không tối ưu:

```
var list = new List<int> { 1, 2, 3, 4, 5 };  
bool exists = list.Contains(3); // Độ phức tạp O(n)
```

Tối ưu:


```
var dictionary = new Dictionary<int, bool> { { 1, true }, { 2, true }, { 3, true },  
{ 4, true }, { 5, true } };  
bool exists = dictionary.ContainsKey(3); // Độ phức tạp O(1)
```

8. Sử dụng `ConfigureAwait(false)` trong các ứng dụng không phải giao diện

Giúp giảm chi phí khi không cần quay lại thread UI.

Không tối ưu:

```
await SomeAsyncOperation();
```

Tối ưu:

```
await SomeAsyncOperation().ConfigureAwait(false);
```

9. Giảm kích thước struct

Struct lớn có thể gây tốn kém khi truyền qua tham số hoặc trả về. Tối ưu bằng cách giữ struct nhỏ gọn.

Không tối ưu:

```
struct LargeStruct  
{  
    public int A, B, C, D, E, F, G, H;  
}
```

Tối ưu:

```
struct SmallStruct  
{  
    public int A;  
    public int B;  
}
```

10. Sử dụng `Parallel.For` để xử lý song song

Giúp tận dụng đa lõi CPU cho các tác vụ lặp lớn.

Không tối ưu:

```
for (int i = 0; i < 1000; i++)  
{  
    ProcessItem(i);  
}
```

Tối ưu:

```
Parallel.For(0, 1000, i =>  
{  
    ProcessItem(i);  
});
```

8.4. Best practices for clean, maintainable code.

1. Tuân thủ nguyên tắc SOLID

- **S:** Single Responsibility Principle (Nguyên tắc trách nhiệm duy nhất)
 - Mỗi lớp chỉ nên có một lý do để thay đổi. Điều này giúp giảm độ phức tạp và tăng khả năng tái sử dụng.
- **O:** Open/Closed Principle (Nguyên tắc mở/đóng)
 - Các lớp nên được mở rộng mà không thay đổi mã nguồn hiện tại. Điều này có thể đạt được qua kế thừa hoặc sử dụng interface.
- **L:** Liskov Substitution Principle (Nguyên tắc thay thế Liskov)
 - Các lớp con phải có thể thay thế các lớp cha mà không làm thay đổi tính đúng đắn của chương trình.
- **I:** Interface Segregation Principle (Nguyên tắc phân tách giao diện)
 - Không nên ép các lớp phải triển khai những phương thức mà họ không sử dụng. Cần chia nhỏ giao diện thành các phần nhỏ hơn.
- **D:** Dependency Inversion Principle (Nguyên tắc đảo ngược phụ thuộc)
 - Các module cấp cao không nên phụ thuộc vào module cấp thấp, mà cả hai nên phụ thuộc vào các abstraction (interface).

2. Tên biến và phương thức rõ ràng, dễ hiểu

- Sử dụng tên biến và phương thức dễ hiểu và mô tả đúng chức năng.
- Tránh sử dụng tên viết tắt hoặc tên không rõ ràng.
- Đặt tên cho các lớp, phương thức, và biến theo các quy ước chuẩn trong C# (PascalCase cho class, method; camelCase cho biến).

3. Tách biệt logic và UI

- Tách biệt mã logic và giao diện người dùng (UI) để dễ dàng kiểm thử và bảo trì mã nguồn.

- Áp dụng các mẫu thiết kế như MVC (Model-View-Controller) để phân chia công việc rõ ràng.

4. Không viết mã lặp lại (DRY - Don't Repeat Yourself)

- Tránh việc sao chép và dán mã lặp đi lặp lại. Thay vào đó, hãy tạo các phương thức hoặc lớp tái sử dụng.
- Sử dụng kế thừa hoặc các phương thức mở rộng (extension methods) khi cần.

5. Sử dụng Dependency Injection (DI)

- Áp dụng Dependency Injection để quản lý phụ thuộc giữa các lớp, giúp mã dễ kiểm thử và dễ bảo trì hơn.
- DI giúp giảm thiểu sự phụ thuộc chặt chẽ giữa các lớp và dễ dàng thay đổi các phần của ứng dụng mà không ảnh hưởng đến các phần khác.

6. Viết mã dễ kiểm thử (Testable Code)

- Thiết kế mã sao cho dễ dàng viết unit test.
- Tách biệt logic phức tạp thành các hàm nhỏ để kiểm thử dễ dàng hơn.
- Sử dụng mock objects và các framework như NUnit hoặc xUnit để kiểm thử.

7. Thực hiện xử lý lỗi hợp lý

- Sử dụng các cơ chế xử lý lỗi như try-catch để quản lý ngoại lệ.
- Tránh việc xử lý ngoại lệ trong vòng lặp hoặc những chỗ có thể gây rối cho mã.
- Cung cấp thông tin lỗi rõ ràng khi có sự cố, giúp phát hiện và sửa lỗi dễ dàng hơn.

8. Giữ mã nguồn ngắn gọn và rõ ràng

- Mã không nên quá phức tạp. Nếu có thể, hãy viết mã đơn giản và dễ đọc.
- Tránh viết các hàm quá dài hoặc phức tạp, chia nhỏ chúng thành các phương thức con dễ hiểu.

9. Sử dụng các công cụ phân tích mã

- Sử dụng các công cụ phân tích mã tĩnh như SonarQube để phát hiện các vấn đề về chất lượng mã.
- Cải thiện mã nguồn qua các công cụ phân tích này để tuân thủ các quy chuẩn về mã sạch và dễ bảo trì.

10. Tuân thủ các chuẩn và hướng dẫn lập trình

- Lập trình theo các chuẩn của C# như mã hóa đúng kiểu dữ liệu, sử dụng các lớp chuẩn của C# để giảm thiểu mã tự viết lại.
- Tuân thủ các hướng dẫn lập trình tốt như viết comment khi cần thiết, và sử dụng các chuẩn mã hóa như indentation hợp lý.

11. Refactor định kỳ

- Refactor mã nguồn để làm cho mã dễ đọc và dễ duy trì hơn theo thời gian.
- Đánh giá lại thiết kế của mã khi có sự thay đổi hoặc mở rộng tính năng.

12. Sử dụng các mô hình thiết kế phần mềm

- Áp dụng các mô hình thiết kế như Factory, Singleton, Observer hoặc Strategy để giúp mã trở nên linh hoạt hơn và dễ dàng mở rộng.

8.5. Clean Code in C#

1. Đặt tên biến, phương thức rõ ràng

Mã bẩn:

```
public int Calc(int a, int b)
{
    return a * b + 10;
}
```

- **Vấn đề:** Không rõ ràng `Calc` là gì, các tham số `a`, `b` không mang ý nghĩa cụ thể.

Mã sạch:

```
public int CalculateTotalPrice(int unitPrice, int quantity)
{
    const int tax = 10;
    return unitPrice * quantity + tax;
}
```

- **Cải tiến:** Tên phương thức và tham số rõ ràng, dễ hiểu hơn về mục đích.
-

2. Xử lý lỗi rõ ràng

Mã bản:

```
public void SaveFile(string path)
{
    File.WriteAllText(path, "Some data");
}
```

- **Vấn đề:** Không kiểm tra lỗi hoặc ngoại lệ khi tệp không tồn tại hoặc không thể ghi.

Mã sạch:

```
public void SaveFile(string path)
{
    try
    {
        File.WriteAllText(path, "Some data");
    }
    catch (UnauthorizedAccessException ex)
    {
        Console.WriteLine($"Access denied: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}
```

- **Cải tiến:** Xử lý ngoại lệ giúp chương trình an toàn hơn và cung cấp thông tin chi tiết khi có lỗi.

3. Tránh lặp lại mã (DRY - Don't Repeat Yourself)

Mã bản:

```
public double GetCircleArea(double radius)
{
    return 3.14159 * radius * radius;
}

public double GetSphereVolume(double radius)
{
    return (4 / 3.0) * 3.14159 * radius * radius * radius;
}
```

- **Vấn đề:** Hằng số `3.14159` được lặp lại, không dễ bảo trì nếu cần thay đổi.

Mã sạch:

```
public class MathUtilities
{
    private const double Pi = Math.PI;

    public double GetCircleArea(double radius)
    {
        return Pi * radius * radius;
    }

    public double GetSphereVolume(double radius)
    {
        return (4 / 3.0) * Pi * radius * radius * radius;
    }
}
```

- **Cải tiến:** Hằng số `Pi` được định nghĩa một lần, dễ bảo trì.
-

4. Sử dụng cấu trúc điều kiện gọn gàng

Mã bẩn:

```
if (user != null)
{
    if (user.IsActive)
    {
        return "Active";
    }
    else
    {
        return "Inactive";
    }
}
else
{
    return "No User";
}
```

- **Vấn đề:** Nhiều tầng lồng ghép, khó đọc.

Mã sạch:

```
if (user == null) return "No User";
return user.IsActive ? "Active" : "Inactive";
```

- **Cải tiến:** Logic đơn giản hơn, sử dụng biểu thức điều kiện.
-

5. Sử dụng Dependency Injection (DI)

Mã bẩn:

```
using System.Net.Mail;

public class EmailService
{
    private SmtpClient _smtpClient = new SmtpClient();

    public void SendEmail(string to, string message)
    {
        _smtpClient.Send("from@example.com", to, "Subject", message);
    }
}
```

- **Vấn đề:** Lớp `EmailService` phụ thuộc chặt chẽ vào `SmtpClient`, khó kiểm thử.

Mã sạch:

```
public interface IEmailClient
{
    void Send(string to, string message);
}

public class SmtpEmailClient : IEmailClient
{
    public void Send(string to, string message)
    {
        // Gửi email logic
    }
}

public class EmailService
{
    private readonly IEmailClient _emailClient;

    public EmailService(IEmailClient emailClient)
    {
        _emailClient = emailClient;
    }

    public void SendEmail(string to, string message)
    {
        _emailClient.Send(to, message);
    }
}
```

- **Cải tiến:** Sử dụng Dependency Injection, để kiểm thử và thay đổi thành phần phụ thuộc.
-

6. Sử dụng LINQ thay cho vòng lặp phức tạp

Mã bản:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
List<int> evenNumbers = new List<int>();
foreach (var number in numbers)
{
    if (number % 2 == 0)
    {
        evenNumbers.Add(number);
    }
}
```

- **Vấn đề:** Vòng lặp tường minh dài dòng.

Mã sạch:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
var evenNumbers = numbers.Where(n => n % 2 == 0).ToList();
```

- **Cải tiến:** Sử dụng LINQ giúp mã ngắn gọn, rõ ràng hơn.
-

7. Viết hàm đơn nhiệm

Mã bản:

```
public void ProcessData(List<int> numbers)
{
    // Lọc số chẵn
    var evenNumbers = new List<int>();
    foreach (var number in numbers)
    {
        if (number % 2 == 0) evenNumbers.Add(number);
    }
    // Tính tổng
    var sum = 0;
    foreach (var number in evenNumbers)
    {
        sum += number;
    }
    Console.WriteLine($"Sum: {sum}");
}
```


- **Vấn đề:** Hàm thực hiện quá nhiều nhiệm vụ.

Mã sạch:

```
public List<int> FilterEvenNumbers(List<int> numbers)
{
    return numbers.Where(n => n % 2 == 0).ToList();
}

public int CalculateSum(List<int> numbers)
{
    return numbers.Sum();
}

public void PrintSumOfEvenNumbers(List<int> numbers)
{
    var evenNumbers = FilterEvenNumbers(numbers);
    var sum = CalculateSum(evenNumbers);
    Console.WriteLine($"Sum: {sum}");
}
```

- **Cải tiến:** Chia nhỏ hàm thành từng nhiệm vụ cụ thể.

Kết luận

- **Mã bẩn:** Khó đọc, khó bảo trì, dễ gây lỗi.
- **Mã sạch:** Dễ hiểu, dễ bảo trì, dễ kiểm thử, và thân thiện với đội phát triển trong dài hạn.

Hãy luôn nhớ: *"Code được viết cho con người đọc, không phải chỉ cho máy tính hiểu!"*

9. Security in C# Development

9.1. Best Practices for Secure Coding in C#.

1. Giới thiệu về bảo mật trong lập trình

- Tầm quan trọng của bảo mật trong phát triển phần mềm.
- Những mối đe dọa phổ biến: SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), và bảo mật lưu trữ.
- Mục tiêu của việc lập trình bảo mật.

2. Input Validation and Sanitization

- **Kiểm tra và xác thực dữ liệu đầu vào:** Tránh SQL Injection, XSS bằng cách sử dụng phương pháp chuẩn.
- Sử dụng các phương thức của C# như **Regex** để kiểm tra định dạng dữ liệu.
- **Ví dụ:** Lọc dữ liệu nhập từ người dùng, như email, mật khẩu, số điện thoại, sử dụng các thư viện bảo mật (như **Microsoft.Security.Application**).

3. Sử dụng Prepared Statements và Parameterized Queries

- Giới thiệu về việc sử dụng **SqlCommand** với tham số thay vì chèn trực tiếp dữ liệu vào câu truy vấn SQL.
- **Ví dụ:** Sử dụng **SqlCommand** với các tham số an toàn thay vì **string.Format** hoặc nối chuỗi để tránh SQL Injection.

Cách làm:

```
string query = "SELECT * FROM Users WHERE Username = @username AND Password = @password";
SqlCommand cmd = new SqlCommand(query, connection);
cmd.Parameters.AddWithValue("@username", username);
cmd.Parameters.AddWithValue("@password", password);
```

4. Sử dụng Hashing và Salt cho Mật khẩu

- Giải thích tầm quan trọng của việc không lưu mật khẩu dưới dạng văn bản rõ ràng.
- Sử dụng **SHA-256** hoặc **bcrypt** để mã hóa mật khẩu.

Ví dụ:

```
using System.Text;

using (var sha256 = SHA256.Create())
{
    byte[] hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
    string hashedPassword = BitConverter.ToString(hashedBytes).Replace("-", "");
}
```

5. Sử dụng HTTPS thay vì HTTP

- Tại sao nên sử dụng HTTPS để bảo mật thông tin giữa client và server.
- Giới thiệu về SSL/TLS và cách cấu hình server web để sử dụng HTTPS.

Ví dụ: Kiểm tra nếu một kết nối là HTTPS trong C#:

```
if (Request.IsSecureConnection)
{
    // Xử lý an toàn
}
```

6. Tránh việc lưu trữ thông tin nhạy cảm một cách không an toàn

- Hướng dẫn cách xử lý thông tin nhạy cảm như mật khẩu, số thẻ tín dụng, v.v.
- Sử dụng mã hóa khi lưu trữ thông tin nhạy cảm trong cơ sở dữ liệu.

Ví dụ: Mã hóa thông tin nhạy cảm sử dụng `AesManaged`:

```
using System.Security.Cryptography;

using (AesManaged aesAlg = new AesManaged())
{
    aesAlg.Key = key;
    aesAlg.IV = iv;
    ICryptoTransform encryptor = aesAlg.CreateEncryptor(aesAlg.Key, aesAlg.IV);
    // Thực hiện mã hóa
}
```

7. Xử lý lỗi một cách an toàn

- Tránh tiết lộ thông tin nhạy cảm trong các thông báo lỗi.
- Sử dụng cơ chế log an toàn (ví dụ: `Serilog`, `NLog`), và tránh in stack trace hoặc thông tin về cấu hình hệ thống trong môi trường sản xuất.

Ví dụ: Lỗi an toàn:

```
try
{
    // Code thực thi
}
catch (Exception ex)
{
    Log.Error("An error occurred: {Message}", ex.Message);
}
```

8. Quản lý quyền truy cập (Authorization & Authentication)

Sử dụng xác thực và phân quyền người dùng hiệu quả (ví dụ: OAuth2, JWT).

Ví dụ: Cấu hình xác thực JWT trong ASP.NET Core:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```

```
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidIssuer = "yourIssuer",
        ValidAudience = "yourAudience"
    };
});
```

9. Code Review và Static Code Analysis

- Tầm quan trọng của việc kiểm tra mã nguồn và sử dụng các công cụ phân tích mã tĩnh để phát hiện các lỗ hổng bảo mật.
- Các công cụ như SonarQube, CodeQL có thể giúp kiểm tra lỗ hổng bảo mật trong mã C#.
- **Ví dụ:** Chạy phân tích mã tĩnh trong CI/CD pipeline để đảm bảo mã tuân thủ các best practices.

10. Security Testing và Penetration Testing

- Hướng dẫn thực hiện kiểm tra bảo mật ứng dụng bằng cách sử dụng công cụ tự động như OWASP ZAP hoặc Burp Suite.
- Tầm quan trọng của kiểm tra thâm nhập để phát hiện các lỗ hổng trước khi đưa vào môi trường sản xuất.

11. Tổng kết

- Những thách thức khi lập trình an toàn và cách cải thiện thói quen bảo mật trong đội ngũ phát triển.
- Các tài liệu tham khảo và nguồn lực để cải thiện kiến thức bảo mật trong lập trình C#.

9.2. Implementing Cryptography in C#.

Mục tiêu bài học:

- Hiểu các thuật toán mã hóa và giải mã trong C#.
- Làm quen với các lớp trong .NET Framework hỗ trợ mã hóa.
- Học cách triển khai các thuật toán mã hóa (AES, RSA) và các kỹ thuật bảo mật.

Nội dung bài giảng

1. Giới thiệu về Cryptography

- Khái niệm Cryptography.
- Các thuật toán mã hóa phổ biến: Symmetric (AES, DES), Asymmetric (RSA), Hashing (SHA).
- Sự khác biệt giữa mã hóa đối xứng và bất đối xứng.
- Các ứng dụng của Cryptography trong bảo mật: mã hóa dữ liệu, xác thực thông tin, chữ ký số.

2. Các lớp Cryptography trong .NET

- **System.Security.Cryptography**: Thư viện chính trong .NET hỗ trợ cryptography.
- Các lớp phổ biến:
 - **Aes**: Cung cấp các phương thức mã hóa và giải mã AES.
 - **RSA**: Cung cấp các phương thức mã hóa và giải mã RSA.
 - **SHA256**: Dùng để tạo các giá trị băm (hash).
 - **HMAC**: Tạo mã xác thực tin nhắn (HMAC).
- Sử dụng các lớp này trong mã hóa và giải mã.

3. Mã hóa đối xứng (Symmetric Encryption) với AES

- **AES** (Advanced Encryption Standard): Một trong những thuật toán mã hóa đối xứng mạnh mẽ nhất.
- Ví dụ:
 - Tạo khóa mã hóa và khởi tạo đối tượng **Aes**.
 - Mã hóa và giải mã dữ liệu sử dụng AES.
 - Quản lý khóa và IV (Initialization Vector) trong mã hóa AES.

```
using System;
using System.Security.Cryptography;
using System.Text;

class AesExample
{
    static void Main()
    {
        string plaintext = "This is a secret message!";
        string password = "my_secret_password";

        // Sử dụng AES để mã hóa và giải mã
        using (Aes aesAlg = Aes.Create())
        {
            aesAlg.Key = Encoding.UTF8.GetBytes(password.PadLeft(16,
'0').Substring(0, 16));
            aesAlg.IV = new byte[16]; // IV mặc định
        }
    }
}
```

```
// Mã hóa
ICryptoTransform encryptor = aesAlg.CreateEncryptor(aesAlg.Key,
aesAlg.IV);
byte[] encrypted =
PerformCryptography(Encoding.UTF8.GetBytes(plaintext), encryptor);

// Giải mã
ICryptoTransform decryptor = aesAlg.CreateDecryptor(aesAlg.Key,
aesAlg.IV);
byte[] decrypted = PerformCryptography(encrypted, decryptor);

Console.WriteLine("Encrypted: " + Convert.ToBase64String(encrypted));
Console.WriteLine("Decrypted: " + Encoding.UTF8.GetString(decrypted));
}
}

static byte[] PerformCryptography(byte[] data, ICryptoTransform cryptoTransform)
{
    using (var memoryStream = new System.IO.MemoryStream())
    using (var cryptoStream = new CryptoStream(memoryStream, cryptoTransform,
CryptoStreamMode.Write))
    {
        cryptoStream.Write(data, 0, data.Length);
        cryptoStream.FlushFinalBlock();
        return memoryStream.ToArray();
    }
}
}
```

4. Mã hóa bất đối xứng (Asymmetric Encryption) với RSA

- **RSA:** Thuật toán mã hóa bất đối xứng sử dụng một cặp khóa công khai và riêng tư.
- Ví dụ:
 - Tạo cặp khóa RSA.
 - Mã hóa và giải mã dữ liệu với RSA.

```
using System;
using System.Security.Cryptography;
using System.Text;

class RsaExample
{
    static void Main()
    {
        string plaintext = "This is a secret message!";
        using (RSA rsa = RSA.Create())
        {
            // Mã hóa
            byte[] encryptedData = rsa.Encrypt(Encoding.UTF8.GetBytes(plaintext),
RSAEncryptionPadding.OaepSHA256);

            // Giải mã
            byte[] decryptedData = rsa.Decrypt(encryptedData,
RSAEncryptionPadding.OaepSHA256);
        }
    }
}
```

```
        Console.WriteLine("Encrypted: " +  
Convert.ToBase64String(encryptedData));  
        Console.WriteLine("Decrypted: " +  
Encoding.UTF8.GetString(decryptedData));  
    }  
}
```

5. Hashing và HMAC

- **Hashing:** Sử dụng thuật toán băm như SHA256 để tạo ra các giá trị băm không thể phục hồi.
- **HMAC:** Hash-based Message Authentication Code, dùng để xác minh tính toàn vẹn của dữ liệu.
- Ví dụ:
 - Tạo giá trị băm SHA256 từ dữ liệu.
 - Sử dụng HMAC để tạo mã xác thực.

```
using System;  
using System.Security.Cryptography;  
using System.Text;  
  
class HashingExample  
{  
    static void Main()  
    {  
        string message = "This is a message to hash";  
  
        // SHA256 Hash  
        using (SHA256 sha256 = SHA256.Create())  
        {  
            byte[] hash = sha256.ComputeHash(Encoding.UTF8.GetBytes(message));  
            Console.WriteLine("SHA256 Hash: " +  
BitConverter.ToString(hash).Replace("-", "").ToLower());  
        }  
  
        // HMACSHA256  
        using (HMACSHA256 hmac = new  
HMACSHA256(Encoding.UTF8.GetBytes("secret_key")))  
        {  
            byte[] hmacHash = hmac.ComputeHash(Encoding.UTF8.GetBytes(message));  
            Console.WriteLine("HMACSHA256: " +  
BitConverter.ToString(hmacHash).Replace("-", "").ToLower());  
        }  
    }  
}
```

6. Quản lý khóa và bảo mật trong Cryptography

- Cách lưu trữ và bảo vệ khóa mã hóa.
- Sử dụng **Key Vault** trong Azure hoặc các giải pháp lưu trữ bảo mật khác.

7. Tổng kết và thực hành

- Tổng kết các thuật toán mã hóa và các lớp hỗ trợ trong .NET.
- Thực hành: Thực hiện một dự án nhỏ với các kỹ thuật mã hóa và giải mã đã học.

Kết luận

Bài giảng giúp người học hiểu cách triển khai các thuật toán mã hóa trong C#, sử dụng các lớp từ thư viện .NET để bảo vệ dữ liệu và đảm bảo tính bảo mật cho ứng dụng.

9.3. Secure Authentication and Authorization.

1. Giới thiệu về Authentication và Authorization

- **Authentication:** Quá trình xác thực người dùng (AI là người dùng).
- **Authorization:** Quá trình phân quyền cho người dùng (Người dùng này có quyền làm gì?).
- Sự khác biệt giữa Authentication và Authorization trong bảo mật ứng dụng.

2. Các phương pháp Authentication phổ biến trong C#

- **Forms Authentication:** Cách xác thực sử dụng biểu mẫu (thường cho các ứng dụng web ASP.NET).
- **Windows Authentication:** Xác thực qua tài khoản Windows (thường dùng trong các ứng dụng nội bộ).
- **JWT (JSON Web Token):** Xác thực và ủy quyền thông qua token trong các API RESTful.
- **OAuth 2.0:** Quy trình xác thực với dịch vụ bên ngoài (Google, Facebook, v.v.).

3. Xây dựng Secure Authentication trong C#

- **Hashing Passwords:** Sử dụng SHA-256 hoặc bcrypt để mã hóa mật khẩu.
- **Salt Passwords:** Thêm một chuỗi ngẫu nhiên vào mật khẩu trước khi hash để chống lại các cuộc tấn công rainbow table.
- **Sử dụng Identity Framework:** ASP.NET Identity cung cấp một hệ thống xác thực và phân quyền sẵn có.

4. JWT Authentication

- **Khái niệm về JWT:** Là một chuỗi chứa thông tin mã hóa, dùng để xác thực người dùng trong các hệ thống phân tán.

- **Tạo JWT Token:** Sử dụng thư viện `System.IdentityModel.Tokens.Jwt` để tạo và xác minh JWT trong C#.
- **Cấu trúc của một JWT:** Header, Payload, và Signature.
- **Bảo mật với JWT:** Sử dụng HTTPS để bảo vệ token và tránh để token bị rò rỉ.

5. Sử dụng OAuth 2.0 trong C#

- **Khái niệm OAuth 2.0:** Một phương thức xác thực cho phép truy cập vào các dịch vụ mà không cần chia sẻ mật khẩu.
- **Cấu hình OAuth với ASP.NET Core:** Cách cấu hình ứng dụng để sử dụng OAuth 2.0 cho xác thực.
- **Flow OAuth 2.0:** Authorization Code Flow, Implicit Flow, và Client Credentials Flow.

6. Authorization trong C#

- **Role-based Authorization:** Phân quyền dựa trên vai trò người dùng (Admin, User, Guest).
- **Claims-based Authorization:** Phân quyền dựa trên các claims (thông tin về người dùng như email, ID, v.v.).
- **Policy-based Authorization:** Tạo các chính sách để kiểm soát quyền truy cập phức tạp hơn.

7. Best Practices để Bảo vệ Authentication và Authorization

- **Sử dụng HTTPS:** Đảm bảo rằng tất cả giao tiếp đều được mã hóa.
- **Token Expiration:** Đặt thời gian sống cho token để hạn chế khả năng bị lợi dụng.
- **Refresh Tokens:** Cung cấp các token làm mới để gia hạn phiên đăng nhập mà không yêu cầu người dùng nhập lại mật khẩu.
- **Multi-factor Authentication (MFA):** Yêu cầu người dùng xác thực qua hai bước (ví dụ: mật khẩu + mã OTP).
- **Kiểm tra các lỗ hổng bảo mật:** Sử dụng các công cụ như OWASP ZAP để kiểm tra bảo mật ứng dụng.

8. Thực hành

- **Tạo ứng dụng sử dụng JWT Authentication:**
 - Xây dựng API sử dụng JWT để xác thực và phân quyền.
 - Sử dụng ASP.NET Core Identity để tạo tài khoản người dùng và đăng nhập.
- **Thực hiện OAuth 2.0 với Google hoặc Facebook:**
 - Cấu hình OAuth trong ứng dụng và cho phép người dùng đăng nhập bằng tài khoản Google/Facebook.

9. Kết luận

- Tóm tắt các phương pháp bảo mật trong Authentication và Authorization.
- Khuyến khích học viên tiếp tục khám phá các công nghệ bảo mật hiện đại như OAuth 2.0, OpenID Connect, và bảo mật API.

10. Interoperability and Integration

10.1. Consuming and Building RESTful APIs.

Mục tiêu bài giảng

- Hiểu rõ về các khái niệm cơ bản của RESTful API và HTTP methods.
 - Học cách tiêu thụ (consume) RESTful API trong C#.
 - Học cách xây dựng (build) RESTful API với ASP.NET Core.
-

1. Giới thiệu về RESTful API

REST (Representational State Transfer) là một kiến trúc cho việc xây dựng các dịch vụ web. Các API RESTful sử dụng các phương thức HTTP chuẩn như GET, POST, PUT, DELETE để tương tác với tài nguyên.

Đặc điểm của RESTful API:

- **Stateless:** Mỗi yêu cầu đều độc lập, không phụ thuộc vào yêu cầu trước đó.
 - **Sử dụng HTTP methods (GET, POST, PUT, DELETE).**
 - **Cấu trúc URL dễ hiểu và nhất quán.**
-

2. Các phương thức HTTP và ứng dụng trong RESTful API

- **GET:** Lấy dữ liệu từ server.
 - **POST:** Gửi dữ liệu đến server (thường dùng để tạo mới tài nguyên).
 - **PUT:** Cập nhật dữ liệu trên server.
 - **DELETE:** Xóa tài nguyên khỏi server.
-

3. Tiêu thụ RESTful API trong C#

Để tiêu thụ API RESTful trong C#, chúng ta có thể sử dụng lớp `HttpClient`. Dưới đây là một ví dụ cơ bản:

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

public class ApiClient
{
    private static readonly HttpClient client = new HttpClient();

    public static async Task Main(string[] args)
    {
        var response = await client.GetAsync("https://api.example.com/endpoint");
        if (response.IsSuccessStatusCode)
        {
            var content = await response.Content.ReadAsStringAsync();
            Console.WriteLine(content);
        }
        else
        {
            Console.WriteLine("Request failed.");
        }
    }
}
```

Giải thích:

- `HttpClient`: Lớp được dùng để gửi các yêu cầu HTTP.
 - `GetAsync`: Gửi yêu cầu GET đến API.
 - `IsSuccessStatusCode`: Kiểm tra xem yêu cầu có thành công không.
 - `ReadAsStringAsync`: Đọc nội dung phản hồi từ API.
-

4. Xử lý dữ liệu JSON trong C#

API RESTful thường trả về dữ liệu dưới dạng JSON. Để làm việc với JSON trong C#, chúng ta sử dụng thư viện `Newtonsoft.Json` hoặc `System.Text.Json`.

Ví dụ sử dụng `Newtonsoft.Json` để phân tích cú pháp JSON:

```
using Newtonsoft.Json;
using System.Net.Http;
using System.Threading.Tasks;

public class ApiClient
{

```

```
private static readonly HttpClient client = new HttpClient();

public static async Task Main(string[] args)
{
    var response = await client.GetAsync("https://api.example.com/endpoint");
    if (response.IsSuccessStatusCode)
    {
        var content = await response.Content.ReadAsStringAsync();
        var data = JsonConvert.DeserializeObject<MyData>(content);
        Console.WriteLine(data.Property);
    }
}

public class MyData
{
    public string Property { get; set; }
}
```

5. Xây dựng RESTful API với ASP.NET Core

Để xây dựng RESTful API trong C#, ASP.NET Core là công cụ mạnh mẽ. Dưới đây là các bước cơ bản:

- Tạo dự án ASP.NET Core Web API:**
 - Mở Visual Studio và chọn "ASP.NET Core Web API".
 - Lựa chọn phiên bản .NET 8 hoặc mới hơn.
- Tạo Controller:**
 - Trong thư mục **Controllers**, tạo một lớp mới kế thừa **ControllerBase**.

Ví dụ tạo API đơn giản:

```
using System.Runtime.InteropServices;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private static readonly List<Product> Products = new List<Product>
    {
        new Product { Id = 1, Name = "Laptop", Price = 1000 },
        new Product { Id = 2, Name = "Smartphone", Price = 500 }
    };

    [HttpGet]
    public IActionResult GetProducts()
    {
        return Ok(Products);
    }

    [HttpPost]
```

```
public IActionResult CreateProduct(Product product)
{
    Products.Add(product);
    return CreatedAtAction(nameof(GetProducts), new { id = product.Id },
product);
}

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Giải thích:

- `[ApiController]`: Thuộc tính chỉ ra rằng lớp này là controller API.
 - `[Route]`: Định nghĩa đường dẫn của API.
 - `[HttpGet]`: Phương thức xử lý yêu cầu GET.
 - `[HttpPost]`: Phương thức xử lý yêu cầu POST.
3. **Chạy ứng dụng:**
- Chạy ứng dụng và sử dụng Postman hoặc Swagger để thử nghiệm API.
-

6. Xử lý lỗi và trả về mã trạng thái HTTP

Cần xử lý các lỗi khi xây dựng API để đảm bảo tính ổn định. Sử dụng các mã trạng thái HTTP như 400 (Bad Request), 404 (Not Found), và 500 (Internal Server Error) là điều quan trọng.

Ví dụ xử lý lỗi:

```
[HttpGet("{id}")]
public IActionResult GetProduct(int id)
{
    var product = Products.FirstOrDefault(p => p.Id == id);
    if (product == null)
    {
        return NotFound();
    }
    return Ok(product);
}
```

7. Kết luận

- Tiêu thụ và xây dựng RESTful API trong C# là một kỹ năng quan trọng trong phát triển phần mềm hiện đại.
- C# cung cấp công cụ mạnh mẽ như `HttpClient` để tiêu thụ API và ASP.NET Core để xây dựng các API.
- Kiến thức về HTTP methods, xử lý JSON và mã trạng thái HTTP là cần thiết khi làm việc với RESTful API.

10.2. Working with gRPC.

Mục tiêu học:

- Hiểu và ứng dụng giao thức gRPC trong C# để xây dựng các dịch vụ vi mô (microservices).
-

I. Giới thiệu về gRPC

1. Khái niệm gRPC

- gRPC là một framework RPC (Remote Procedure Call) được phát triển bởi Google, sử dụng HTTP/2 và Protobuf (Protocol Buffers) để truyền tải dữ liệu.
- Làm việc nhanh hơn, giảm độ trễ so với REST API truyền thống.

2. Ưu điểm của gRPC

- Hiệu suất cao và hỗ trợ đa nền tảng.
- Tự động sinh mã client/server từ file `.proto`.
- Hỗ trợ streaming và call bất đồng bộ.

3. Các kiểu giao tiếp trong gRPC

- **Unary**: Client gửi 1 request và nhận 1 response.
- **Server streaming**: Client gửi 1 request và nhận nhiều response từ server.
- **Client streaming**: Client gửi nhiều request và nhận 1 response từ server.
- **Bidirectional streaming**: Client và server có thể gửi và nhận dữ liệu liên tục.

4. Cài đặt gRPC trong C#

- Tạo một dự án C# gRPC Server và Client.
- Cài đặt các NuGet packages: `Grpc.AspNetCore` cho server, `Grpc.Net.Client` cho client.

5. Định nghĩa Service với Protobuf

- File `.proto`: Định nghĩa các dịch vụ và messages.

Ví dụ:

```
syntax = "proto3";

service Greeter
{
    rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest
{
    string name = 1;
}

message HelloReply
{
    string message = 1;
}
```

6. Triển khai gRPC Server và Client trong C#

- Cấu hình gRPC server trong ASP.NET Core.
- Gọi gRPC từ C# client.

IV. Bài tập thực hành

1. **Bài tập 1:** Xây dựng một ứng dụng gRPC đơn giản với nhiều phương thức RPC.
-

V. Kết luận

- Hiểu rõ cách gRPC có thể giúp xây dựng các hệ thống vi mô và ứng dụng có hiệu suất cao.
- Khả năng triển khai giao tiếp bất đồng bộ và streaming giúp tăng tính linh hoạt và khả năng chịu tải của hệ thống.